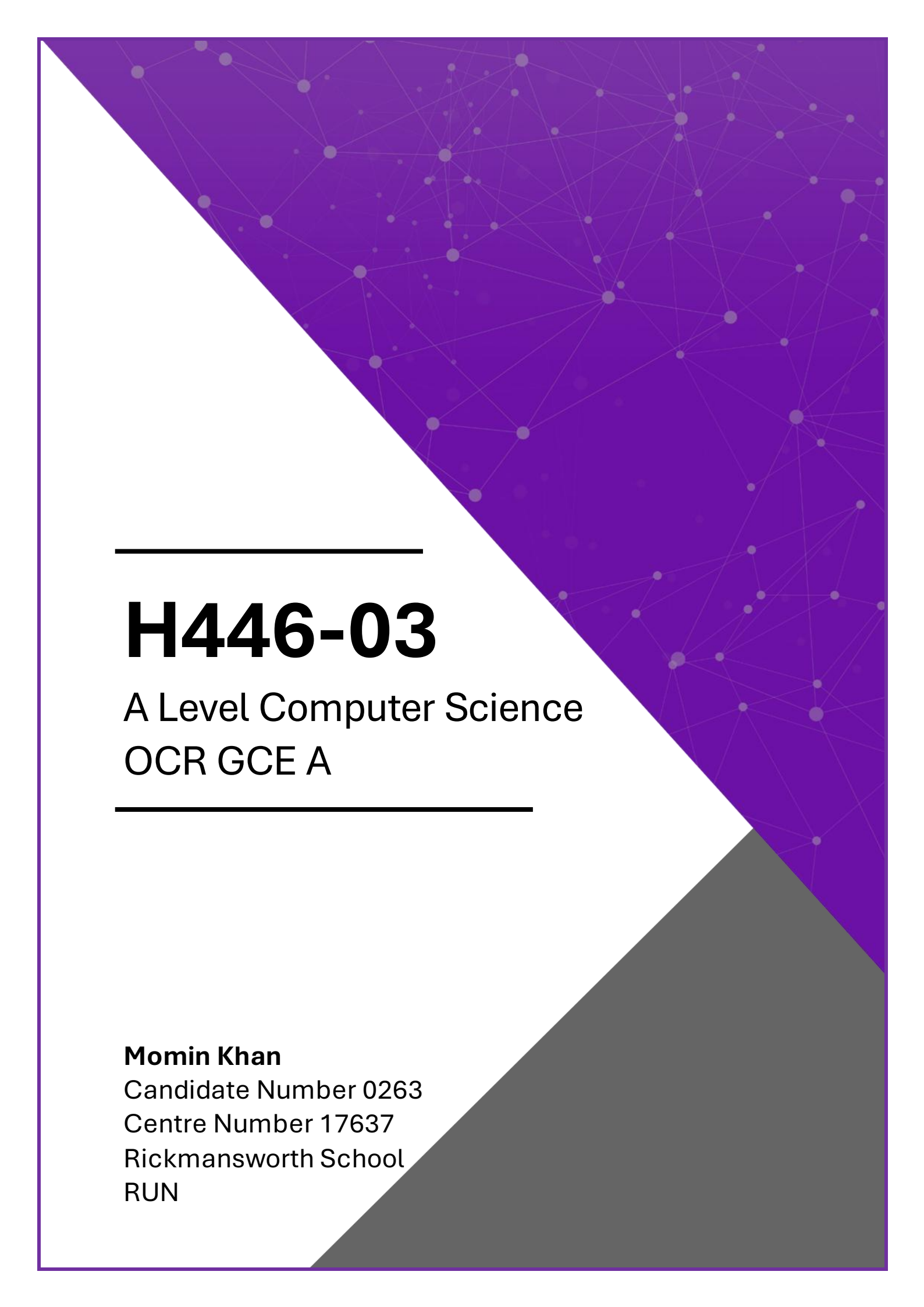




H446-03

A Level Computer Science
OCR GCE A

Momin Khan

Candidate Number 0263

Centre Number 17637

Rickmansworth School

RUN

Table of Contents

Proposal.....	5
Analysis.....	6
Research & Comparison	6
Rayman Legends.....	6
Information about the game	6
Gameplay Screenshots	6
Pros & Cons	8
Features I like from this game, and to include in mine	8
Super Mario Bros. (1985).....	9
Information about the game	9
Gameplay Screenshots	9
Pros & Cons	10
Features I like from this game, and to include in mine	11
Geometry Dash.....	11
Pros & Cons	13
Features I like from this game, and to include in mine	13
Celeste.....	14
Information about the game	14
Gameplay Screenshots	15
Pros and Cons	16
Feature I like from this game, and to include in mine.....	16
Stakeholders.....	16
Microsoft Form Results.....	17
Solving Problems with Computational Methods.....	18
Thinking Abstractly.....	18
Thinking Logically	19
Thinking Concurrently.....	19
Thinking Ahead	19
Thinking Procedurally	20
Hardware and Software Requirements.....	20
Limitations	21
Feature List	22
Success Criteria.....	24
Design.....	28
Intro Screen.....	29
Menu.....	29
Main Game.....	29

Collectibles	29
Levels	30
Movement.....	30
Scoring	30
Timer.....	31
Characters	31
Screen Designs.....	32
Interview	40
Other Resources	41
TileMap	41
Variable List.....	42
Test Table.....	45
Flowcharts.....	51
Start-up.....	51
Menu.....	52
Level Selector.....	53
Pause Menu.....	54
Settings Screen.....	55
Controls Screen.....	56
Audio Settings.....	57
Movement.....	58
Animations	59
Coins	60
Falling death	61
Development	62
Player.....	62
Idle Sprite	62
Game Scene	64
Basic Movement.....	64
Better Movement.....	66
Level Design.....	70
Foreground.....	72
Background.....	73
Interview	75
Title Screen	76
Settings Screen.....	79
Control Screen.....	81
Pause Menu	84

Player Animations.....	89
Climbing	89
Idle Climbing.....	90
Falling.....	90
Running.....	91
Jumping.....	91
Idle Crouch	92
Crouch Running.....	93
Crouch Jumping.....	94
Collectibles.....	97
Coins	97
Score	99
Killzone	102
Better Climbing.....	103
The problem	103
Wall climbing function	103
Wall jumping function	108
Level 2	109
Platforms.....	110
Transition between levels	112
Level Selector.....	114
Music.....	116
Audio Screen	121
Spikes	123
Dash	126
Dash Animation.....	133
Level 3	135
Parallax2D Background	135
Interview	138
Checkpoints.....	139
Game Progress	142
Game saving and loading.....	142
HUD.....	146
Exporting the project	148
Evaluation.....	150
Testing for the evaluation.....	150
Has the Success Criteria been met?.....	160
Unimplemented features	163

Usability Features	165
Title screen	165
Options screen	166
Level Selector screen	167
Level 1	168
Level 3	169
Maintenance & Limitations	170
Current and future maintenance	170
Limitations	170
Project Appendix	171
audio_screen.gd	171
checkpoints.gd	172
coin.gd	173
control_screen.gd	174
global.gd	174
hud.gd	176
killzone.gd	176
level_1.gd	177
level_2.gd	177
level_selector.gd	178
music_level_1.gd	179
music_level_2.gd	180
music_level_3.gd	181
pause_menu.gd	182
player.gd	185
settingsScreen.gd	192
spikes.gd	193
titleScreen.gd	193
Sources/References	193

Proposal

I am making a platform adventure game using the program Godot and will be writing in GDScript. This game is largely inspired by “Super Mario Bros. 64” and “Celeste”. It will have a basic movement of running, jumping and climbing. I will also include a dynamic background which will change by the panning camera which will be focused on the character's current positions.

Each level will have an entirely different landscape and background (of which there will be 3). The movement will include the basics, such as jumping, running and climbing. Players can move left, right, up and down on the platforms as it is a 2D game. There will also be areas that players must avoid, otherwise they will die, such as enemies or holes in the ground.

There will only be one playable character. The GUI will be small and undistracting, showing a timer, in which the player must complete the level before it ends, and the remaining number of lives before a game over screen is displayed.

I will retrieve some aspects of my game from online services that are non-copyrighted, and I will seamlessly incorporate them into the game. One example will be the background. The 3 levels will each have a simple background made by me. The character models will also be created, as well as the movement control and POV. I have done this to make the game more unique, but I have made sure not to sacrifice the quality of the art in the game especially with the time I have.

There will be no multiplayer options.

Analysis

Research & Comparison

Rayman Legends



Information about the game

Rayman Legends is a 2D platformer game which revolves around a main character, Rayman, and his friends (other playable characters) and their journey. While focusing on a painting that shows a medieval land, they are suddenly sucked into the painting, entering the world, and the adventure begins. The gang must run, jump and fight their way through each world to save the day and discover the secrets of every legendary painting.

Gameplay Screenshots



In this screenshot, we can see 4 characters running across a platform, with what seems to be a rockstar playing music, which is actually for the background music of the level itself. This creates a more inclusive atmosphere for the atmosphere as the rockstar follows the players along the level.

We can also see that the playable characters are all of different sizes, indicating that they will have different animations, which we can see from this screenshot. This will include killing animations, running, jumping, walking and taking animations.

As seen in the screenshot, there is a castle in the background. Now as the level is played, the background will move left and right (depending on the position of the character in the level), resulting in a dynamic parallax background. There is also a foreground which includes various small details (as well as the rockstar), but do create an immersion into the level.

The platforms, additionally, seem to be falling/crumbling, which is an example of dynamic platforms, where the platforms, in which the player interacts with, are constantly moving around, causing the player to have to actually concentrate while moving from one to the other.



In this screenshot, we see the main character (Rayman) attacking one of the many enemies that are included in the game (pink people). We can see the vivid visual effects that pop up between the characters as the enemy dies. We can also see that the foot of the player doesn't actually hit the enemy, instead just does the action. This is to both mute the violence of the game to retain the target audience, and also to be ignored by the developers so they don't have to write another algorithm to make sure contact between characters is made and clearly shown. This is an example of abstraction.

We can also see a gap in platforms, which resemble a cliff. This is another example of dynamic platforms, which I referred to in the previous screenshot. This creates a variety in obstacles/platforms. We can also observe the details on the platforms themselves. Again, we see a detailed background which will pan left and right depending on the players movements.

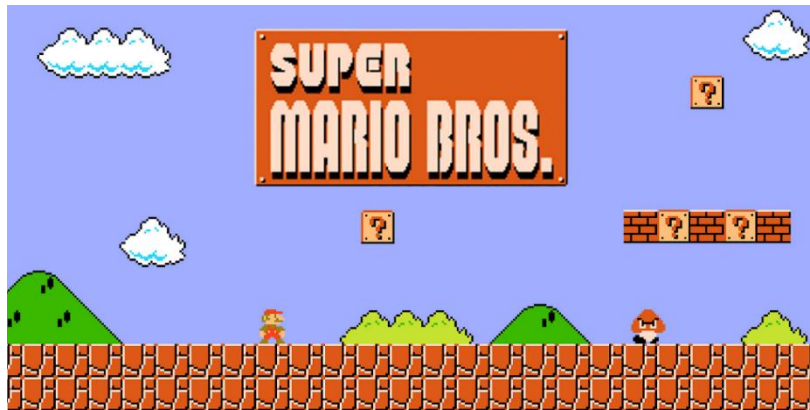
Pros & Cons

Pros	Cons
Easy to learn mechanics & controls	Very basic movement
Friendly for all users	May be too easy for players who want a real challenge out of the game
Dynamic platforms in the levels	Levels can become repetitive for an experienced player
Character-dependent animations (different animations play depending on the character chosen)	Basic sound effects
Child-friendly death animations for enemies (small implosion) and players (turns into a bubble)	No GUI
Background music which changes based on the position in the level	

Features I like from this game, and to include in mine

- The animations in this game really stand out to me. It's mostly the fluidity that the animations allow the gameplay to be that attract me and, I would assume, most players.
- The ability for the game to be violent and intense, but at the same time, child-friendly and casual is extremely appealing.
- The art style is clearly cartoony, and I like this 2D art style. I might include something that resembles this style in my game.

Super Mario Bros. (1985)



Information about the game

Super Mario Bros. was the first game of its kind when it was released in 1985, and it is considered the first platformer game. This game sparked the creation of the game series described as the best in the world; Mario.

It follows the well-known story of Princess Toadstool being kidnapped by King Bowser, and Mario and Luigi, depending on the game being played single or multiplayer, must strive to get her back.

It is one of the most successful games in the world, selling more than 40 million copies to this day.

Gameplay Screenshots



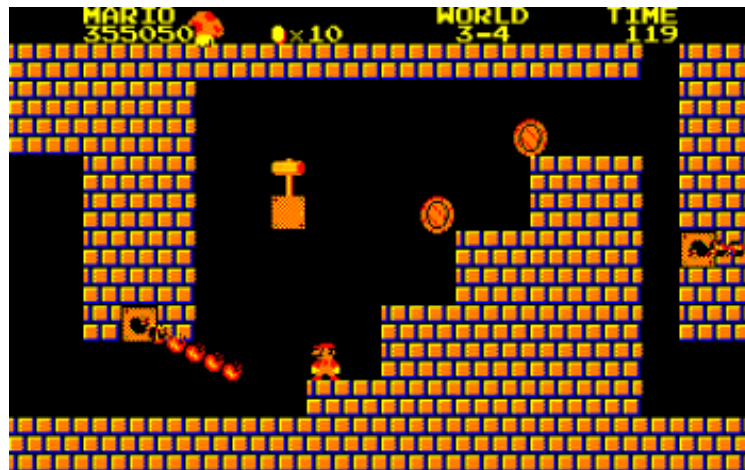
This screenshot shows many things that are included in the game.

It shows a variety in the types of obstacles, which is essential in a game to retain the players focus and concentration, as well as different enemies to defeat to get to the end of the level. Speaking of the end of the level, I like how it is clearly in view as the player reaches closer to the end. Background features can also be noticed in the form of, in

this screenshot, small trees, fences and clouds. This simply fills the empty space left by the lack of constant obstacles.

Collectibles are available for the player, and considering that this is such an old game, the mechanic of collectibles have exploded in the 38 years since this game was released. Collecting items could either give the player a power-up, resulting in a new ability, or contributing to the player's total score.

Speaking of total score, there is a scoring mechanism which is located in the main GUI. Additionally, enemies are in a variety as well, involving moving entities in the form of bullets, cannonballs etc.



As we can see in this screenshot and comparing it to the last, there is a large variety of the different types of platforms, like stairs, large drops and holes in the ceilings. This is the same for the obstacles, like moving walls of fire.

Again, we see the collectibles which can both contribute to the player score and sometimes give the player a power up as an advantage in the level.

Pros & Cons

Pros	Cons
Has the fundamentals of the platform game	Old design (outdated for 2023)
Basic but rather informative GUI	Basic GUI, looks too basic
Animated art (background, players, enemies)	Basic obstacles

Features I like from this game, and to include in mine

- This game is a main inspiration for me, because of the legacy that it holds.
- The platforms and course design are the main thing I will take from this game. They are simple but yet effective in giving the player a challenge out of the game.
- The enemies are also a very good aspect of the game. Their animations and designs are varied, as well as their method of attacks towards the player.
- The collectibles are the last feature of Mario that stand out to me. They are all different and all effective in giving the game more character and variation, from giving the player a power-up and XP, to giving the player a companion such as Yoshi.
- The graphics of the game are appealing, being simple yet effective in representing a natural but dream-like world.

Geometry Dash



Information about the game

Geometry Dash is a fully 2D platformer game, which contains multiple obstacle-ridden levels, which the player must strive to get through with the jumping mechanism. The game creates a sense of chaos throughout the game via sound effects, music which is relevant to the obstacles themselves and the automatic force that moves you forward, so all the user has to do is jump.

Gameplay Screenshots



Because all these games are platform games, they will share some attributes. In this screenshot, we see a collectible in the form of a large, golden coin, which can be used as a currency to purchase more avatars.

Avatars are available in the game, meaning the player can purchase and choose to customise their appearance, which is a popular feature in many successful games.

We can see that there are 3 paths to which the player is able to go through, showing the various types of platforms that are present in this game, but also you can notice the very loud icons (“!”, “->”) that guide the player to the right path.



Again, we can see the variety of obstacles throughout the level, from the spikes on the left, to the floating orbs you can bounce off in the middle, to the normal block on the right.

We can again see the different avatars that are available for the player.

Throughout this screenshot which represents the entirety of the level, a central theme is made obvious, which is a more futuristic and geometric style. We can see this, as well as the background, but also the obstacles themselves. For example, there are

geometric shapes everywhere, both circular and squared, such as the floating orbs with a thick outline. The spikes also have an under glow to them. We can also see a trail left by the avatar as it is jumping between the floating orbs. This is simply another unique detail which contributes to the overall art style.



This image doesn't really show us anything new, except from the portal. The portal is used to teleport the player from ground jumping to a list of many new environments, from flying, jumping on air or rolling.

But it does show us the movement animations in proportion to the avatar itself. As the player is jumping, the orientation of the avatar switches from right side up to upside down.

Pros & Cons

Pros	Cons
Enticing art	Extremely basic movement (only jumping)
Various types of areas	Can easily become too difficult for some users
Various types of obstacles	No GUI

Features I like from this game, and to include in mine

- The various types of areas are a very good idea, as it opens many doors for challenging modes, as well as increases the limit for the number of ideas I can put into each level.
- The obstacles are also very well designed, as they can look menacing to the player

- The collectibles are desirable to remake, as they can give the player an extra goal, apart from simply completing the level.

Celeste



Information about the game

Celeste is a 2D platformer game with a thrilling story. It is basic but effective; detailing the point of view of a girl who wants to climb a mountain, and shows all the obstacles she must pass to do so; from meeting friends, to getting trapped in a hotel, to meeting herself; this is one of the best platformers I have ever played. The gameplay also does not disappoint. It contains climbing mechanisms, dashes and sprinting. Rather basic, yes, but it is effective in use.

Gameplay Screenshots



This shows the main character of the game (Madeline) in a platform level. We can see many features that make this game stand out to others, like the scenery in the background, the different materials in the platforms and obstacles, the plants coming out the ground and especially the pixelated art style of the entire game.



We can again see the protagonist, now isolated in a lush cave. I will point out again how well this game uses the art style to present an accurate but vibrant and saturated. In both the foreground and background, we can see waterfalls, providing some layers to the game and therefore making the game feel fuller. The obstacles also vary in terms of the manner of them; you are able to swim below the normal platform as seen in the screenshot above. This is a puzzle game, so they will be included, but not always as expected.

Puzzles can be obvious, from getting a key to unlock a door, to simply finding a way to get past a solid wall.

We can see an example of abstraction here, as the developers have ignored the detailing on the inner caving (look at the top of the screen) and remained focus on the part that the character will actually interact with.

Pros and Cons

Pros	Cons
Enticing art style	
Vibrant colours	
Immersion into the game without GUI	

Feature I like from this game, and to include in mine

The pixelated art is something I will definitely be incorporating into my game

The dash function in the game is a new and exciting way to complicate the movement and make it deeper.

Stakeholders

Ismail Ahmed - Has played a large variety of 3D games, parkour games included, and will provide fresh insight and helpful feedback to the development of the game. The game is appropriate for him because he is a massive fan of video games and plays them very regularly. This project is suitable for his needs because he games a lot both for fun and to take a break from his A-Levels.

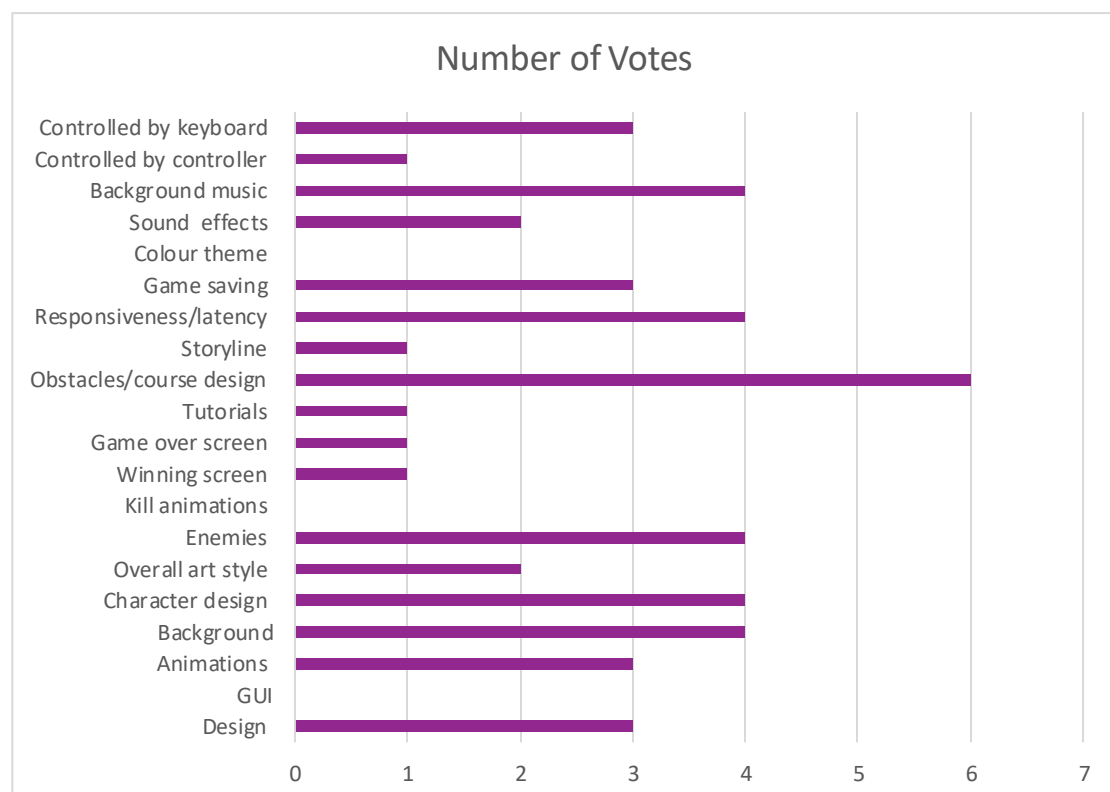
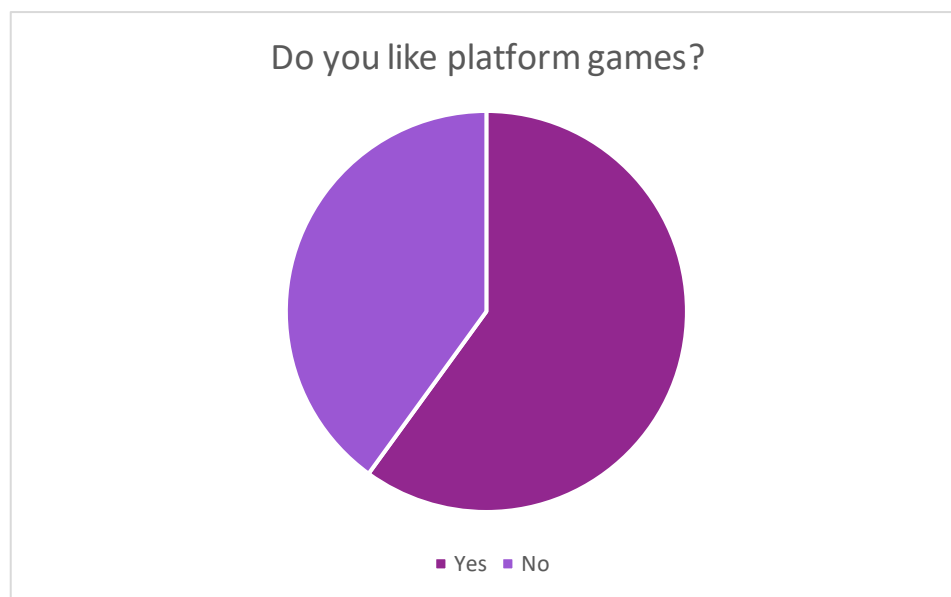
Shazil Ghauri - Has played 2D mobile games, console games and PC games, and will provide relevant feedback and suggestions to keep my game relevant to the target audience. This project is suitable for his needs because he plays an abundance of games in his free time, so he is well developed in video games. He also is an avid coder, so he will be very helpful with feedback and advice for the development process.

Adham Hegazy - Has played a short range of games, and will provide a new sense of direction that none of the other stakeholders may have. He will also test if the gameplay is easy to pick up and get a hang of, since he is not experienced in playing video games. This project is suitable because it will be one of the first 2D games that he plays, so this might convince him to start playing more video games.

All of these people are useful to my project as they study the computing subject at A-Level, and they also play games regularly (except from Adham), so I believe that their advice will be extremely useful to me.

Microsoft Form Results

I created and gave my stakeholders and my class a Microsoft Form to fill out, which involved a few brief questions asking for the basic features of the average platform game.



The first pie chart is to ensure that the majority of people answering the form have knowledge of platform games, and therefore can give relevant and useful opinions/responses.

I will use the bar graph above to know what to focus on the most in terms of quality and what to somewhat ignore. Examples of what to focus on include mainly obstacles and the course design of each level. And the colour theme of the levels is clearly something to not focus on too much.

Solving Problems with Computational Methods

The main problem I am trying to solve with this game project is the boredom that some people feel in their free time. Many people find themselves without a beneficial task that they look forward to. This is the main objective of my game; to provide an activity to do and look forward to, without the unnecessary hardships of another more difficult game. Mine will be laid back, easily to play and just enjoyable. The reason why its suitable for computational methods is I'm creating a game with lots of digital artefacts that gets harder the longer the payer survives. This isn't the case for a board game. I will think abstractly, concurrently, logically, and I will also think ahead moving forward to create the solution for this problem.

Thinking Abstractly

Before starting the development stage, I need to figure out what features to include and what to ignore, so I waste as little time as possible on unnecessary details. I can have dozens of different sprite states as the player is moving around the screen, but it will take up a lot of processor power and it will take too much time to design, so I'll use simpler sprites and designs. With the help of my stakeholder form, I can use abstraction to help me make this choice.

I will keep the design of the game itself simple, while making it pleasing and appealing to the player by:

- » Not including anything related to 3D animation to retain simplicity
- » Not including an overly detailed GUI
- » Not having a storyline
- » Deciding that kill animations are irrelevant
- » Not focusing on the game over or winning screens too much
- » Not spending too much time on the visual design on the levels and the art

I will also be including icons that are colour coded in the game that I will ensure look natural and not out-of-place, as well as highlighting things of interest for the player to easily find. These are examples of uses of abstraction that I will use because I am bringing focus onto relevant features in my program.

Thinking Logically

Before I start coding, I can think of the logistics of my game. Then I can make sure I know what everything does, mechanic wise before I start actually making it. I can make sure I know what everything will do before I start coding at all.

For example, I know already that when the player presses Esc at any part of the level, the menu will cover the entire screen and present the player with the menu choices. For when they are in the menu, different buttons can do different things. For example, the return to main menu button will present the start menu to the player's screen, and pressing quit game will close the game entirely.

The start button will open a separate mini-menu which holds the choices to select any of the 3 levels to play (once they have been unlocked). Each choice will summon a different function, which either start the game from the first level, or summons a function for each level that the player chose.

The order of things will also be important when loading up the game. For example, the first thing to come up on the player's screen will be the Godot logo (because it is made in Godot). Then the start-up screen will come up, with 3 options: START, OPTIONS & QUIT GAME. The START and OPTIONS buttons, when pressed, will lead to a separate mini-menu with some more options, from control mappings to level selections. Logical conditions are very important. They can lead to the program branching into a loop which will keep looping until that condition is fulfilled. An example is if the player has not reached the ground, the player will not be able to jump again until it is on the floor. The logical condition here is checking whether or not the player is on the floor or not.

Thinking Concurrently

In a game, there are lots of things happening simultaneously, like movement mechanics (strafing – moving diagonally in a direction) that require more than 1 key to be pressed at once. This is why I need to think concurrently to fix this problem and optimise my code to work simultaneously.

Another example is when sound effects are being played, the background music keeps playing in the back.

Music playing at the same time as the character and enemies are moving.

The program must be able to update constantly to keep track of lots of things, such as timers, experience points and more.

My `_physics_process` function will have to cater for all of these events

Thinking Ahead

Thinking ahead will allow me to lose less time when I approach new problems, because I have thought about them already and how to solve them ahead of time.

I know that the controls of my game need to be known, so the player doesn't need to spend a lot of times, because if they do, they might get drawn away from the game. So, I'll be using the standard control layout of WASD for basic movement, space for jumping, and holding SHIFT to sprint and holding S to crouch

I am wondering if there are going to be any problems or bugs with holding down SHIFT to sprint. I have research for some code to make sure it is possible to continually detect if SHIFT is being held down or not, and run a function to carry out an action depending on that result.

I need to think about how long the game is going to be, as well as how I'm going to save the game, to retain the players' progress

I can make the development process easier for myself later on by thinking ahead and figuring out what the existing preconditions are for the program. Examples could be how many levels there are going to be etc. I also need to think about the inputs and outputs of the game, so that they are easier to figure out and code later. The inputs of my game are the keyboard controls being pressed, and the outputs of the game are the sound effects and the screen being updated to show gameplay.

I can also think about what libraries I can use. I am using Godot for my game, which already supplies me with a good number of libraries I can use in order to save time on development. I will use libraries for the movement controls and the gravity value.

Thinking Procedurally

I'll need to decompose the problem into bite sized pieces this will make it easier to solve the overall problem. I need my program to be made of modules, containing functions which are relevant to parts of the problem. This will allow me to work on each module individually, which would mean that I can work on increasing the quality of each module.

For example, I could break the game into 2 major chunks:

- Menu State
- Playing State

Hardware and Software Requirements

The game is being created inside the Godot program, so the hardware requirements for the game are only the hardware requirements for Godot (this is information that was retrieved from the official Godot website, meaning:

Minimum Requirements	Windows	macOS	Linux
OS version	7+	10.13 or 10.15+	Linux distribution released after 2016
CPU	x86_32 CPU with SSE2 instructions, x86_64 CPU, ARMv8 CPU	x86_64 or ARM CPU (Apple Silicon)	x86_32 CPU with SSE2 instructions, x86_64 CPU, ARMv7 or ARMv8 CPU

RAM	4GB	4GB	4GB
GPU	NVIDIA GeForce 6200	NVIDIA GeForce 6200	OpenGL 3.2+ or Vulkan-capable, Nvidia and AMD GPUs
Storage	200MB	200MB	200MB

But the general hardware requirements will be:

Feature	Reasoning
Monitor	A method of output to see the game itself
Keyboard	A method of input to control the game using different keys and buttons
Mouse	A method of input to control the game with the movements of the mouse
Speaker	A method of output to hear the sounds of the game

Limitations

Limitation	Justification
Platforms	Godot is able to run on all desktop OSs, like macOS, Windows and Linux, so my game will be available for all of those platforms. But the game will only be on desktop, because the screen size has to be optimised, as well as the controls on different platforms, like a console or phone, will be too different. So, because of this, the game will be limited to a desktop or laptop. A phone will have a touchscreen, and I am writing my code for a keyboard. The game requires fast movements, which won't be available on a phone.
Multiplayer	This is an unnecessary detail to include, especially in the timeframe that I have. Another reason is my stakeholders have opted against a multiplayer game. Additionally, single player games, on average, are more likely to be successful. This also means that I can spend more

	time focusing on the quality of the game instead of the quantity of features
Levels	There will only be 3 levels, due to the timescale that I am provided with, but there is no reason for the game to be updated later on with more features, levels being one of them
Time	I don't have a lot of time to make this project; less than 2 years, so not all of my and my stakeholders' ideas will be implemented. But again, there is no reason for the game to be updated in the future with these features.

Feature List

Feature	Justification
1. Score	There will be no score box that remains on the screen, but the score of each individual collectible will be shown on the screen (for example, when the player collects a coin, there will be "+125 XP") and the total score will be revealed on the screen at the end of the level. I want to include this because it can keep track of the player's skill in some instances, as well as create a new goal for the players to reach the maximum XP in the level, for example.
2. Single – player	Single player games are easier to play, as there is no pressure to play as well in a community. They can also allow for the player to get immersed in the gameplay. Also from a programmer's perspective, I don't have the knowledge to incorporate multiplayer into my game, nor do I have the time to learn it
3. The game will be colourful, natural but rather realistic	I want the game to be vibrant but I don't want it to be entirely magical because it is an unoriginal theme. On the other hand, I want the player to look at the game and be interested and that is impossible if it looks too realistic and bland, especially because the game itself is not based off of real life

4. The graphics will be cartoony and pixelated	To prevent the act of including unnecessary detailing in the art, but at the same time look appealing
5. There will be sound effects that are relevant to each feature that they are used for (enemies, collectibles etc.)	To provide a sound reward for the task completed. For example, a shiny sound effect with a glint noise will be played when a coin is collected, and another will be played when a power-up is collected, or an enemy killed.
6. An ending screen	This is to reward the player and provide them with a sense of accomplishment. This is also where they can reflect on their score, which will be presented to them.
7. Secret sections of a level	I will include this as a surprise to the player as well as an additional goal. These secret sections will contain a collectible, maybe a super coin or a power-up
8. Enemies	These opponents provide a challenge for the player, preventing the game from becoming too easy and boring. They come in various types: some shoot, some attack with melee, and some simply walk towards the player, causing death upon contact.
9. Keyboard controls	This felt more appropriate because it is a platformer game, and because it is 2D there is no need for a change in perspective for the player, therefore eradicating the need for mouse controls. The controls will fall into the common usage of WASD and space, as well as some other keys for different abilities
10. Add power-ups like a dash or super jump	This would take direct inspiration from the games I have researched. I am including this because it is a good way to enlarge the range of movement and a new, cool mechanic to the game.

Success Criteria

The success criteria of the game will be a set of features and requirements I will include in the game and it will act like a list with aspects that I can check off as I go along through the development process.

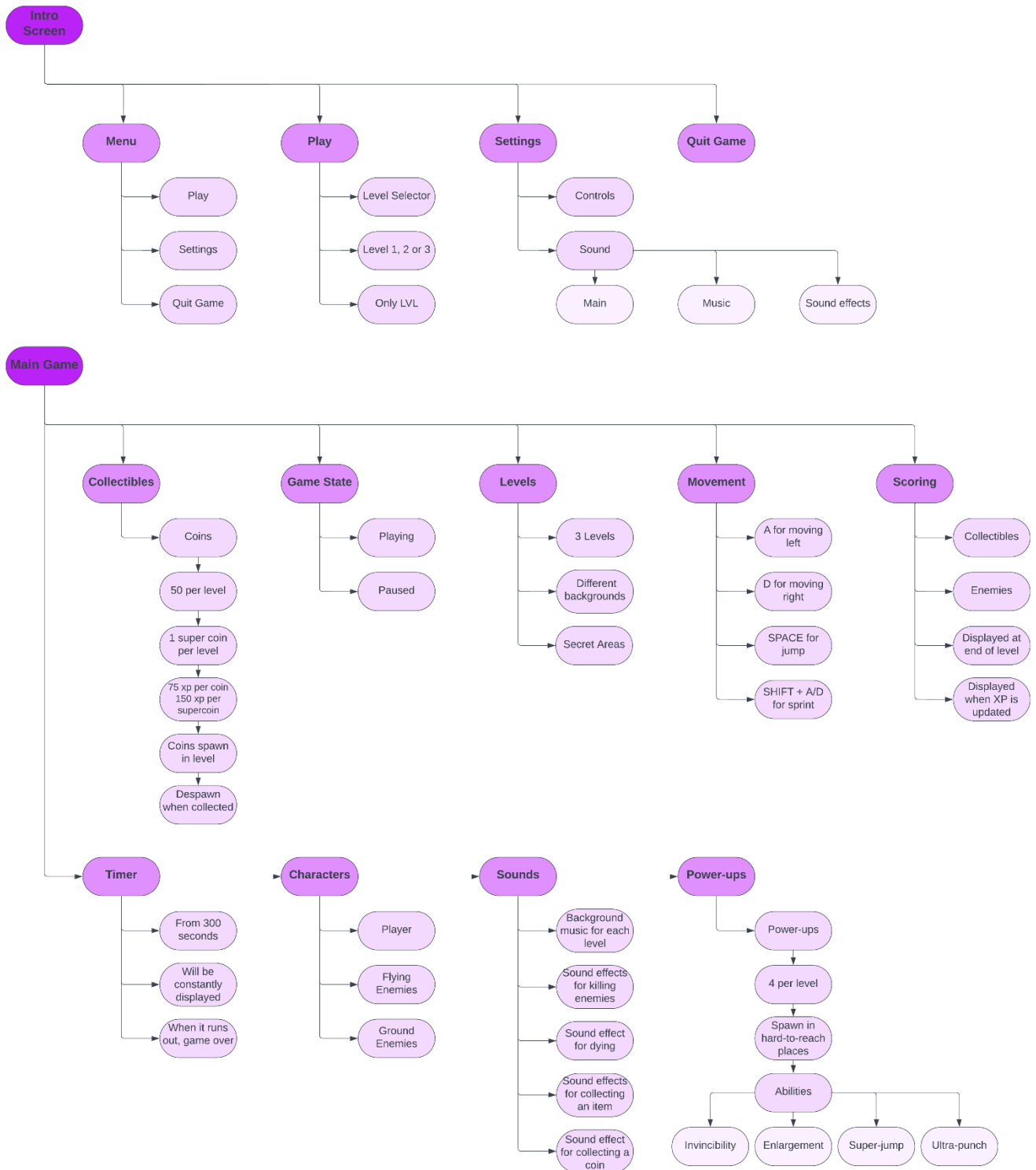
	Justification
1. Include a start screen on startup	I will include this to present the main theme of the game to the player as the first thing that they see. A good start screen can hook a player in, as seen in Celeste, or Mario or Rayman, so this is an essential but overlooked feature, that I must include. This will be measured with a screenshot.
2. Use a simple loading screen.	A loading screen is not a hugely noticed thing in games, but a black screen is not sufficient. My stakeholders have advised me to not spend too much time on it, but make it effective nonetheless. This will be measured with a screenshot.
3. Include a minimum of 3 levels	This was done to make the game long, so the player will be invested in the game, as by request of my stakeholders. I have chosen this over one much larger level in order variety in themes and colours per level. This will be measured with a counter of levels.
4. Create a 3rd person perspective from the side	This means that the POV will be from the side of the character, in order to display a large area of the level, as well as the player. This was chosen instead of others, like a first-person view, because it is a well understood and frequently used feature in the majority of platform games, like Celeste or Rayman Legends. A 3 rd person view was also chosen over a 1 st person by my stakeholders. This will be measured with a screenshot of gameplay.
5. Make the obstacles/platforms challenging to the player but still make sure it is enjoyable to play	This is to ensure the game isn't too flat and easy; the game isn't that long, so I need to make sure the player most likely can't complete the entire game first try. I want to lengthen their play time as much as possible while preventing them from getting bored or fed up of it.

6. Have only 1 life per checkpoint	This means that there is no health system. The player will die if it is hit by anything, and then respawn from the latest checkpoint. This was done in order to provide a challenge for the players, as well as taking inspiration from other games such as Geometry Dash and Celeste.
7. Include a variety of enemy types	Different types of enemies are essential in games, to avoid the gameplay becoming repetitive and to keep it challenging and exciting, especially when the players discover them for the first time. This criterion has come from different games with enemies, like Mario and Rayman Legends. This will be measured through videos of gameplay showing the enemies.
8. Have a different biome for each level	This can provide a some like story, because each level will present a rise in altitude until the last level at the top. There will be a minimum of 3 levels, per request of my stakeholders, as well as my opinion on a suitable and sufficient number of levels in the given time period for the project.
9. Have Esc bring up the in-game menu	This is a way to pause the game, which is an essential feature of any game, but also to bring up a menu for the player to change things or leave the level. This criterion is taken from the majority of PC games.
10. It will be a single-player game	This game will be a singleplayer, in order to create immersion in the game and reduce dependencies on other features like other players, which will hopefully increase the pleasure and enjoyability of the game itself. This was done per request of Ismail and Shazil.
11. Include a random number of collectibles per level	Again, this won't be a completely random number, but an appropriate amount. This will be done to make the level as detailed as possible and ensures the player explores everything. They will appear in the form of normal coins, or powerups or special coins.
12. Include a life count system and game over	The mechanic will work like this: the player has a central life count (of 5 starting off), that will be decremented each time the

	player dies. The life count can be increased with a threshold of XP in one level. If the life count reaches 0, a GAME OVER screen is presented and the game restarts, from the first level. This idea was taken from Mario, for the purpose of making a proper challenge for the player.
13. New lives can be earned for XP threshold in each level	If the player reaches a certain amount of XP in one level, I will make a function that gives them one more life/heart, on top of the central life count. This function is taken from Mario, and I included it because it creates a smaller secondary goal for the player, to go out of their way to collect things, and give the opportunity for an advantage in the game to be found.
14. Include a score system	This calculates the net value of XP that has been collected throughout the level. The items that contribute to the score are collectibles and power-ups. This has been included because it is a good way to measure the players skill and progress throughout the level. This is also a well-known feature in many games like Mario and Rayman Legends. This can be measured with a screenshot of the code that creates this function.
15. At the end of the level, the total XP will be calculated and presented	This is included in order to give an achievement to the player, which emphasises that smaller, secondary goal. I found this mechanic from Rayman Legends, which is present in my research. It can be measured with a screenshot of the code and a video of the game being played, showing the total XP being displayed.
16. Include a game saving system.	This is an essential feature that is in the majority of modern games. This is also in all of the games in my research. The game will automatically save at each checkpoint, but the player will be able to save the game at any time in the paused menu. This can be measured with a video of the game being saved, being closed, and then the game being loaded back up from the save.

17. Include a range of background music which is unique for each level and situation	This is a criterion because background music is a useful thing to use to create immersion into the game, and to create an atmosphere. If the game is silent, it might get awkward and the player might get bored. This is especially true in this game because there will be no dialogue, so I have to fill the space with something.
18. Include necessary sound effects	Sound effects are a crucial part of a game, because it increases interactivity with aspects in the game. I will include this because it is a prominent feature present in the majority of video games.
19. Create an informative but not distracting heads up display (HUD)	This is a feature in all games where user data is constantly changing. The purpose of this is to inform the player of their current data. I can use this to display things like the time left in the level before the timer runs out or the player's current score and XP.
20. Make menu navigational by mouse	Although there are no mouse controls in the game, I will allow for the menu to be navigated and interacted with a mouse, because it is the natural mechanic for the majority of players (as pointed out by my stakeholder), because it is a feature that is used in almost every video game.
21. Include a cool animation for when the player collects the collectibles	This is included to act as a reward for the player for collecting the coins. This will hopefully encourage them to attempt to get them all.
22. Include a snappy and satisfying dash with a good animation	This is included to add a further piece of movement to the player, and make the overall game more complex. This can also be used to enhance the game in terms of difficulty with different obstacles they can only avoid by using the dash.
23. Player will not fall into the void while on a platform	I am including this crucial feature because it is an unspoken necessity in games that make the game playable, and prevent the player making the game go into an infinite loop of falling while spawning.
24. Player will be able to stand and walk normally on a platform	

Design



Intro Screen

Menu

Play

This button will be at the top of the menu and will open another mini-menu, which will include:

- Level Selector
 - Where players can select which level to play, and their choice of level will then load up
 - There will be a choice of the 3 levels
 - Only Level 1 will be available when first playing the game

Settings

This will again open another mini-menu which will include:

- Controls
 - This will open yet another mini-menu which includes the options of control remapping, allowing the player to change the keys that they press to do different actions. There will be another option to view controls to become familiar with the controls in the game
- Sound
 - This will open a screen where there are different dials which affect different audio aspects of the game such as:
 - Main
 - Music
 - Sound effects

Quit Game

This will close the game when pressed.

Main Game

Collectibles

One of the collectibles available will be coins. These will be gold and shiny in order to emphasise that these are special collectibles and to stand out to the player.

There will be 50 coins per level, but there will not be a coin counter to provide a small difficulty for the player. There will be also be 1 secret prize super coin in a level, hidden in a secret area only found by discovering each intricate detail of the level.

There will be a reward for finding coins as per success criteria 20:

- 75XP per coin
- 150XP per super coin

The coins will spawn in when the level loads, and they will despawn when collected by the player. This will be done when the player simply walks into them. When they collide

with each other, the coin will present a small, subtle animation and it will then despawn, notifying the player that the coin has been collected.

There will be another form of collectibles which are the power-ups. These will serve as an advantage to the player and will have several abilities (which will be random). The power up list is sorted in terms of rarity descending:

- Invincibility
- Ultra-punch
- Enlargement
- Super-jump
- Dash

There will be 4 power-ups that spawn per level, and they will spawn in hard-to-reach places, so they are not too easy to find by the player.

Levels

There will only be 3 levels in this game, due to the time frame I have been provided with and I feel like this is sufficient. But there is no reason that updates cannot be made to the game with more levels being added in the future.

Each level will have its own background, in order to create a sense of adventure; to feel like you're actually moving; going somewhere.

There will also be secret areas in levels to provide an extra goal, past just completing the level as per feature list 7.

Movement

Movement will be basic and simple, following the WASD sequence and space for jump. But because this is a 2D game, there won't be 3-directional movement, so the controls will be:

- A for walking left
- D for walking right
- SPACE for jumping
- SPACE + A for jumping left
- SPACE + D for jumping right
- SPRINT + A for dashing left
- SPRINT + D for dashing right

Scoring

There will be several factors that contribute to the total score, such as:

- Killing enemies (25 XP)
- Collecting coins (25 XP)
- Collecting power-ups (50 XP)

When XP is collected by the player by completing one of these factors, a small pop-up will appear on the screen, showing a small animation for the coin. The total XP will not be displayed at all until the end of the level, in order for the player's focus to be constantly on completing the level to its entirety. The final reveal of the XP will be much larger than the smaller display, and will be presented in the middle of the screen in order to capture the players attention and to look captivating.

Timer

The timer will be the only feature of the GUI apart from the life count, and it will be constantly displayed on the player's screen. For every level, there will be a custom timer for them to complete the level, and when it runs out, the player will lose and a game over screen will be displayed. When the respawn button appears on the game over screen, you will be reset to the start of the level that you are on.

Characters

The characters will fall into 3 categories:

- The player
- The enemies
 - Ground enemies
 - Flying enemies

I am including different types of enemies to increase the variety of features, and also provide a further challenge for the player.

Screen Designs



This is the first screen that will be displayed to the user. There are some foreground details that resemble stalactites hanging from the ceiling and stalagmites from the floor. The stalagmites have some rocks in them which have been vaguely inspired from images online, in order to add more detail and depth to the foreground. The background are simply 3 colours which are aligned in a gradient and blending together in a checkered pattern. The obstacles have a stoney grey colour with a core colour of purple to represent hotter materials. I have also added little dashes of darker versions of the colours to add texture and detail to the obstacles. They also have a black outline. I have done this to remind the theme of an 8-bit game. The text has shadows to differentiate it from the rest of the screen and make it stand out. Finally, I have included a sprite in order to give the player a look as to what it will look like during actual gameplay. The Press to Start button needs to have a transparent background so when it's placed on the screen



When the player starts, this menu will immediately be displayed. Everything else is identical with the details, background and obstacles. This is a practical use of abstraction. There will be no mouse controls in my game, so the menu will be navigated with the arrow keys and the space bar. When a button is selected, it will display a darker yellow, shown in the screenshot above. Pressing the play button will present this screen:



This is the screen that will be displayed when the Play button is pressed. There won't be several saves being made that the player can select from, but instead there will be one save file that will be updated each time the player plays the game. Each level can be unlocked by fully completing the previous one, except from Level 1. The unlocked level will show an image of the level itself, whereas the locked versions will display a slightly blurred and monochrome image of that level. When a level is unlocked, the lock will open and the image will change.



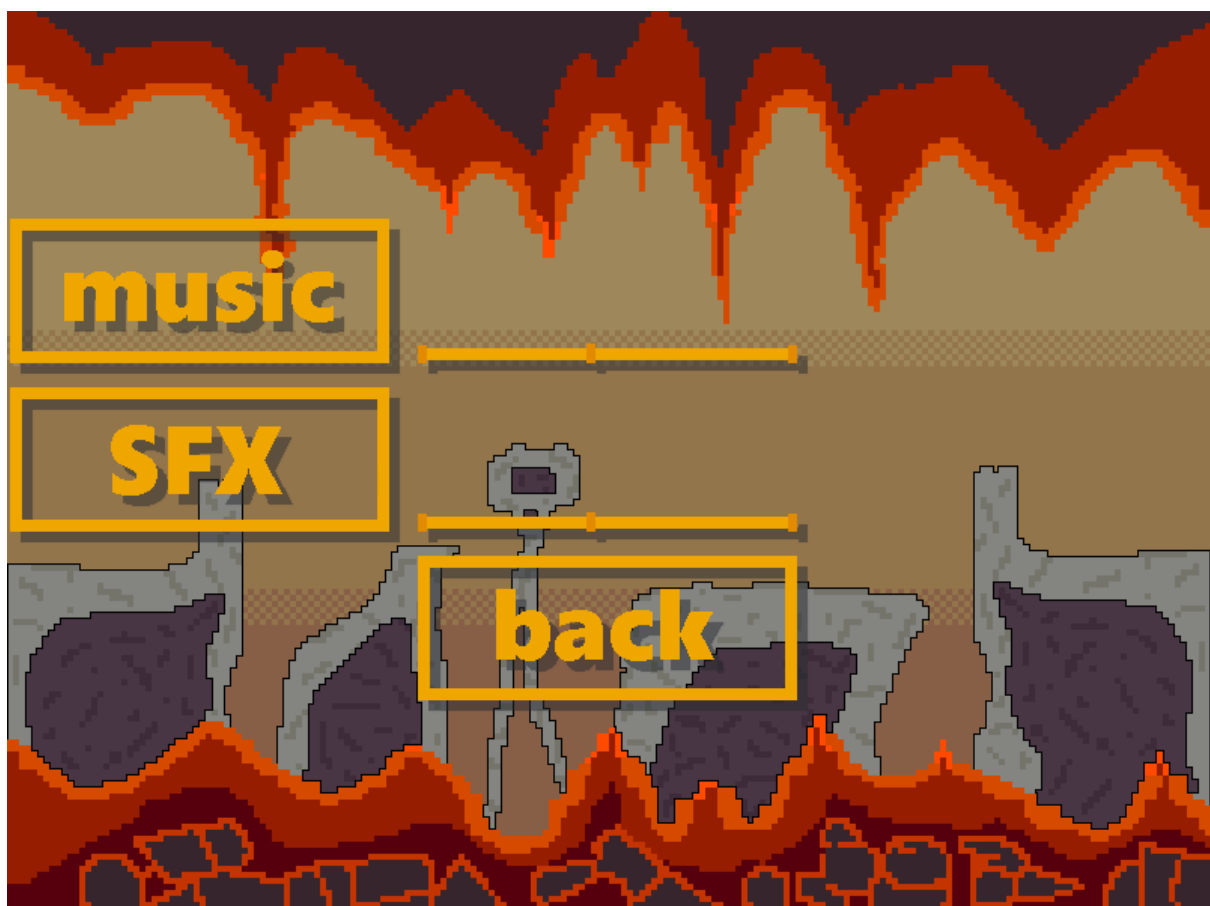
Now the other option in the main menu is shown in the screenshot. If this is selected, then the mini-menu will be shown, as seen in the image above.

This screen shows Controls, Audio and an option to go back to the main menu.

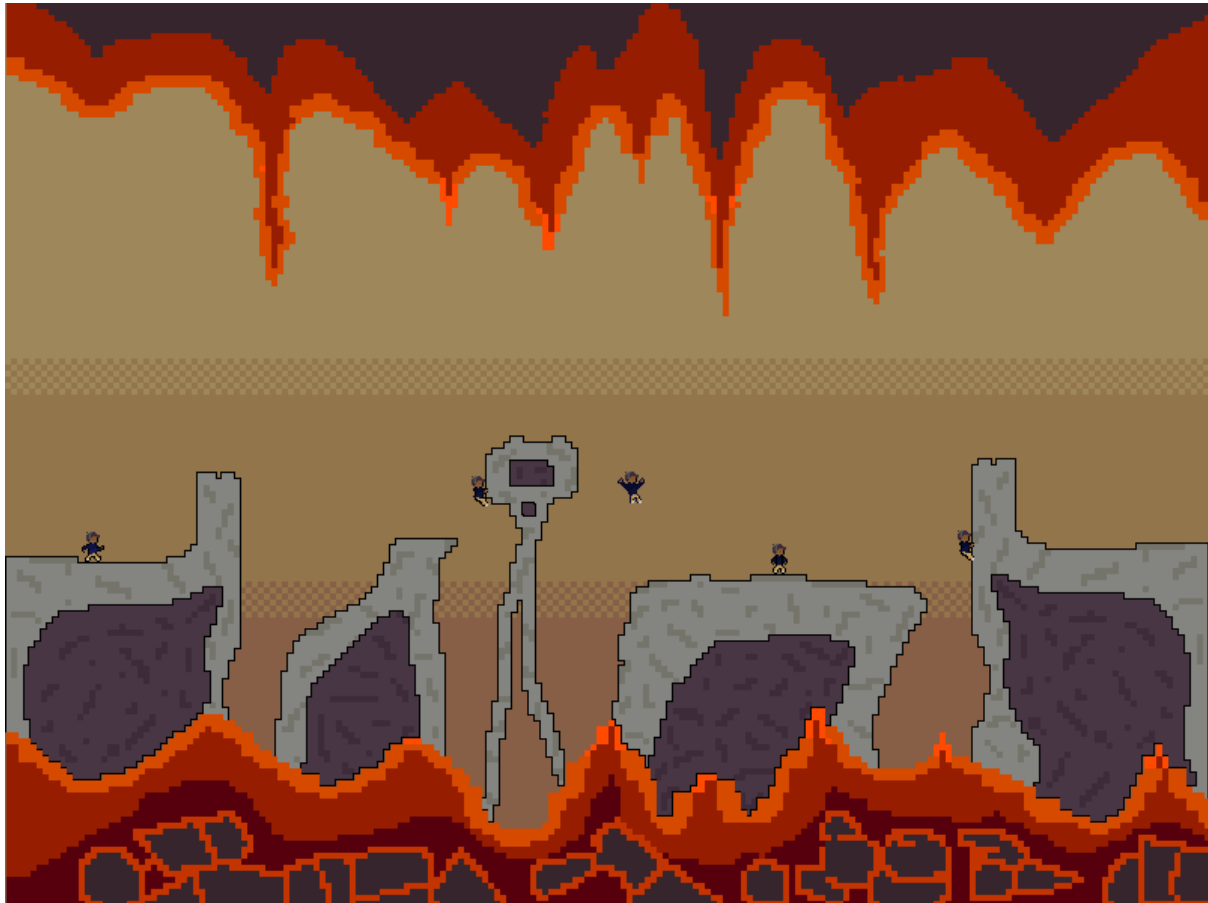
Again, everything else is identical in the foreground, background and obstacles in order to maintain efficiency in the development process.



This screen shows the controls of the game in a clear and simple-to-understand format. I have added a back button after a mistake was made and it was left out. My stakeholders reminded me of this problem and it has been amended now.



This is the audio screen. To save time in the development process, I limited the sound options to 2 essential things; the music playing in the background and the sound effects. Sound effects could include noises being made when a player hits something, or has been hit, or collecting an item etc. The player can change the volume of the value with the slider in the centre of the screen. I have added this screen to make the game more interactive and add more customisation, as this is bound to look appealing over some other games.



This is the first gameplay screen. It is again identical to the title screen detailing, with the obstacles, foreground and background being the same. This was done to provide a seamless transition from the menu to the game itself. The grey and purple rocks act as the obstacles for the player, where they can run across and climb. If at any point the player falls off of the platforms onto the orange foreground, the player will die and then respawn at the start of the screen, with one life taken away from the total. In this screen, all sprites are displayed where they are necessary. I have done this to show what they will look like during gameplay.

Sprites	Explanation
 Stationary   	These are the sprites that will cycle in a loop to provide a small animation for the sprite when it is stationary. I have done this to make the game look more visually appealing, even when the game is not actively being played.
 Running  	When pressing either A or D, these 3 running animations will cycle, in order to present a realistic running animation.
 Climbing	When the character is climbing, this sprite will replace the other, in order to present the idea that the character is climbing. There are 2 versions of this sprite, to give the impression of real climbing movements.
 Falling	Whenever the character falls in any direction, this sprite will appear and replace the other.

These are all the screens and images from the main menu of the game.

Interview

At this point. I interviewed my stakeholders on their opinion on the screens and sprites.

Ismail

Do you think the menu is visually appealing while remaining informative, as well as simple and clear?

I believe that some parts are visually appealing, however the grey colour for the locked levels on the play screen is rather unappealing. Also, the control screen is too simplistic; there is no back button. Also, the background is dull and boring.

Do you think the alignment of the buttons is clear?

Yes, it being in the centre brings the player's attention to it immediately

Do you think the menu should be navigated with the arrow keys, the mouse or both?

The menu should never be navigated with just arrow keys, since it would cause confusion with the players. Therefore, the menu should be navigated with either the mouse or both.

Do you think that all the options that are available are sufficient?

The menus are sufficient with options, but the controls screen has no back key

NOTE – This feature has now been added to the controls screen.

What other options would you add to the menu?

When the QUIT button is pressed, put a verification screen to ask if the player is sure they want to quit, to avoid accidents

Were there enough options in the audio menu? Was there anything you expected to see but didn't?

Make the indicator more clear and wider. Align the entire slider against the middle of the name box

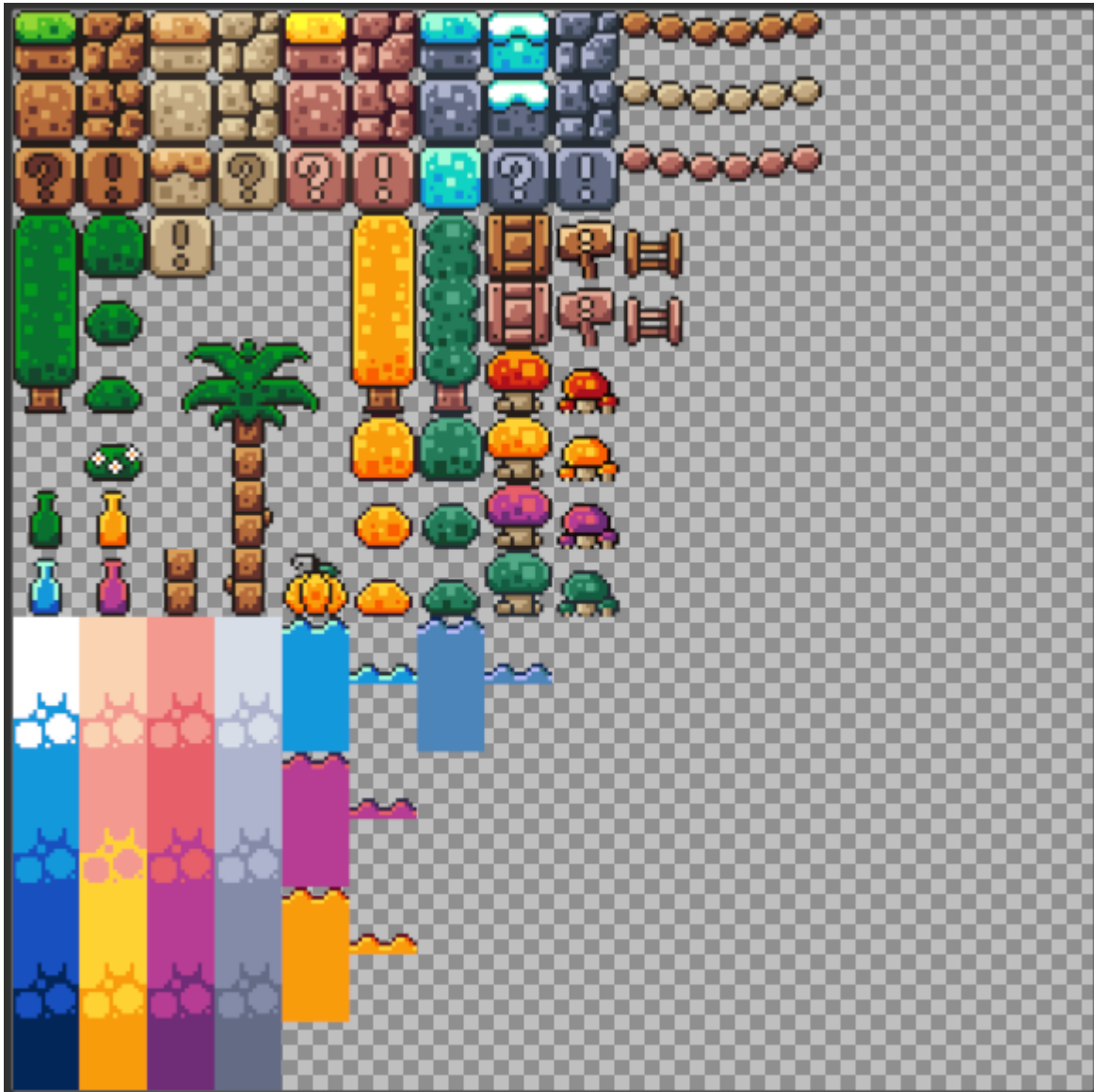
What would you change, if you wanted, about the background

Make it more appealing with more detail and more vibrant colours

Other Resources

TileMap

There are other resources, away from the player sprites, that I will use to design the visual aspect of my game. One example is the TileMap, which I will use to make the foreground details, background details and main obstacles of each level. I have chosen to retrieve these assets from the Godot community (<https://rottingpixels.itch.io/four-seasons-platformer-tileset-16x16free>). These resources are CC0, therefore not under copyright laws and fine for me to use in this project.



This is the first copy of the TileMap I will be using. The main 4 types of ground material at the top will most likely be the ones I will mainly be using. To fit this to match my design screens as much as possible, I will recolour these pixels so that they fit the theme of my screens. I will also use these textures at the bottom for the background (these will also

be recoloured for the same reason). This TileMap contains small details I could also use to make the level look nicer and more decorated.

I will also use another resource for the texture of the coins and platforms. I have retrieved this from <https://kevins-moms-house.itch.io/four-seasons-platformer-sprites>, and these resources are also CC0.



Variable List

Class – Player	Explanation
Attributes	
Alive State	This attribute will hold a Boolean value which determines if the player is alive or dead. True means alive, False means dead. I have included this instead of a health value because I will make the player die from even one hit.
Power-up State	Another Boolean value will be stored in this attribute, which determines if the player is carrying a power-up. True if yes, False if no.
Dashing State	This Boolean value will store the fact if the player is dashing or not. I am including this so that I can easily control the player and start/stop their dash when necessary.

Power-up	This attribute will hold a string value which stores the name of the power-up that the player is carrying.
Jump strength velocity	This will be an attribute which stores the value of the height of which the player can jump at one time, as this may change during gameplay due to power-ups
Speed	This private attribute will store the value of the speed that the player can run in a certain direction at a time. This will not change at any time in the game so it will be private.
Location	The player's current coordinates (X & Y). This is stored so that it can be used in the calculations of moving the character while running, climbing or jumping.
Methods	
set_aliveState	To change the alive state if the player has died or if they are respawning
set_powerUpState	To change the power-up state if the player has picked up a power-up or the effect has run out
set_powerUp	To change the name of the power-up that is active. If no ability is active, this will be set to "NONE"
set_jumpStrength	To set the jump strength of the character if the player receives a power-up for jump
set_location	This method will be to change the location of the player, while it moves around by running, jumping or climbing.
get_aliveState	To fetch whether or not the player is still alive. If they are not, then the player will respawn.
get_powerUpState	To fetch whether or not the player has a powerup or not, in order to then be able to select which powerup the player currently has
get_powerUp	Method that fetches the string that says the name of the power up that is active. If no power up is active, the string will contain "NONE"
get_jumpStrength	Method that fetches the integer value of jump strength to check if the effect of a powerup is active or not.
get_location	To check the location of the player at the current time.

Class – “Enemy”	Explanation
Attributes	
Enemy Type	Private attribute which determines which type of enemy it is (flying, ground etc.)
Methods	
get_location	A function to retrieve the position of the NPC to verify whether there is a collision between it and the player.
set_location	To change the position of the NPC when spawning them in

Class – “Power-up Collectible”	Explanation
Attributes	
Location	Variable which holds the location of the item in the level
Ability	Variable that determines which power-up is given to the player when collected
Methods	
get_location	To check where the coin should be spawned while loading the level
set_location	To set a certain location to spawn the item
get_ability	To check the power-up ability that will be gifted to the player when collected
set_ability	To determine the ability that will be given to the player when collected

Class – “Coin Collectible”	Explanation
Attributes	
Location	Variable which holds the location of the item in the level
XP	The amount of XP that will be gifted to the player when collected
Methods	
get_location	To check where the coin should be spawned while loading the level and to

	check whether there was a collision between it and the player.
get_XP	To check the amount of XP that will be gifted to the player when collected
set_location	To set a certain location to spawn the item
set_XP	To set a certain amount of XP depending on the type of coin (i.e. super coin or normal collectible)

Test Table

Test number	Name of test	Type of test	Expected output
1	Collision with wall/floor	Valid	Stops movement into wall
2	“W”	Valid / borderline	Jump
3	“A”	Valid / borderline	Move left
4	“S”	Valid / borderline	Move down
5	“D”	Valid / borderline	Move right
6	“Spacebar”	Valid / borderline	Jump
7	Up	Invalid	Do nothing
8	Left	Invalid	Do nothing
9	Down	Invalid	Do nothing
10	Right	Invalid	Do nothing
11	“Esc”	Valid	Pause game

Test number	Name of test	Type of test	Expected output
12	Do nothing	Valid	Play idle animation
13	Jump movement	Valid	Jump animation plays
14	Run left movement	Valid	Flip sprite horizontally and play run animation

15	Run right movement	Valid	Play run animation
16	Climb on wall	Valid	Change to climb sprite
17	Climb up/down wall	Valid	Play climbing animation
18	Fall down	Valid	Play falling animation
19	Crouch and do nothing	Valid	Play crouch idle animation
20	Crouch and jump	Valid	Play crouch jump animation
21	Crouch and run left	Valid	Flip sprite horizontally and play crouch run animation

Test number	Name of test	Type of test	Expected output
22	Select “Play” button (LMC)	Valid	Open Level Selector scene
23	Select “Settings” button (LMC)	Valid	Open Settings scene
24	Select “Quit” button (LMC)	Valid	Quit game
25	Click anywhere where there is no button (LMC)	Invalid	Do nothing
26	Select “Play” button (RMC)	Invalid	Do nothing
27	Select “Settings” button (RMC)	Invalid	Do nothing
28	Select “Quit” button (RMC)	Invalid	Do nothing
29	Click anywhere where there is no button (RMC)	Invalid	Do nothing

Test number	Name of test	Type of test	Expected output
30	Click “Music” button (LMC) while unmuted	Valid	Mute music
31	Click “SFX” button (LMC) while	Valid	Mute SFX

	unmuted		
32	Click “Music” button (LMC) while muted	Valid	Unmute music
33	Click “SFX” button (LMC) while muted	Valid	Unmute SFX
34	Click “Music” button (RMC)	Invalid	Do nothing
35	Click “SFX” button (RMC)	Invalid	Do nothing

Test number	Name of test	Type of test	Expected output
36	Dash (W being held)	Valid	An upwards dash
37	Dash (A being held)	Valid	A leftwards dash
38	Dash (S being held)	Valid	A downwards dash
39	Dash (D being held)	Valid	A rightwards dash
40	Dash (W+A being held)	Valid	An upward left (diagonal) dash
41	Dash (W+D being held)	Valid	An upward right (diagonal) dash
42	Dash (S+A being held)	Valid	A downward left (diagonal) dash
43	Dash (S+D being held)	Valid	A downward right (diagonal) dash
44	Dashing without a direction (none of WASD being held)	Invalid	No dash would occur
45	Dashing into a wall	Invalid	Player would collide with the wall and not go through
46	Dashing into the floor	Invalid	Player would collide with the floor and not go through

47	Dashing into the ceiling	Invalid	Player would collide with the ceiling and not go through
----	--------------------------	---------	--

Test number	Name of test	Type of test	Expected output
48	Walking into a coin to collect it	Valid	Plays collected animation and despawns
49	Collecting a coin twice before it despawns	Invalid	Only able to collect once, despawns before can be collected again
50	Collecting coin to add to XP	Valid	XP will be added by 25 per coin collected
51	Collecting coin to add to coin count / score	Valid	Score will be added by 1 per coin collected
52	Dying to reset score & XP	Valid	Score & XP will reset upon death

Test number	Name of test	Type of test	Expected output
53	Select "Level 1" button (LMC)	Valid	Open Level 1 scene
54	Select "Level 2" button (LMC)	Valid	Open Level 2 scene
55	Select "Level 3" button (LMC)	Valid	Open Level 3 scene
56	Select "Level 1" button (RMC)	Invalid	Do nothing
57	Select "Level 2" button (RMC)	Invalid	Do nothing
58	Select "Level 3" button (RMC)	Invalid	Do nothing

Test number	Name of test	Type of test	Expected output
59	Launch game	Valid	Level 1 music plays

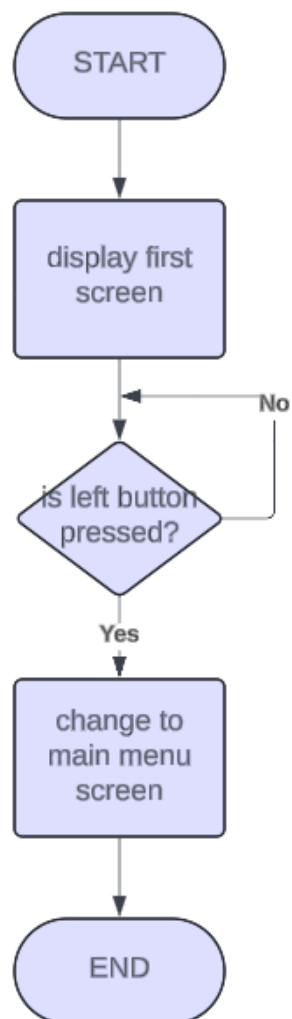
60	Select "Options"	Valid	Level 1 music plays
61	Select "Controls"	Valid	Level 1 music plays
62	Select "Audio"	Valid	Level 1 music plays
63	Select "Play"	Valid	Level 1 music plays
64	Select "Level 1"	Valid	Level 1 music plays
65	Select "Level 2"	Valid	Level 2 music plays Level 1 music stops
66	Select "Level 3"	Valid	Level 3 music plays Level 1 music stops

Test number	Name of test	Type of test	Expected output
67	Save Level 1 progress (while in Level 1)	Valid	No visual output Write player position to text file
68	Load Level 1 progress from save (while in Level 1)	Valid	Teleport player to saved position
69	Load Level 1 progress from save (while in Level 2)	Invalid	Do nothing (because Level 1 coordinates are not compatible in Level 2)
70	Load Level 1 progress from save (while in Level 3)	Invalid	Do nothing (because Level 1 coordinates are not compatible in Level 3)
71	Save Level 2 progress (while in Level 2)	Valid	No visual output Write player position to text file
72	Load Level 2 progress (while in Level 2)	Valid	Teleport player to saved position
73	Load Level 2 progress from save (while in Level 1)	Invalid	Do nothing (because Level 2 coordinates are not compatible in Level 1)

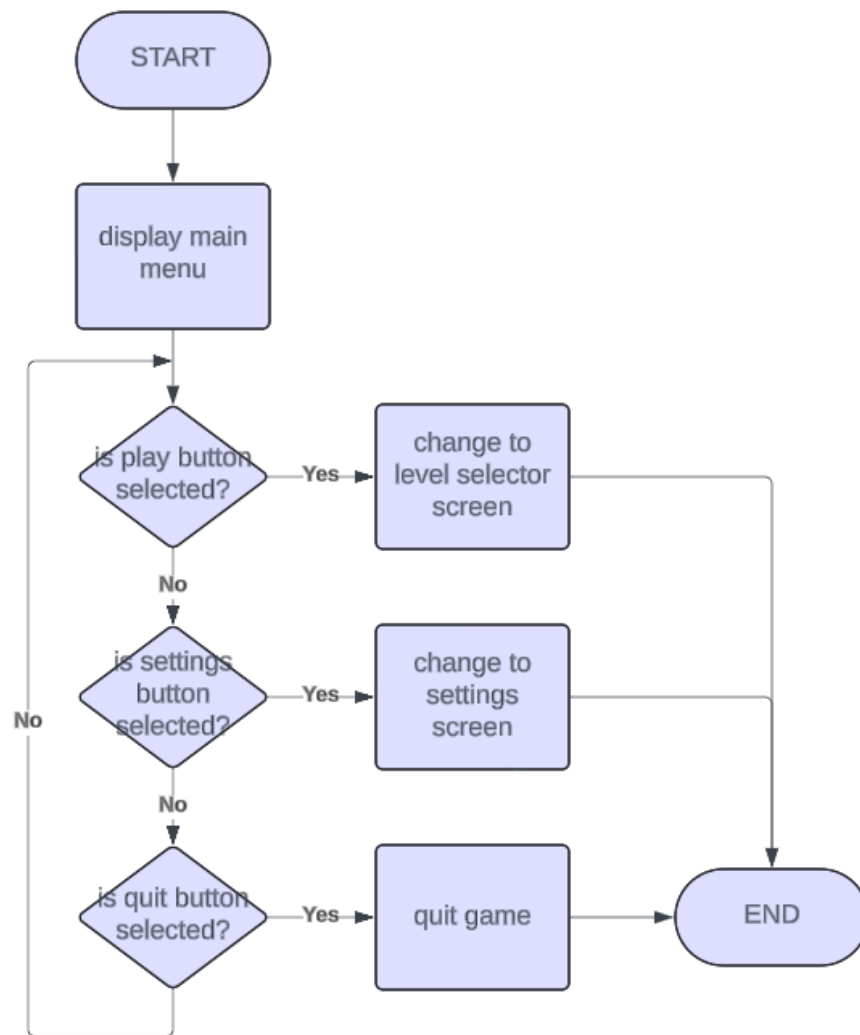
74	Load Level 2 progress from save (while in Level 3)	Invalid	Do nothing (because Level 2 coordinates are not compatible in Level 3)
75	Save Level 3 progress (while in Level 3)	Valid	No visual output Write player position to text file
76	Load Level 3 progress (while in Level 3)	Valid	Teleport player to saved position
77	Load Level 3 progress (while in Level 1)	Invalid	Do nothing (because Level 3 coordinates are not compatible in Level 1)
78	Load Level 3 progress (while in Level 2)	Invalid	Do nothing (because Level 3 coordinates are not compatible in Level 2)
79	Load progress without initial saves	Invalid	Do nothing (because there is no saved data)
80	Load progress from save after relaunching game	Valid	Teleport player to saved position
81	Load progress without initial saves after relaunching game	Invalid	Do nothing (because there is no saved data)
82	Save coin count	Valid	No visual output Write coin count to text file
83	Load coin count from save	Valid	Update coin count to saved value
84	Load coin count without initial saves	Invalid	Do nothing (because there is no saved data)
85	Load coin count without initial saves after relaunching game	Invalid	Do nothing (because there is no saved data)
86	Load coin count from save after relaunching game	Valid	Update coin count to saved value

Flowcharts

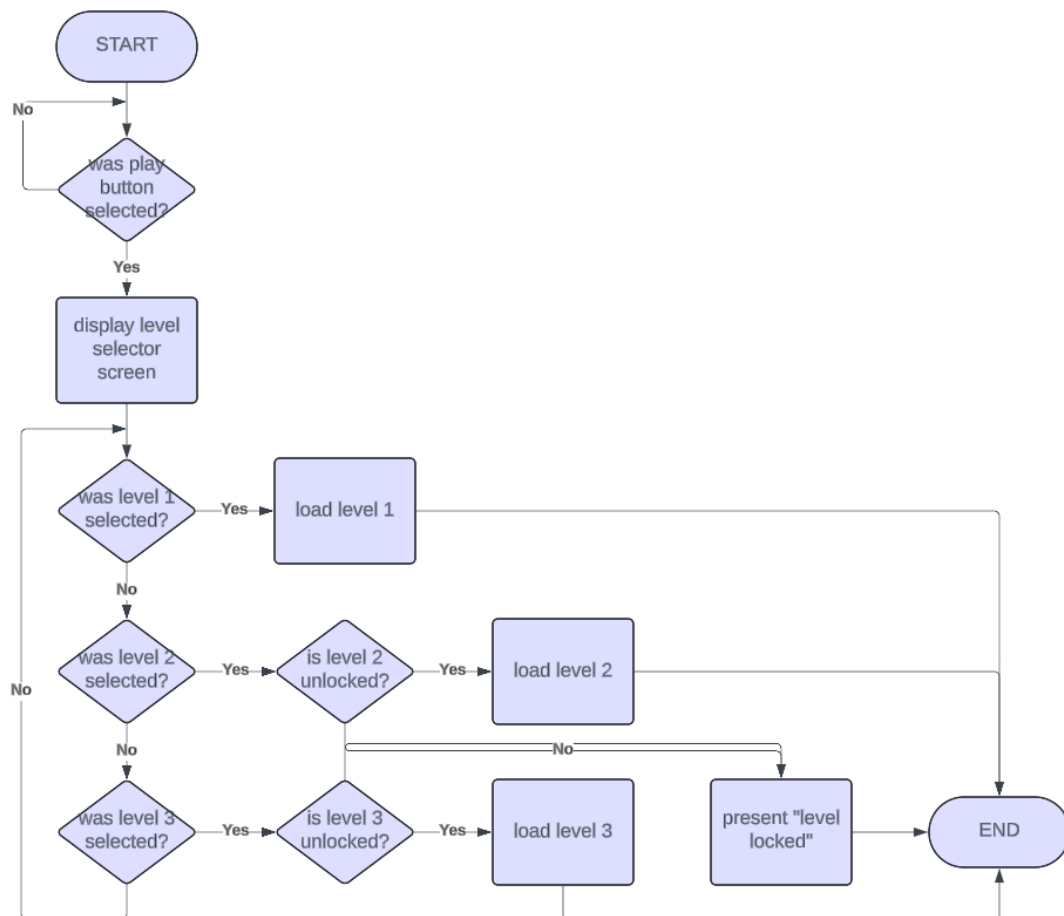
Start-up



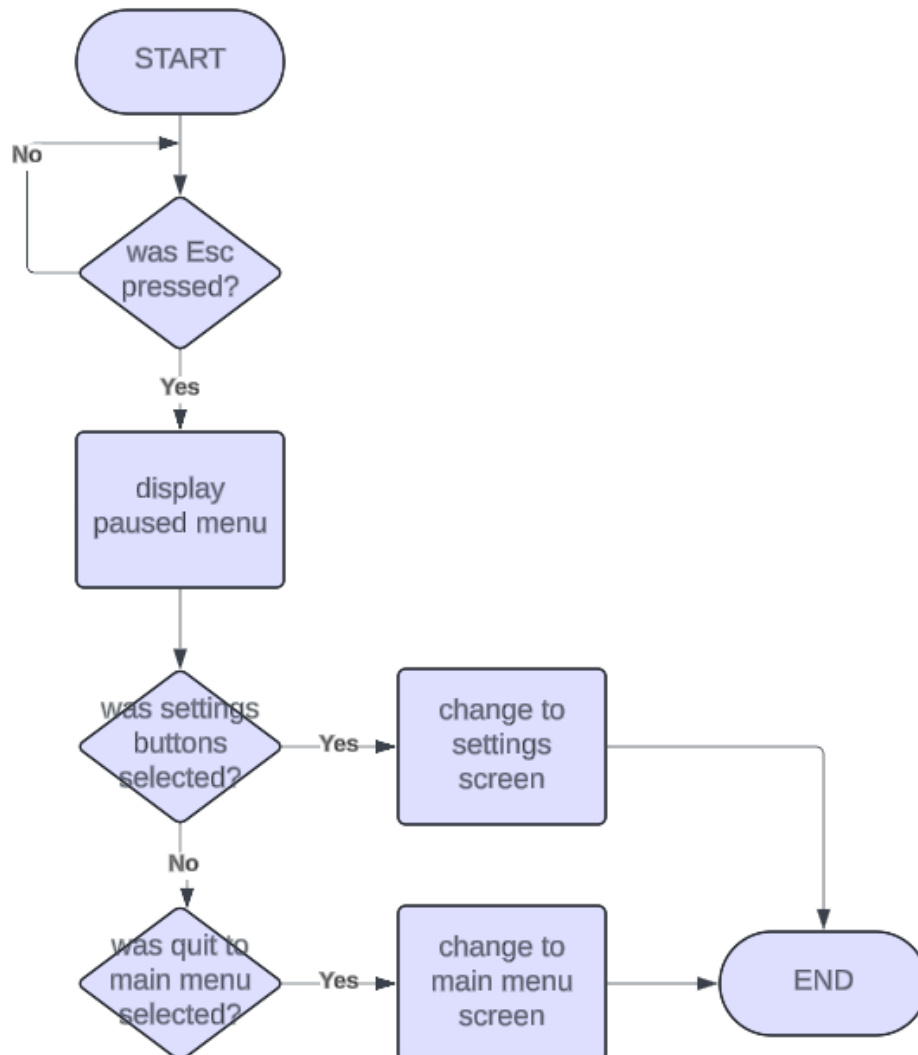
Menu



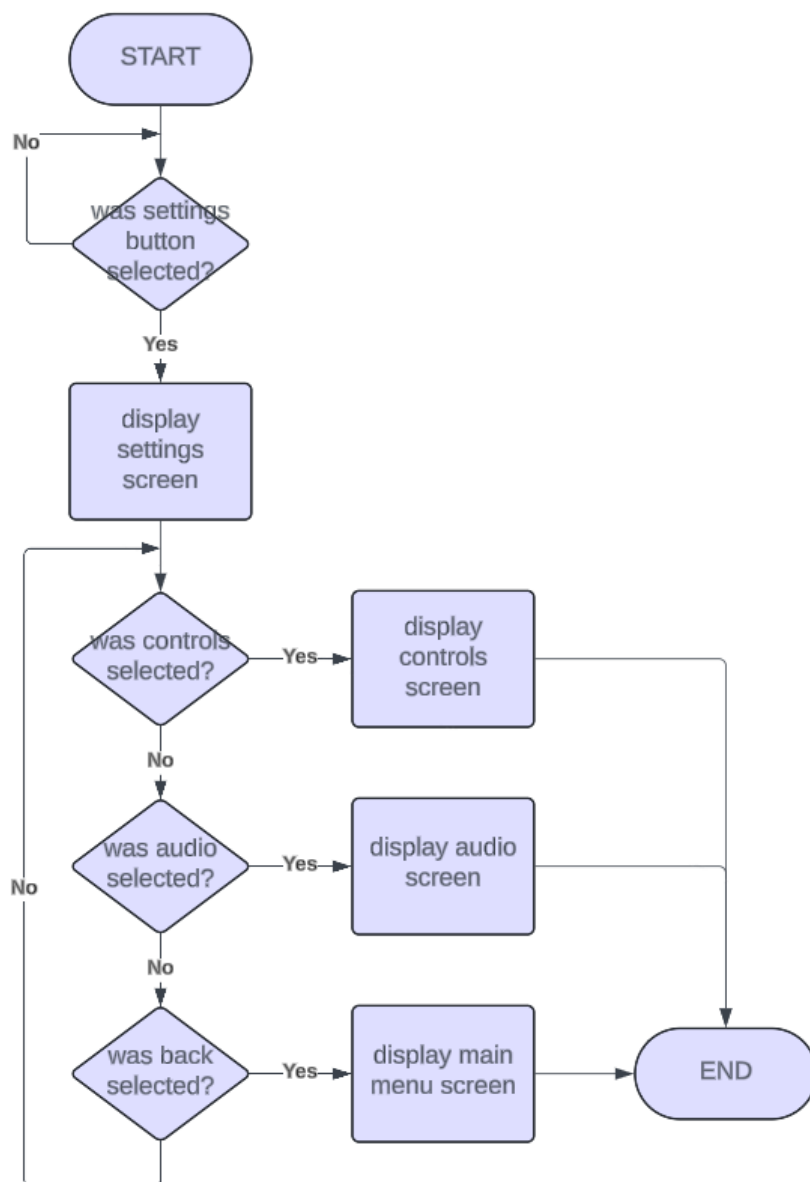
Level Selector



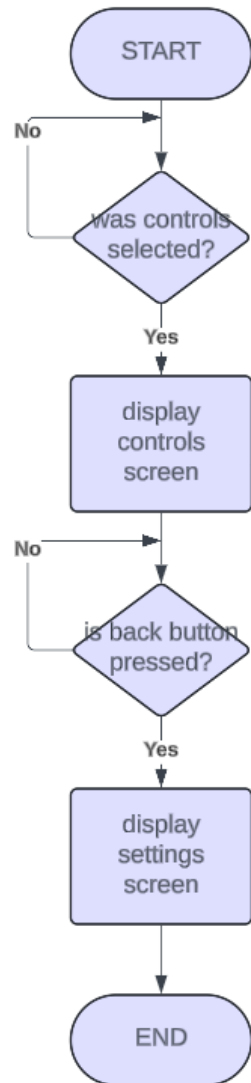
Pause Menu



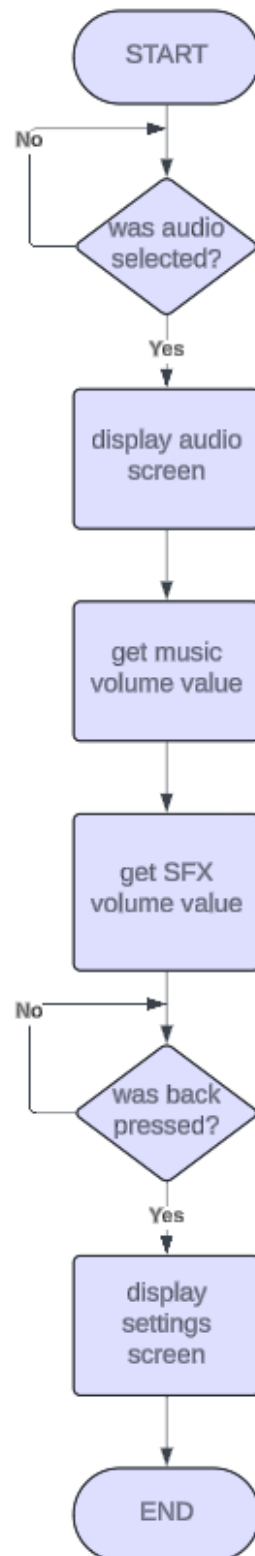
Settings Screen



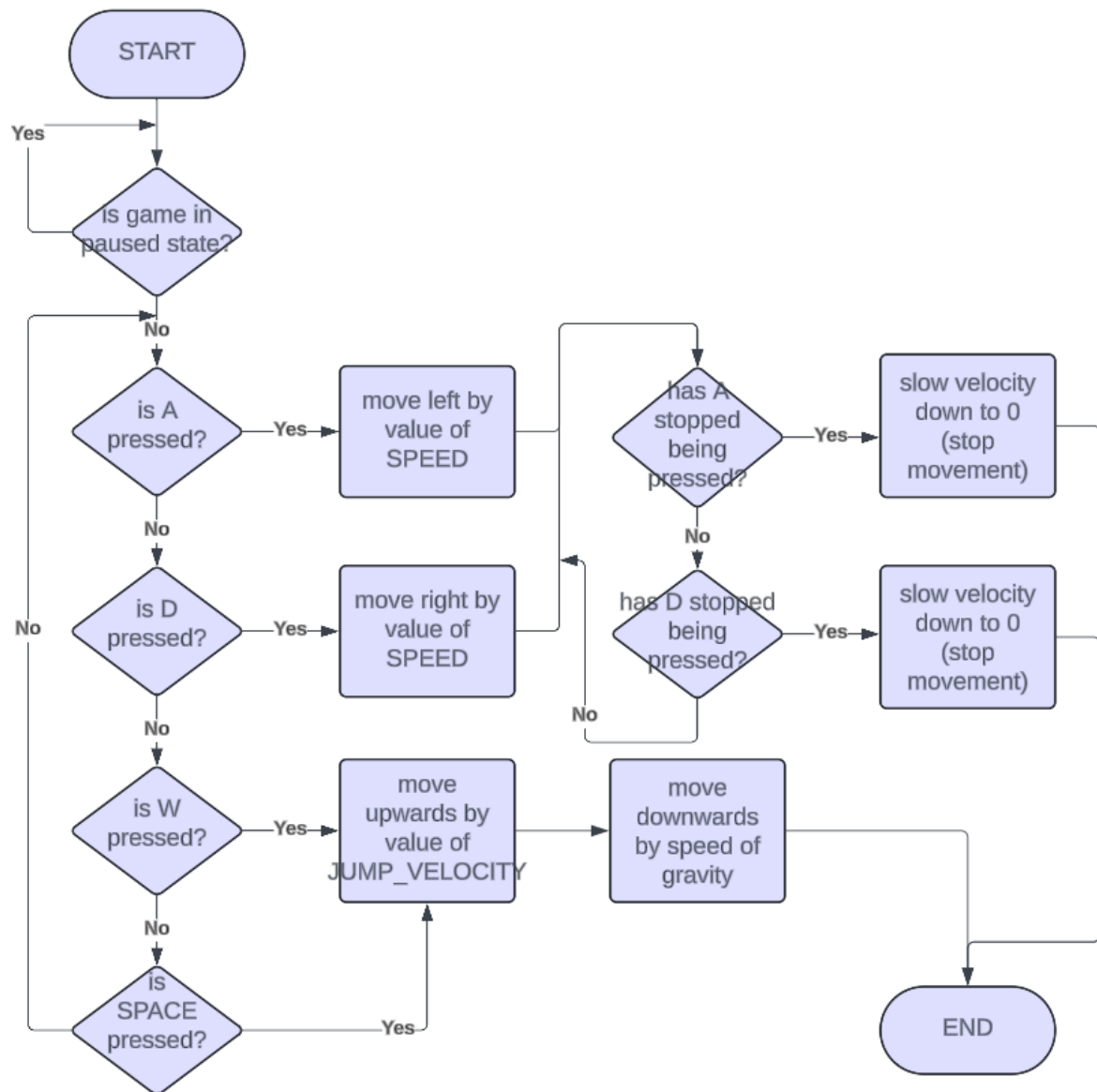
Controls Screen



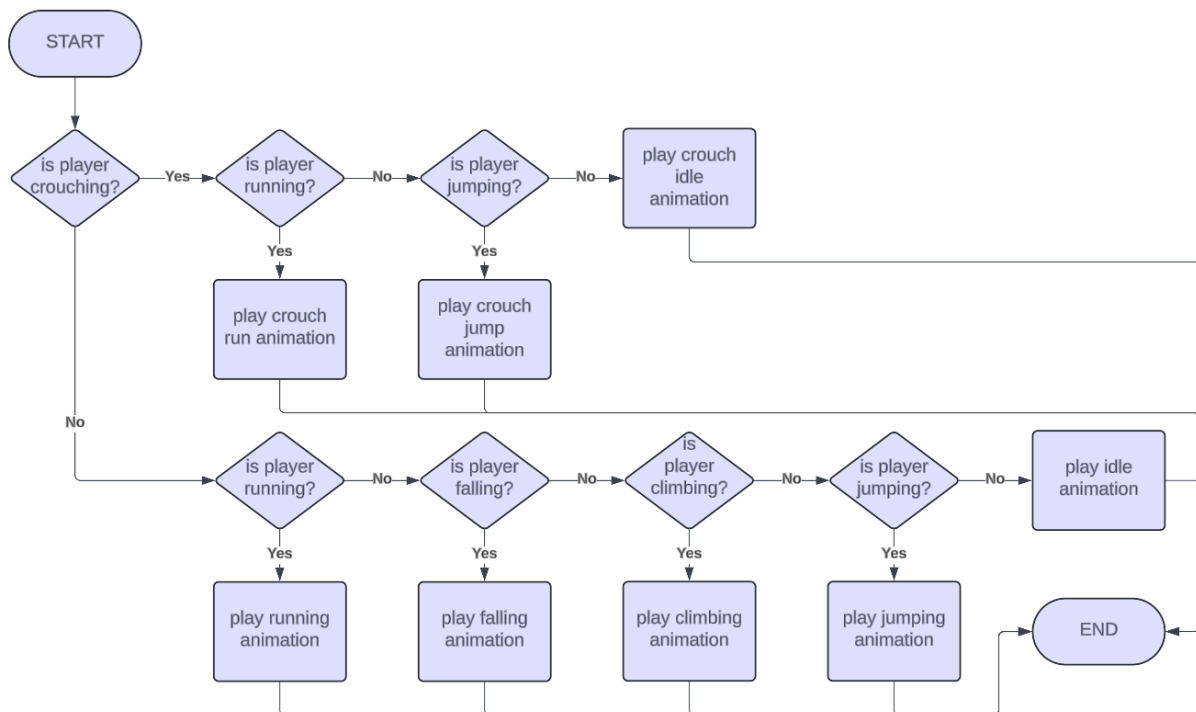
Audio Settings



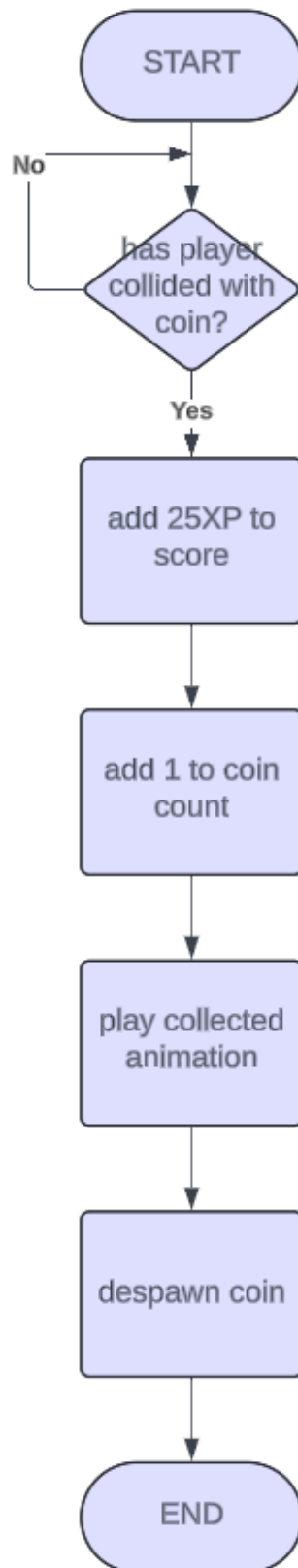
Movement



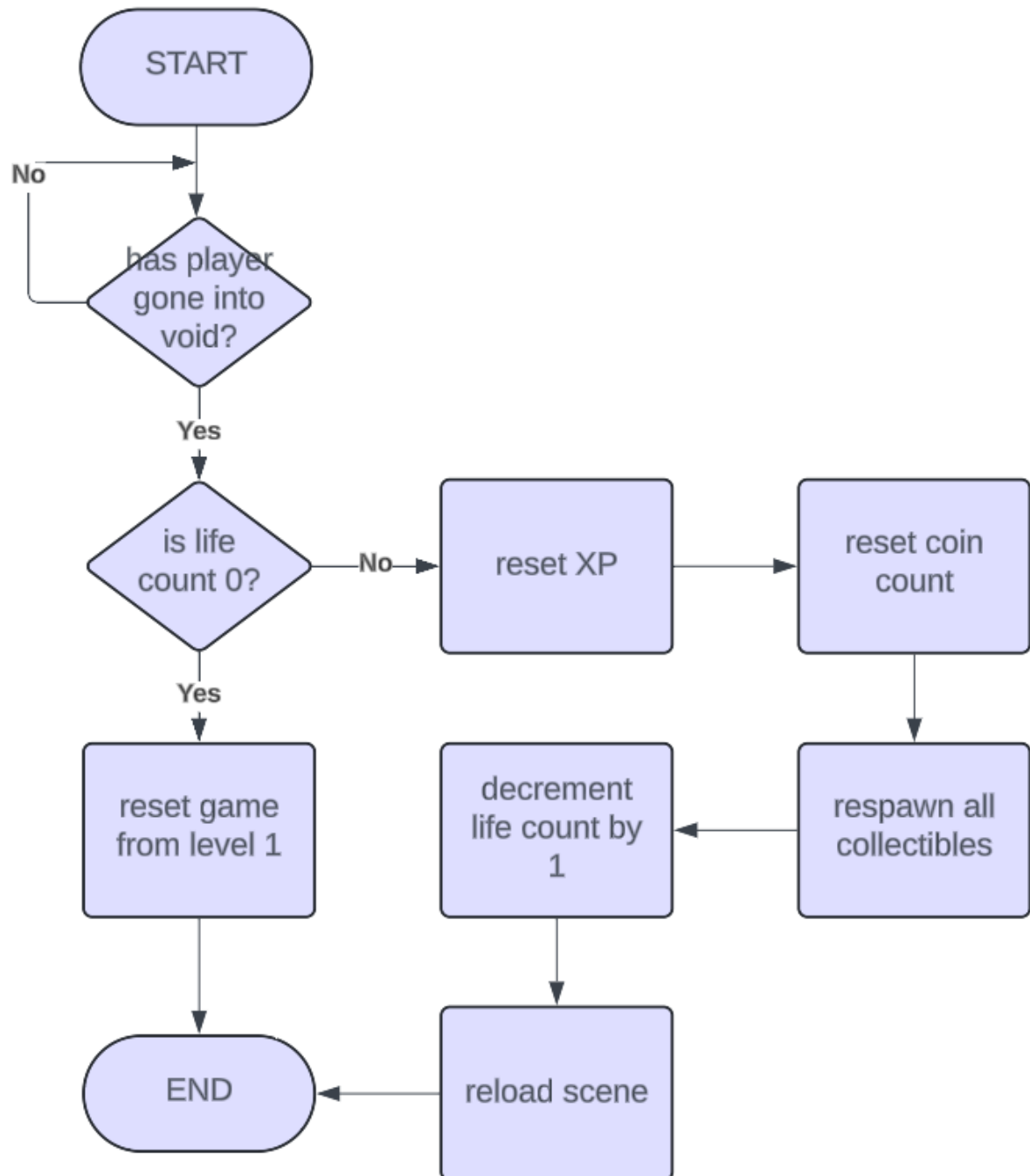
Animations



Coins



Falling death



Development

Player

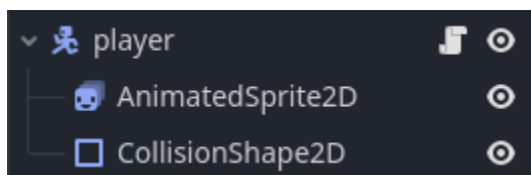
Idle Sprite

The first thing I did when I created my project was make the Player scene, as this is the most essential thing in the entire game. I wanted the sprites to be animated to improve the interaction of the game itself, so this meant making a set of sprites that could create an animation for every possible state of sprite. I began with the idle animation, because this is the most basic and the first step I should make.



This screenshot was taken from my sprite map, which is an image where I create, design and store all the sprites. There are 4 sprite models here, all incredibly similar but only with 1 change per model. Overall, these 4 models create an animation for the character breathing and bending their legs while idle.

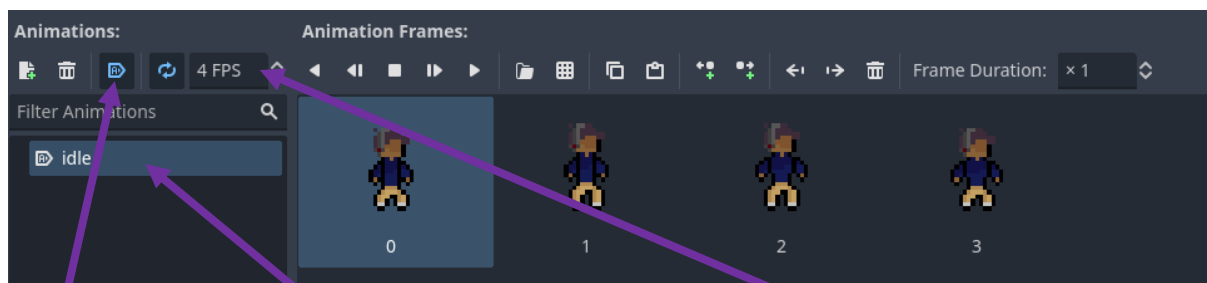
Now, the models are made, I needed to put them into Godot itself, and loop the animation so it would present.



First, I made the main parent node and called it Player. Then as a child node, I created an animatedSprite. Then, in the Inspector tab of the animatedSprite node, I went to the Animation drop-down menu and selected a new spriteFrames. This creates a new element which will be read when the animation is playing. From then, a menu appeared, and I imported my sprite map as a file in.



Godot recognised it and opened a Select Frames menu, which splits the entire image into separate boxes. You can then tweak the size of these boxes until each box contains one frame model of the sprite. You then select all required boxes and press “Add X frames”. This imports all the models and can create the animation

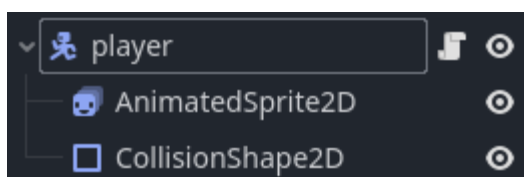


This button is Autoplay on Load. This means that the animation will immediately and always play

Field to rename the animation file

Field to change the speed of the animation playing

The idle animation has now been finished.

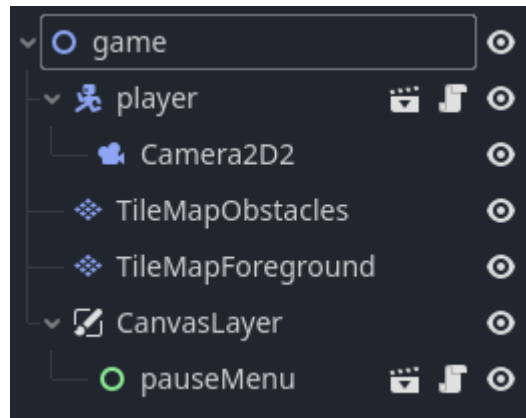


Next, I created another child node called CollisionShape2D. After creating it, I went the Inspector tab, and selected a new shape for the node. I selected the rectangular one, because it would fit the shape of my sprite most appropriately. I then reshaped it to fit the size of the sprite.

The CollisionShape2D node will act as a block, which the physics will act on. The physics engine will not affect the sprite itself (AnimatedSprite2D), but they will affect the CollisionShape2D node. This is essentially what the sprite actually is, but the AnimatedSprite2D is what the sprite actually looks like.

This is all for now, but I will add more animation sprites to the player scene later.

Game Scene



I created a new 2D scene, then added several nodes, including dragging in the player scene created earlier, as well as a camera node, 2 TileMaps, and a canvas layer with the pauseMenu scene as a child of that. This and the TileMap nodes will be explained later. The camera node was included because this is where the visual aspect of the game comes from; this is where the player sees from (POV). This follows one of my success criteria that said to “create a 3rd person perspective from the side” (4). Originally, I added the camera node regularly. But while testing the game, I found that the animation would play correctly, but while moving around (with movement explained below) the player would simply move off the screen if you got too far. This is because the camera's position is static; I needed it to follow the player. Fortunately, this is easily solved by making the camera a child node of the player. This way, the camera follows the player's movements.

Basic Movement

I then moved on to the physics and movement of the gameplay. Fortunately, Godot is able to provide some basic movement controls in a template which I utilised to save time and provide a basis for the game easily.

```
1  extends CharacterBody2D
2  class_name Player
3
4  const SPEED = 155
5  const JUMP_VELOCITY = -245
```

```
14  #gravity value (created by godot)
15  var gravity = ProjectSettings.get_setting("physics/2d/default_gravity")
```

Line 1 connects the script to the node

Line 2 creates the player class

Line 4 creates a constant variable; calls it SPEED and puts an integer value in it that will determine the characters speed throughout the game.

Line 5 does the same thing as Line 4, but it names it JUMP_VELOCITY, and will determine how high the character can jump.

Line 7 creates a variable called gravity. This then imports the physics library which is provided by Godot. This is needed because physics is essential for movement, and I found it more efficient in the development process to use Godot's template instead of creating one myself which would've wasted time.

```
10 func _physics_process(delta):
11     if not is_on_floor():
12         velocity.y += gravity * delta
13
14     if Input.is_action_just_pressed("move_up") and is_on_floor() or Input.is_action_just_pressed("spacebar") and is_on_floor():
15         velocity.y = JUMP_VELOCITY
16
17     var direction = Input.get_axis("move_left", "move_right")
18     if direction:
19         velocity.x = direction * SPEED
20     else:
21         velocity.x = move_toward(velocity.x, 0, SPEED)
```

Line 10 creates a new function called _physics_process which takes in the parameters of the variable "delta". The value of this is provided by the physics engine by Godot.

Line 11 provides an IF statement which checks if the player is not on the floor. Only if the player is on the ground, Line 12 will run.

Line 14 provides an IF statement which checks if either "W" or [space] has been pressed, and also if the player is on the floor. If all these conditions are true. Line 15 will run.

Line 15 changes the vertical velocity from 0 to the value of JUMP_VELOCITY (the constant created earlier). Overall, this provides a jump.

Line 17 creates a variable called direction and in it goes the result of the function "Input.get.axis()". This function checks if the keys mapped to "move_left" (a) or "move_right" (d) have been pressed. The value returned is stored as a float data type ranging from -1 to 1. The result is 0 if nothing is pressed.

Line 18 provides an IF statement which checks if a key is pressed. This is because "if direction:" checks if there is a value in the variable direction. If there is, then a key has been pressed and the value of direction is either -1 or 1.

Line 19 changes the horizontal velocity from 0 to the value of direction multiplied by the SPEED constant variable. The value of direction determines the direction of which the player moves at which speed.

Line 20 provides an else statement.

Line 21 changes the horizontal velocity to the result of the "move toward()" function. This is a built-in Godot function that will gradually change the value of "velocity.x" towards "0". SPEED is also mentioned, because this determines how fast the horizontal velocity reaches 0. This means that when no input is given, the player will come to a stop.

While coding this part, I encountered a problem. Godot only had the command labels for the arrow keys, which were labelled as ui_up/down/left/right. So, to solve this problem, I had to create custom key binds for the controls I wanted. I did this by going to project settings, navigating to input map, and from that point I created a new action, named it, and entered the input to create the new custom key bind.



I then referenced it in my code, which can be recognised “move_up/down/left/right” as well as “spacebar”.

Now that the movement had fully been created for the player, along with working physics. I ran the program to test it. Although, as soon as I tested it, the player started to immediately fall off the screen. I deduced that this happened because physics were present, and the player had nothing to stand on. So, I created a new node, CollisionShape2D, and created it as a rectangle, placed it under the player, and ran the program. This fixed the problem, and I could now move about freely.

However, now there was a new problem. While moving, the player could easily go off the screen and the player would not be able to see it and therefore make accurate movements. I easily solved this by dragging the camera node under the player node to make it a child node. I did this so that the camera would follow the player sprite’s movements, preventing the player from ever being able to move off the screen.

I tweaked the value of the SPEED & JUMP_VELOCITY to my liking after.

Now the movement has been finally created and tested. The testing of the movement can be found in the video file labelled “**1. basic movement test**”. In this video, you can see the progress of the idle animation for the sprite and the movement. This video fulfils tests 1 through 10 from my test table.

My focus now went to the design of the level itself.

Better Movement

A few weeks later, I realised that my code was not organised and clear, neither was it as efficient as possible. This was because a large portion of the code was built in and provided by the Godot 2D basic movement library. I then chose to rewrite all my code myself, making it much better. I went through this by putting each feature/function in its own procedure. I used this modular design because it is easier for testing as I can tweak and delete separate features without affecting any others. This improves the readability,

modularity and scalability of the entire program. These are advantages of decomposition.

```
15  #Calls each movement function in this function so that they are always active and being called
16  func _physics_process(delta):
17      >| get_direction()
18      >| apply_gravity(delta)
19      >| handle_movement(get_direction())
20      >| handle_jump()
21      >| handle_wallClimbing()
22      >| handle_crouchWalk(get_direction())
23      >| handle_wallJump(get_direction())
24      >| handle_animations(get_direction())
25      >| move_and_slide()
```

Line 16 creates a function called `_physics_process` which runs automatically after each frame of the game running, so long as it is part of the script also being run (which it is). It essentially shares the functionality of a “while True” loop. It takes in the parameter “delta” which gives the time since the last `_physics_process` call, allowing frame-rate-independent movement by adjusting speeds accordingly. This function is where I will be calling and initialising the rest of my movement functions.

```
43  #function retrieving the players direction
44  func get_direction():
45      >| #retrieves integer value (-1, 0, 1) based off left/right being pressed
46      >| var direction = Input.get_axis("move_left", "move_right")
47      >| return direction
```

On Line 46, I create a variable called `direction`. I created this because this will be used as a parameter in several movement functions later. I use the function “`get_axis`” to get a x axis value of either 1, 0 or -1, depending on whether the “`move_right`” button or the “`move_left`” button (either D or A) is pressed. Later on, I decided to put this process in its own function, which is shown above. All other functions that take `direction` in as a parameter would change to the name of the function that retrieves the direction.

```
49  #creates the gravity function
50  func apply_gravity(delta):
51      >| #checks if player is in mid-air only
52      >| if not is_on_floor() and Global.onWall == false:
53          >| #changes vertical velocity to go down at speed of time (delta) X gravity
54          >| velocity.y += gravity * delta
```

This is my first movement function which applies gravity to the player. It takes in the `delta` variable as a parameter to ensure smooth movement which also is in time. It checks if the player is not on the floor, and if so, the vertical velocity is the gravity value of gravity (provided by Godot) multiplied by the time so that the movement is in time with smooth movement.

```

58 #creates player jumping function
59 func handle_jump():
60     #can only jump if on ground
61     if is_on_floor():
62         #checks for jump keys being pressed
63         if Input.is_action_pressed("spacebar") or Input.is_action_pressed("move_up"):
64             velocity.y = JUMP_VELOCITY

```

This is the jump function. It checks if the player is on the floor, because that is the only place where they can normally jump, and if so, it then checks if either jump buttons are pressed (either spacebar or W). If this is true, then it sets the vertical velocity to the constant of JUMP_VELOCITY.

```

66 #creates player moving function
67 func handle_movement(direction):
68     #if direction has any value apart from 0 (i.e. moving anywhere at all)
69     if direction:
70         #moves in the direction being pressed at the speed of SPEED
71         velocity.x = direction * SPEED
72     else:
73         #slows down player until horizontal speed = 0, at the speed of SPEED
74         velocity.x = move_toward(velocity.x, 0, SPEED)

```

This is the main running/movement function. It takes in the direction parameter that was made earlier. Line 69 checks if direction has a value, which essentially checks if either movement keys are current being pressed. If they are, the horizontal velocity is set to the SPEED constant multiplied by the direction which ensures that the player goes in a direction and the right direction. Once the movement keys have stopped being pressed, the “else” statement runs and changes the horizontal velocity so that the value goes to 0 which slows down the player. In the function, there are 3 values. The first value represents what is being changed, in this case it is the velocity. The second one represents what it is being changed to, which is 0. The third value represents the speed of which the velocity moves towards 0 (how fast the player slows down).

```

90  ▾ func handle_wallClimbing():
91  ▾ >|    if Input.is_action_pressed("shift"):
92  ▾ >|    >|    if is_on_wall():
93  ▾ >|    >|    >|    if Input.is_action_pressed("move_up"):
94  ▾ >|    >|    >|    >|    velocity.y = WALLJUMP_VELOCITY
95  ▾ >|    >|    >|    elif Input.is_action_pressed("move_down"):
96  ▾ >|    >|    >|    >|    velocity.y = -WALLJUMP_VELOCITY
97  ▾ >|    >|    >|    else:
98  ▾ >|    >|    >|    >|    velocity.y = 0

```

This is the wall climbing function which allows for the player to climb up walls in the game. It doesn't need to take in any direction parameters because it only affects the vertical velocity. All other velocities/movements are handled in other functions. The function checks if the player is on the wall to climb and if they are pressing shift. I put the shift button requirement because otherwise the player would always climb even if they are against a 1 block high wall, which I didn't want. I also wanted to input a climb button so it is easier for the player to understand how to climb. The function then checks if W is being pressed, in which case the vertical velocity will be equal to the JUMP_VELOCITY constant, which causes the player to move upwards. If S is being pressed, then the vertical velocity will be set to the negative of JUMP_VELOCITY, causing the player to move downwards. And if neither W or S are being pressed, but SHIFT is and the player is on a wall, the player's velocity is 0, cancelling out the gravity function and giving the impression that the player is grabbing onto the wall and staying there.

```

119 ▾ func handle_wallJump():
120 ▾ >|    if is_on_wall() and Input.is_action_pressed("shift"):
121 ▾ >|    >|    if Input.is_action_just_pressed("spacebar"):
122 ▾ >|    >|    >|    velocity.y = WALLJUMP_VELOCITY * 2.5
123 ▾ >|    >|    >|    velocity.x = WALLJUMP_PUSHBACK

```

This is the wall jumping function. This acts as another method of mobility up walls, so that the player doesn't find the wall climbing feature too boring or slow; this is to add variety to the movement. It takes in the direction as a parameter, and then checks if the player is on the wall as this is the only place where the wall jump can occur. It then checks if the keybind for the wall jump is pressed (spacebar), and if so, the horizontal velocity is set to the opposite direction that the player is facing multiplied by the WALLJUMP_PUSHBACK. This causes the player to be pushed off the wall slightly to provide some resistance to the jump. The vertical velocity is then set to the value of WALLJUMP_VELOCITY multiplied by 2.5. This is to provide a faster method of moving upwards and to prevent the player getting bored of the original vertical movement.

```

132     #creates crouching
133     func handle_crouchWalk(direction):
134         #checks is player is pressing S
135         if Input.is_action_pressed("move_down"):
136             #can only crouch if on ground
137             if is_on_floor():
138                 #makes them slow down
139                 velocity.x = (direction * SPEED) / 2
140                 velocity.y = velocity.y / 1.5

```

This is the crouch walk function, which acts like crouching/shifting in Minecraft; slowing down the players movements if they wish to. The function takes the direction in as a parameter, then checks if the player is on the floor. If so, their running speed is halved and their jump velocity is divided by 1.5.

This is all the code for now. It will be updated with new features later.

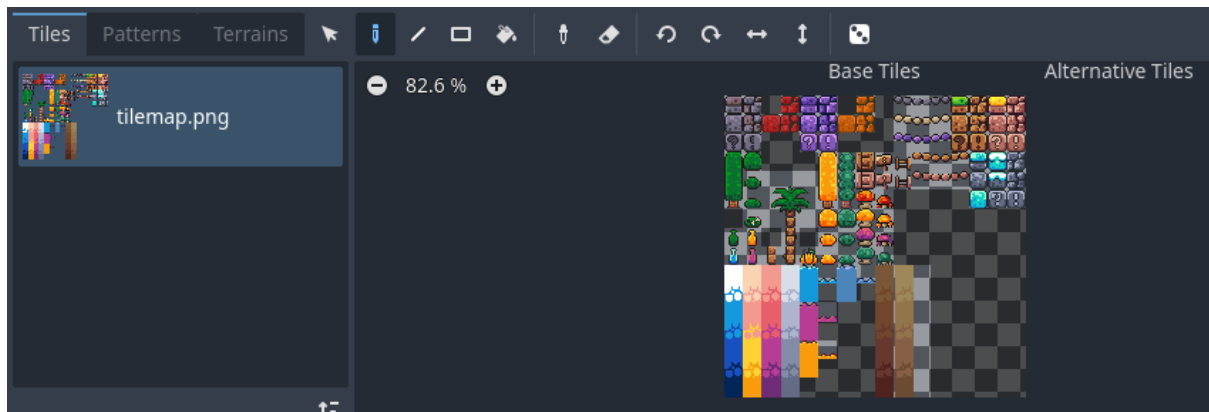
I was advised to implement my code using OOP techniques such as dedicated classes, but because I found very little use for it, because the use and behaviour of a class is similar the scripts of Godot already. So, I opted for modular movement functions.

Level Design

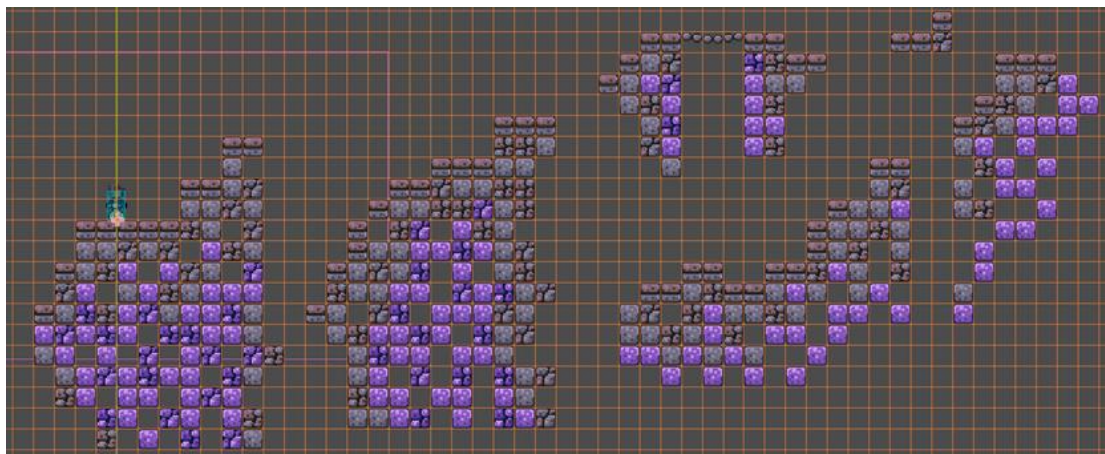
Before I began developing the game. I watched a video tutorial on how to use Godot to create a 2D platformer, so some of my inspiration and methods have been retrieved from this video by Brackeys <https://www.youtube.com/watch?v=LOhfqjmasi0&t=1132s>.

I also downloaded the resources that I would use while designing the levels from <https://brackeysgames.itch.io/brackeys-platformer-bundle>. These resources are CC0, so therefore I am free to use these and call it my own work. After downloading them, I then recoloured and redesigned the tiles myself.

I moved on by first deleting the CollisionShape2D node in the game scene, as this was only used for testing movement, so I created a new node in the game scene called TileMap. This is a node that essentially allows me to draw tiles that act as the floor for the player. I specifically used this node so that it would make the creation for the design of my level much easier and faster. I initiated the TileMap by going to the Inspector sidebar of the node and created a new TileSet. Pressing this then opens a new tab at the bottom of the application, which I navigated to TileSet. From here, I dragged in my downloaded TileMap.png file from my assets. Godot is able to create individual tiles automatically from the image, so it split the .png file into all the tiles I would be drawing with. From this point, I navigated to the TileMap tab, and now all of the tiles were shown to me.

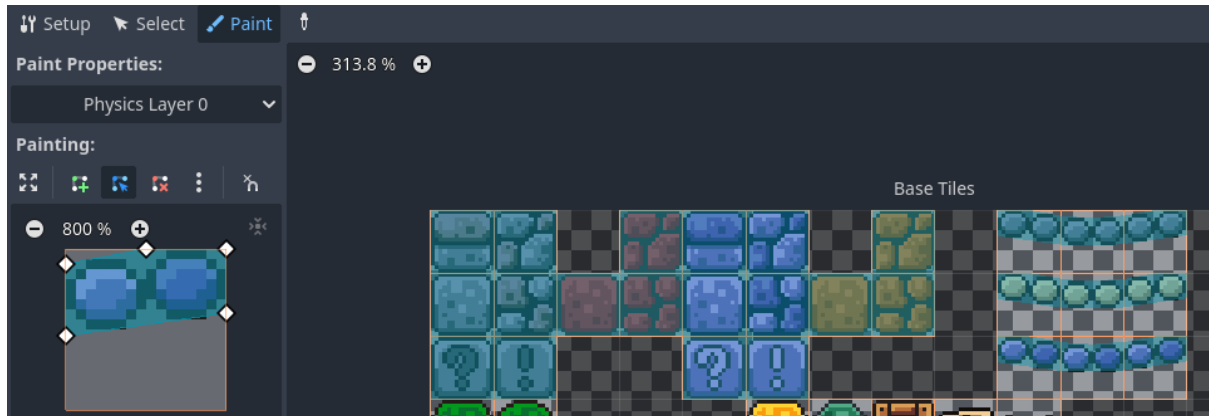


I could now select the pencil tool at the top of the TileMap tab and I could now start designing my level.



I ran the program then, and I immediately fell down infinitely. I realised that this was because there was no actual block there that the physics engine could affect; it was only tiles that cannot be affected by physics. I solved this problem by again going to the Inspector tab of the TileMapLayer node, opening the drop-down menu of TileSet, navigating to physics layers, and creating a new element. This creates a layer that can be applied to the tiles so that they become physical and so therefore can be affected by the physics engine.

Now that that was created, I went back to the TileSet tab and clicked the sub-tab Paint. I selected the Paint Properties drop-down menu and chose my Physics Layer 0. After this layer is selected, it can be selected to specific tiles so that physics will act on those tiles.



In this screenshot above, you can see a specific tile having a custom shape of the physics layer put on it. This is a part of a bridge, and because the tile texture is so short, the entire tile doesn't need to be affected by physics, so I resized it using the white points that are visible.

Now that the physics layer has been added to the TileMap, I ran the program and I could run and jump around my level. This corresponds and fulfils Success Criteria 23 & 24. This level design was shortly created as a prototype just so I could test the movement and if the physics worked. It will be improved and added onto later on in the development process.

Foreground

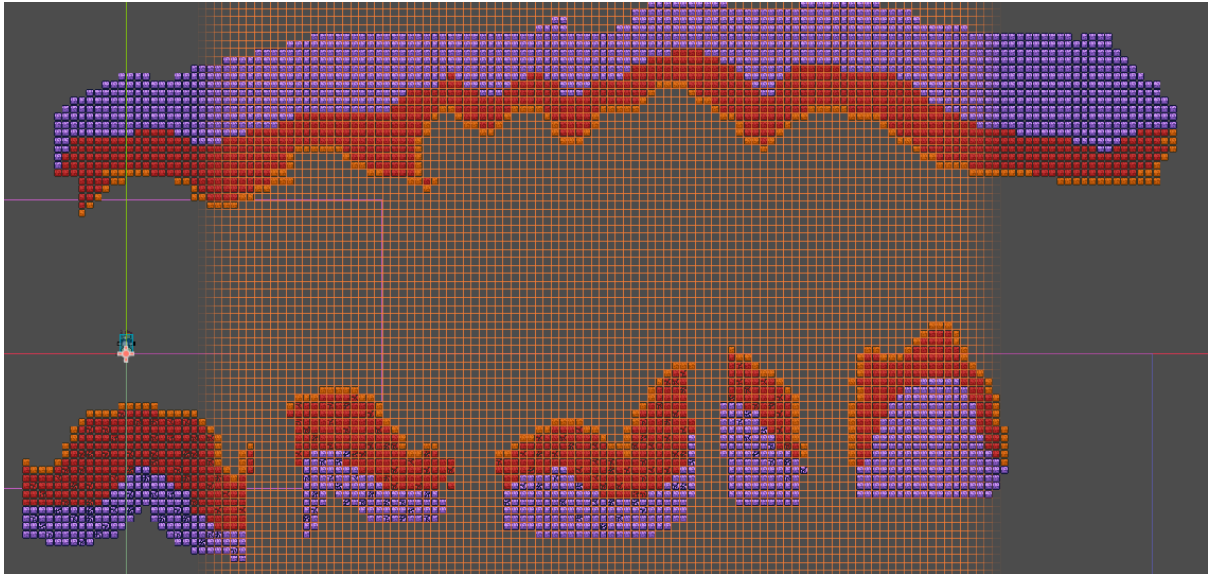
I wanted to include foreground to improve the layering of the game design; to stop it feeling so flat while playing.

While I tried to make it stand out a lot more, I ended up creating smaller versions of the tiles I used in the obstacles but recoloured. Throughout the design of both obstacles and the foreground, I tried to make the design as similar as possible to my original screen designs.



This is the prototype design for the foreground details. These blocks will not have physics, because the player will not be able to interact with the foreground; it is only there for visual design and to make the level look more appealing. To make it, I used the same tools that I used for the obstacles, just without the physics layering process. Again, this will be updated and made larger as I create more of the level itself.

Originally, I wanted the foreground to move along with the player so that I would only need to design a small foreground that wouldn't be dynamic. I would implement this very simply; putting the TileMapLayer node under the player node so that it would follow the player's movements. But when I ran it to test it, it looked horrible. This was because I had previously turned on position smoothing in the Inspector tab of the camera node. This led to the foreground having position smoothing as well, which didn't look natural or right at all. So, I decided to just design the foreground as normal. It's not the most efficient method, but it was the one that worked the best without sacrificing on relevant details.



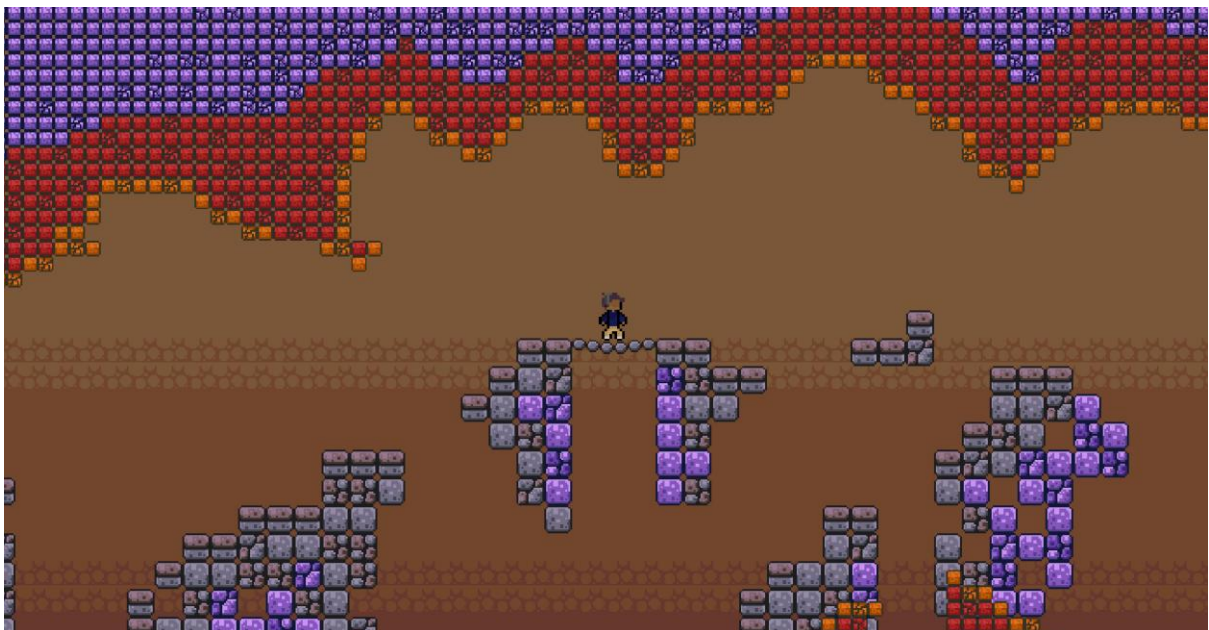
I then ran the game again, testing the progress since the last test. The results can be seen in the video labelled “**2. obstacles&foreground movement test**”.

Background

The next thing to make was the task of implementing a background for the level. I went through this by adding another TileMapLayer. First, I called it TileMapBackground, then I enabled the TileSet and dragged in my tilemap file which stored all the textures. As I downloaded this original tilemap, there were sky textures included. I recoloured these to fit my theme and resemble the design screens I had created in the Design process. When I implemented these colours, however, they looked too bright/not dark enough and the background looked out of place. With all these nodes, it should be noted that I had to rearrange the layout of each node in the scene, so that certain things appeared behind other certain things etc. I have also done this with all other nodes I have put into the game scene (pauseMenu, player etc.) The screenshot on the left was the first sample I came up with, with colours directly from the design screens. When I implemented this, it looked out of place and neither I nor my stakeholder didn't like it when I showed it to him.



It looks more like a sandy cave than a hot, lava cave, which is the theme I am aiming for. So, I redid the colours of the background, making them slightly darker and more saturated, which led to the final design (right screenshot)



In this design, all the colours look a lot more blended together, while the background looks dark enough to resemble a cave, while still keeping the game vibrant through the foreground, which I'm aiming for.

Interview

At this point, I interviewed my stakeholder on their opinion on the design of the level, while comparing it to my screen designs.

Adham

What are your brief thoughts about the design of the level?

It is good and it looks nice, but I don't know if the purple in the grey stones look right.

Do you think it resembles my screen designs nicely?

It resembles the screen designs nicely and the actual design of the level is even more detailed with the cracks and other stuff.

Are the obstacles/platforms challenging while remaining enjoyable (Success Criteria 5)?

Yes, they look good and are pretty manageable for the player. Keep them easier to do until the player is used to the controls and then make it more challenging.

Does the foreground present a perspective to the level to where it resembles a volcano-based cave or lava cave?

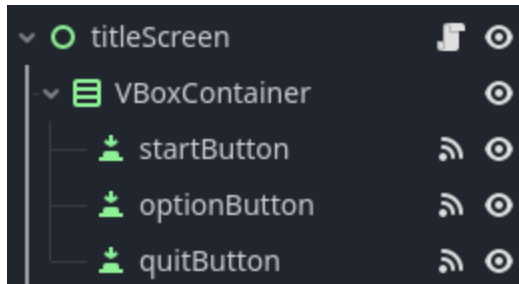
I think it resembles a volcano cave nicely but I think you should add some rocks to make it more realistic in the future. But for right now it looks good and I like it.

Does the foreground protrude too much into the obstacles/platforms too much to the point where it may obstruct the players gameplay?

There are some situations where some people may think that the foreground is lava, so they can't step there and avoid that area, but in reality, that's the way that they're supposed to go. And the same thing goes for some people thinking that there are blocks there so they must be able to be stood on, but in reality, the player cannot stand on them and they will die. But you have designed the foreground to avoid this problem a lot to not really.

Title Screen

Now the game is runnable and playable, I focused on the main menu. This follows the first of my success criteria, which stated “Include a start screen on startup” (1). Additionally, as per success criteria 20, all menu screens, including this one, will be fully navigational by mouse. I started with a basic foundation; 3 buttons for Play, Options and Quit. I implemented this through Button nodes, which I added as children of a VBoxContainer node.



I tweaked the visual settings of the buttons and centred them using Anchors Presets in the Inspector tab of the VBoxContainer and Button nodes. I then added the text to each button.



I then created a script for the scene called “titleScreen.gd”. Before actually coding, I linked the button nodes to the script. I did this by going to the Node tab, clicking pressed() and linking the node to the script of the scene.

```

1  extends Control
2
3  #when start button is pressed
4  func _on_start_button_pressed() -> void:
5      #changes to game scene
6      get_tree().change_scene_to_file("res://scenes/game.tscn")
7
8  #when settings button is pressed
9  func _on_option_button_pressed() -> void:
10     #player is not in playing state (came from title screen, not game scene)
11     Global.playing = false
12     #changes to settings scene
13     get_tree().change_scene_to_file("res://scenes/setting_screen.tscn")
14
15     #when quit button is pressed
16     func _on_quit_button_pressed() -> void:
17         #quits game
18         get_tree().quit()

```

Line 1 connects the titleScreen main node to the script.

Line 4 creates a function that checks if the Start button is pressed

Line 5 contains the contents of the function and changes the scene to the game scene if the Play button is pressed

Line 7 creates another function for the Options button.

Line 8 sets the global variable to false to indicate that the player is not in the game and therefore not playing the game yet. This will be explained why it is implemented later.

Line 9 changes the scene to the setting_screen scene

Line 11 creates another function for the Quit button

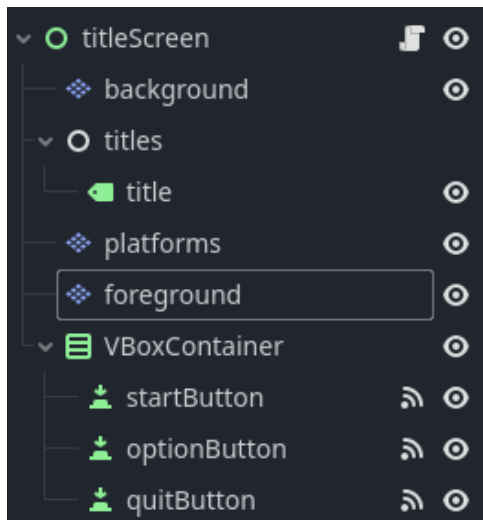
Line 12 is the contents of the function and quits the game is Quit is pressed.

When running the game after this had been written, the game scene loaded, skipping the title screen. I solved this problem by going into Project Settings, going to the RUN tab in the sidebar, and selecting the titleScreen scene as the Main Scene. This means that the program will run the titleScreen first because it is now the main scene.

Now when running the game, the title screen is the first thing to automatically pop up.

Next, I wanted to make the title screen look like it did in my design (Screen Designs). I thought about this for a while, about whether I should put an image of the background directly from my screen designs into the game, or whether I should use TileMaps (these are resources that will be explained later). But after consulting my stakeholders, I decided to use TileMaps, as this would look almost identical to the actual game, and this was the point I was making with the original screen designs.

Making the design was quite easy. Because I wanted it to be the same design as the first level itself, all I had to do was this:



I created 3 TileMap nodes for the background, foreground and platforms. In each, I created the TileSet just like I did in the game scene.

I then went back to the game scene, copied the design of each TileMap, and pasted it into the title scene. I had to tweak the scale and position of the TileMaps, but eventually it did look how I wanted it to.

Next, I wanted to display the title of the game, RUN, just like I did in the screen designs. To do this, I created a new label node, which I then positioned in the centre, above the buttons and wrote RUN. I then changed the colour of the label by going to the Inspector tab, scrolling down to Theme Overrides changing the font colour and font shadow colour to yellow. I also did this for the buttons. This is now the result of the title screen.

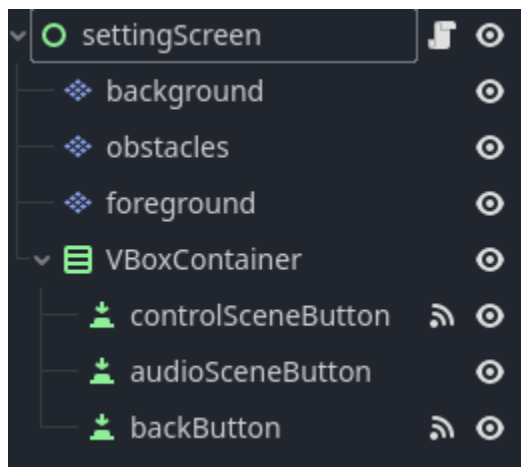


Settings Screen

The next interface to make was the options screen. This scene would consist of a control screen, which lists all the controls and key binds in the game for the player. It would also have an audio scene, which the player can control the master volume, the music and SFX volume. It would also have a back button, which takes them back to the title screen.

The options button in the title screen already works because I have coded it in the script for the title_screen, so when clicked the program navigates to the options screen.

In the new settings scene, I made the root node a user interface, because it is a menu. Next, I created the nodes similar to the title screen. I created a VBoxContainer node and in it, I put 3 button nodes for Controls, Audio & back.



I used a VBoxContainer in all the menu scenes just because it means that I can move all buttons as one, making the process a lot easier and organised.

This is what the screen looks like so far.



Next, I implemented the functionality of the scene. I did this by creating a new script connected to the root node, then adding functions that would run if any of the buttons were pressed, just like in the title screen.

```
▼ func _on_back_button_pressed() -> void:
    >| #checks if player is playing
    ▼ >| if Global.playing == true:
        >| >| get_tree().change_scene_to_file("res://scenes/game.tscn")
    ▼ >| else:
        >| >| #if they're not playing, take player back to title screen
        >| >| get_tree().change_scene_to_file("res://scenes/title_screen.tscn")
        >| >|
    ▼ func _on_control_scene_button_pressed() -> void:
        >| #changes to controls screen
        >| get_tree().change_scene_to_file("res://scenes/control_screen.tscn")

    ▼ func _on_audio_scene_button_pressed() -> void:
        >| #changes to audio screen
        >| get_tree().change_scene_to_file("res://scenes/audio_screen.tscn")
```

The first function is for the back button. If this is pressed, the program checks if the global variable playing is true, which checks if the player is coming from the pause menu or the main menu. This will be explained in depth when it comes to the development of the pause menu itself.

If playing is true, then it changes the scene back to the game scene (because that must be where the player had come from).

If not, then the player has not come from the game, and the only other place they could've navigated from is the main menu, so it takes the player to the title screen.

The second function is for the control screen, and changes the scene to the control screen if the button is pressed.

The third function is for the audio screen, and changes the scene to the audio screen if the button is pressed.

Now the functionality of the scene has been made, now I went to making it look like the screen designs.

I wanted to make it look like the title screen, because that's the point I was going for in my design. So, with the 3 TileMaps I made earlier, I created their necessary TileSets, and copied the design from the title screen.

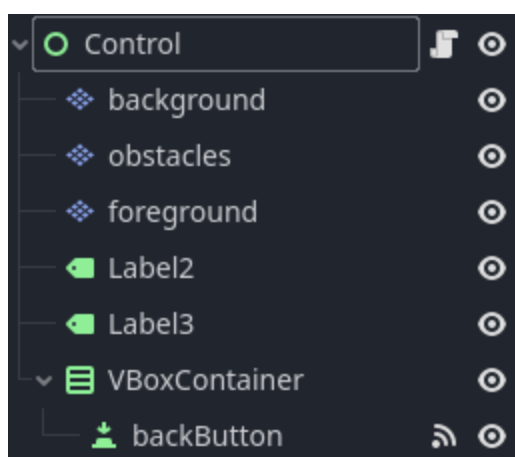


I also changed the font colour of the buttons to the same colour as the title in the title scene. I did this by going to the Inspector tab of each button node, going to Theme Overrides, Colours, and setting all font colours to yellow.

Control Screen

For the control screen, all I wanted to do was display all the possible controls that the player can use in the game. This would mainly include normal movement and climbing movement.

To do this, I created 2 label nodes; 1 for displaying the keybinds; 1 for displaying the action. I also put a back button which is identical to the one in the settings scene.



I then set the text for the labels, as well as the colours, and then centred them and moved them a certain distance away from each other to make them line up but separate.

W	Jump
A	Run left
S	Crouch
D	Run right
Spacebar	Jump
Shift + A	Climb left wall
Shift + D	Climb right wall
Climb wall + W	Climb up wall
Climb wall + S	Climb down wall
Climb wall + Spacebar	Wall jump

W	Jump
A	Run left
S	Crouch
D	Run right
Spacebar	Jump
Shift + A	Climb left wall
Shift + D	Climb right wall
Climb wall + W	Climb up wall
Climb wall + S	Climb down wall
Climb wall + Spacebar	Wall jump

back

Then I made the back button. To do this, I just duplicated the back button from the settings scene and adjusted the position accordingly.

```
▼ ↗ pressed()  
  ↳ ↗ ../../ :: _on_back_button_pressed()
```

```
3 ▼ func _on_back_button_pressed() -> void:  
4   > get_tree().change_scene_to_file("res://scenes/setting_screen.tscn")
```

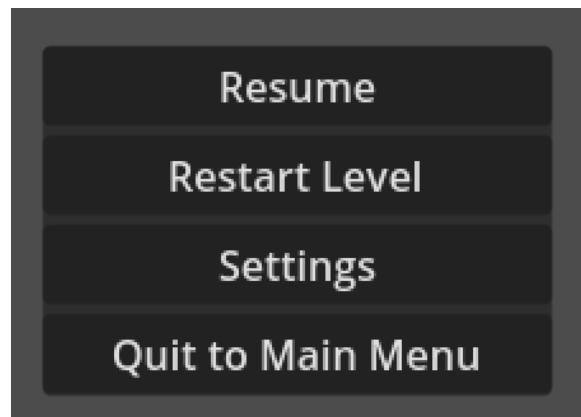
Next was the TileMaps. This was simple, I just had to do the same thing I had done in the title and settings scene.



I was a bit worried about how easy this would be to read, because of the lack of contrast between the text and background. But I consulted my stakeholders, 2 of which wear glasses themselves, and they assured me it was fine to read and looked good.

Pause Menu

Now the game is playable, it is time to add some essential features of games across the market. The first one I decided to implement was the pause menu, following my 9th success criteria. I loosely followed an online tutorial on the basics of how to construct a pause menu (<https://www.youtube.com/watch?v=e9-WQg1yMCY&t=2s>). To do this, I started with creating a new User Interface scene, renaming the Control node to pauseMenu. I then created a VBoxContainer node, in order to create a box node that could store the button nodes. And finally, 4 button nodes as children nodes of that. I renamed the buttons to Resume, Restart Level, Settings and Quit to Main Menu in the "Text" box in the Inspector tab of each node.



After the nodes have been created, I can create a script for the scene.

```
3  #called when the game launches immediately
4  ▼ func _ready():
5      » #hides the pausemenu so it isnt constantly on the screen
6      » hide()
7
8  ▼ func pause():
9      » #sets paused state to true
10     » Global.paused = true
11     » #pauses the game by setting the timescale to 0
12     » Engine.time_scale = 0
13     » #shows the pause menu
14     » show()
15
16  ▼ func resume():
17     » #sets paused state to false
18     » Global.paused = false
19     » #resets time scale back to default value
20     » Engine.time_scale = 1
21     » #hides the pause menu
22     » hide()
```

Line 4 creates a function called `_ready`. This function runs as soon as the game runs. Line 6 is the contents of the function. All this does is hide the parent node `pauseMenu`, making it invisible. It also affects the children nodes as well, so that all of them are hidden.

I have included the `show()` and `hide()` functions later on to solve a problem I was having while testing the program. Once the game scene opened, the `pauseMenu` was stuck on the screen constantly; it would not move, it would not be interactable with; it would not vanish when `Esc` is pressed. So, I implemented the `show()` and `hide()` functions in order to make them vanish and appear when they are needed and when they are not.

Line 8 creates a function called `pause`

Line 10 sets the global paused state variable to `true`, which will be used when checking for `Esc` being pressed.

Line 12 accesses the Godot Engine properties and sets the Engine Time Scale in the game to 0. This freezes the time in the game, essentially pausing it. The actual player can also no longer control the player, because the game is paused.

Line 14 shows the pause menu HUD, for the player to interact with while the game is paused.

Line 16 creates a function called `resume`

Line 18 sets the paused state to `false`

Line 20 sets the engine's time scale back to 1 (default value) which resets the time scale in the game, resuming it. Line 22 then hides the `pauseMenu` node and all of its children, which is the paused menu.

```
24  ▾ func EscPressed():
25  ▾ ▸   if Input.is_action_just_pressed("esc") and Global.paused == false:
26  ▾ ▸   ▸   #if the game isnt paused and player presses esc
27  ▸   ▸   #calls pause function
28  ▸   ▸   pause()
29  ▾ ▸   elif Input.is_action_just_pressed("esc") and Global.paused == true:
30  ▾ ▸   ▸   #can unpause the game if player presses esc again
31  ▸   ▸   #calls resume function
32  ▸   ▸   resume()
33  ▸   ▸
34  #called every frame of the game
35  ▾ func _process(_delta):
36  ▸   #is always checking for esc being pressed
37  ▸   EscPressed()
```

Line 24 creates a function called `EscPressed`. This will check if the escape key is pressed, as this is the key bind that the player will press to pause the game.

Line 25 checks if `Esc` is pressed and if the game is not paused. If these conditions are met, line 22 runs the `pause` function, which pauses the game.

Line 29 checks if `Esc` is pressed and if the game is paused. If these conditions are met, line 32 runs the `resume` function, which resumes the game.

Line 35 creates a function called `_process`. The only purpose of this function is to call the `EscPressed` function so that the program is constantly checking if the keybind `Esc` has been pressed.

By default, `paused` will be set to `true`, so if the player is playing, `pause` will be run. If the player is already in the paused menu, then `paused` will have been set to `true` already, so the `resume` function will be run.

```
28  #button functions
29  func _on_resume_pressed() -> void:
30      resume()
31
32  func _on_restart_pressed() -> void:
33      #unpauses game
34      resume()
35      #resets game scene
36      get_tree().reload_current_scene()
37
38  func _on_settings_pressed() -> void:
39      #player is currently playing
40      Global.playing = true
41      #resumes timescale for music etc.
42      resume()
43      get_tree().change_scene_to_file("res://scenes/setting_screen.tscn")
44
45  func _on_quit_pressed() -> void:
46      #player is no longer playing
47      Global.playing = false
48      resume()
49      get_tree().change_scene_to_file("res://scenes/title_screen.tscn")
```

Line 29 creates a function that links the `Resume` button to the script, so that when it is pressed, line 30 is run and the game resumes through the `resume` function.

Line 32 creates a function linking the `Restart Level` button to the script, so that it runs the `resume` function as well as reloading the current scene, which restarts the level by resetting the player's position and resetting any other changing factors in the level. The `resume` function must be called here in order to automatically resume the game, otherwise once the level is reloaded, the player will still be in the menu, so I called the `resume` function for ease of use in the game.

Line 38 links the `Settings` button to the script so that when it is pressed, the global variable `playing` is set to `true` (I will discuss this in further detail later) in Line 40. The game is then set to not paused in line 42. I did this to solve a problem that I had while clicking buttons in the `pauseMenu`. While `paused = true`, Godot does not allow for any interactivity with any buttons or controls. So, this means that the player cannot press any buttons to go forward or back, making them stuck in the menu. So, I unpause the game as they are going into another menu, therefore it doesn't affect the timescale of

the level because they are leaving the game scene and they can now interact with the menu.

Line 43 changes the scene to the settings scene.

Line 45 links the quit to main menu button to the script so that when it is pressed, `playing = false` to indicate that the player is no longer playing and will no longer be playing. Line 47 sets the game to unpaused so that the user can interact with the buttons and menus. Line 49 then switches the scene to the main menu scene.

A problem I occurred here was that when the user goes to the settings scene from the pauseMenu, and then presses the back button in the settings scene, it would automatically and always take them back to the main menu and out of the game, regardless of whether the user came from the main menu or the game; the settings button was acting as another quit button. I consulted my stakeholders on how to solve this problem. Adham told me to simply create another settings scene for the pauseMenu and one for the main menu. Shazil told me to use a global variable called `playing` and an if statement to check where the player came from. So, I attempted to implement this method because it seemed more of an efficient solution, which can be seen through line 30 and 35 where I set `playing =`. This code is in the script for the pauseMenu scene. I then needed to refer to the variable through an if statement in the settingsScreen script which can be seen below:

```
3  ▾ func _on_back_button_pressed() -> void:
4    ▸ #checks if player is playing
5  ▾ ▸ if playing == true:
6    ▸ ▸ get_tree().change_scene_to_file("res://scenes/game.tscn")
7  ▾ ▸ else:
8    ▸ ▸ #if they're not playing, take player back to title screen
9    ▸ ▸ get_tree().change_scene_to_file("res://scenes/title_screen.tscn")
```

And in the PauseMenu script:

```
38 ▾ func _on_settings_pressed() -> void:
39   ▸ #player is currently playing
40   ▸ playing = true
41   ▸ #resumes timescale for music etc.
42   ▸ resume()
43   ▸ get_tree().change_scene_to_file("res://scenes/setting_screen.tscn")
44
45 ▾ func _on_quit_pressed() -> void:
46   ▸ #player is no longer playing
47   ▸ playing = false
48   ▸ resume()
49   ▸ get_tree().change_scene_to_file("res://scenes/title_screen.tscn")
```

There are errors in both screenshots of code which I will explain occur due to them not being declared in both scripts. The reason for this is that they are both local variables and their scope is limited to the script that each playing variable is in. I needed for the value of playing to be able to be accessed anywhere in the program, so I decided to make playing a global variable.

I went through this by creating a new script (that was not linked to any scene), named it Global.gd and wrote the code below:

```
3  var playing: bool = false
```

This creates a Boolean variable called playing and sets it to false.

I then went to Project Settings; Globals and I then added a new global script and selected the Global.gd script and naming the global script Global. I also checked the tickbox labelled Global Variable.

Now, I could refer to Global.playing and the value would be the same across the entire program. I then implemented this into my code.

```
38  func _on_settings_pressed() -> void:
39      #player is currently playing
40      Global.playing = true
41      #resumes timescale for music etc.
42      resume()
43      get_tree().change_scene_to_file("res://scenes/setting_screen.tscn")
44
45  func _on_quit_pressed() -> void:
46      #player is no longer playing
47      Global.playing = false
48      resume()
49      get_tree().change_scene_to_file("res://scenes/title_screen.tscn")
```

```
29  func _on_back_button_pressed() -> void:
30      #checks if player is playing
31      if Global.playing == true:
32          #takes them back to the game scene
33          get_tree().change_scene_to_file("res://scenes/game.tscn")
34      else:
35          #if they're not playing, take player back to title screen
36          get_tree().change_scene_to_file("res://scenes/title_screen.tscn")
```

Now, the top code will set the global variable playing to true if settings is pressed from the pause menu. This will then cause the if statement in the bottom code to run successfully and it will change the scene to the game scene again. If the player is accessing the settings screen from the main menu, then Global.playing will be unaffected and it will contain its default value, False. This will then trigger the else statement to run and change the scene back to the main menu scene when the back button is pressed. This is a much more efficient solution to the problem, and once I consulted my stakeholders about my solution, they agreed. Ismail told me it was a

much more efficient solution than simply creating 2 settings scenes, which would take up more storage.

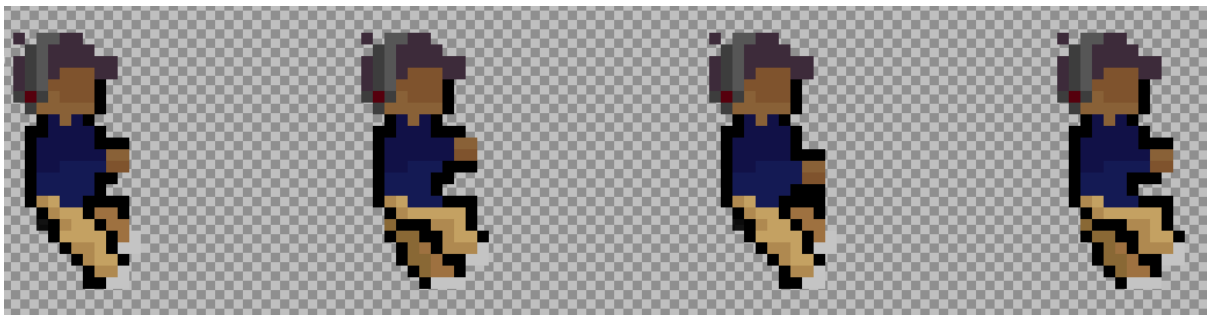
I then tested this process to see if it worked and operated the way my stakeholders wanted. The test can be seen in the video labelled “**3. pause menu test**”. This video demonstrates test 11.

Player Animations

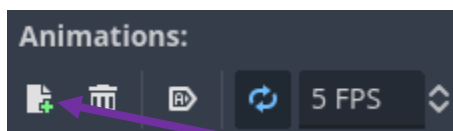
Climbing

This animation will be used when the player is moving up or down a wall.

In the sprite map file, I expanded the canvas size so that I could start designing the sprites for climbing. I kept the head in the same place so that I could make the animation states flow from one to the other as smoothly as possible.



These are the 4 sprites I designed that represent the climbing animation. Each sprite contains one change from the last. I did this to keep the animation smooth with small gradual changes. The animation consists of the player’s legs shifting and the arms moving up and down.



I then implemented the sprites with the same method I implemented the idle sprites with. But this time, I added a new animation to the `AnimatedSprite2D` node. I did this by navigating to the SpriteFrames tab at the bottom, and clicking “Add Animation”. I called it “climbing”. I then inputted the sprite map and chose my frames.



Now that the animation has been made, I tweaked it how I wanted.



I changed the FPS of the animation so that it would play a bit faster.

I made the animation loop infinitely.

I turned off autoplay, because otherwise the climbing animation would always play immediately as the game is launched. This action will be reserved for the idle animation.

Next, I needed to implement the animation in the game. I would utilise another function in the code to do this. The code will be shown later after all of the other animations.

Idle Climbing

This animation is included so it is used when the player is grabbing onto the wall, but not moving up or down so it is stationary.

I reused one of the sprites from the climbing animation (and only one because I want to make it appear like the player is stationary, and a moving animation would look wrong and out of place) so that I didn't have to waste time designing another one.

So, I created another animation in AnimatedSprite2D, calling it climbing idle. I used the exact same steps for the climbing animation, but this time I only selected one frame from the sprite map, and didn't tweak the animation, because they are irrelevant details here.

Falling

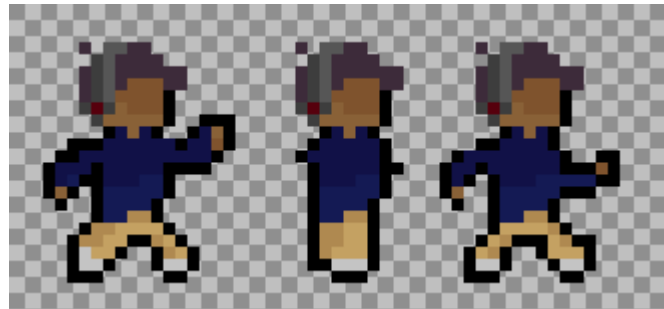


This animation will be used when the character is falling down from an area. This could be from a jump or falling off a platform.

In the animation, the only changes are the player's hands changing position very slightly, only by a few pixels, as well as the shoes. The headphones also slightly fall off his head, to reiterate the idea that the character is falling.

I implemented this animation the same way as the others, by choosing frames from the sprite map. I tweaked the animation and chose not to loop it because it wouldn't make sense for the headphones to reset back on his head again and again.

Running



This animation will be used when the player is simply running across a flat horizontal surface.

This is the original design for the running animation I had come up with. I put this into the program and while my stakeholders were testing this, they quickly realised that this does not look neither realistic nor visually appealing.

I then spent a few hours redesigning the animation, and I realised that highlighting which leg is the primary leg like I did in the climbing animation would be beneficial here. I also created many more frames which detailed how each arm and each leg switched place with each other, before coming out with the final design.

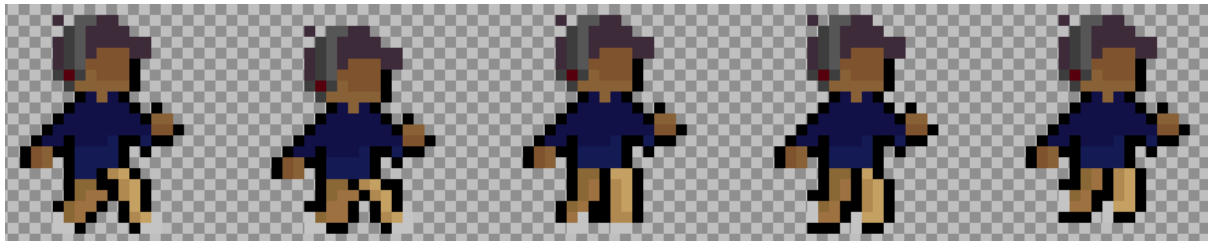


This animation is much more detailed and when running it, they found it to be much more realistic, fluid and visually appealing.

I implemented the animation using the same method as the other animations. I tweaked it so that the FPS was 10 (so the speed of the animation was suitable) and the animation would loop.

Jumping

Next was the jumping animation, which will be used whenever the player jumps.



This is the original design. When I implemented this animation and tested it, I found that I didn't like how it looked, due to the animation taking too long to run (because the first sprite is part of the running animation, and if it was taken away, then the animation playing would look out of place and wouldn't run smoothly) so by the time the animation fully played, the player has already jumped and is back on the floor. I then spent some time redesigning the sprites.



I came up with this simple design, which is very similar to the idle sprite, but with the arms flipped to give the impression that he is jumping. I found this as a much simpler solution for the jumping animation. And because it's so similar, it will run very smoothly. I then tested the animations, which can be seen in the video labelled "**4. Animations test**". This video was made once the function for animations was made. It shows the testing of tests 12 – 19.

After testing, I noticed that the falling animation was too sensitive; it played too quickly so that even when the player jumps, the entire animation plays, making it seem crowded and unrealistic. I fixed this problem by changing the first sprite to the jump sprite above and shifting the rest of the sprites once to the right.

This means that the jump animation will play first, then morph into the regular falling animation. I did this to smoothen out the transition between jumping and falling. On the first sprite, there is a "x 1.7". This causes the first sprite to play for 1.7 times the regular time it would play for. I tweaked it like this so that when the player is normally jumping on a flat surface, the regular falling animation wouldn't play at all to keep it simple and cleaner.

Idle Crouch

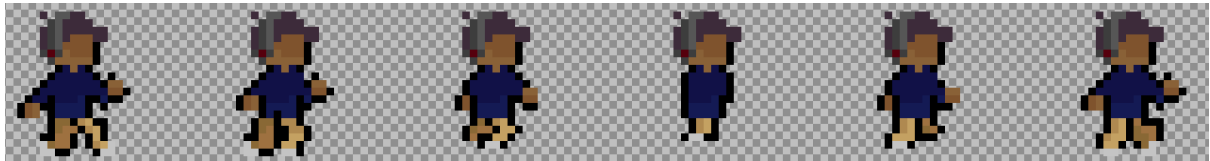
I wanted to include separate, custom sprites for the crouching function, so the variety for movement ability for the player is larger.



I wanted to keep the animation very simple, so I only made one change, making the sprite 1 pixel shorter. I felt that this change was big enough for the player to realise that the player is indeed crouching, but it isn't too big a change where the sprite looks out of place.

The crouching function needs a separate piece of code, because it has different versions of each animation (one normal, one while crouching). This will be shown in the code for the animation function later.

Crouch Running



This is another version of running, but this will be while crouching.

I changed the original running sprites so that they were also 1 pixel shorter, like the idle crouch animation, as well as bending the legs of the sprite more. I also tweaked the FPS so that it was much slower than the regular running animation, giving the impression that the player is moving slower.

I implemented this with the same method as the other animations.

Crouch Jumping

Again, this is another version of jumping, but this will be while crouching. The sprite for this animation is just the regular jumping one, but 1 pixel shorter.



I created a new animation in the AnimatedSprite2D, and created this animation, as I did for the others. Like the crouch idle sprite, this resembles the normal jumping sprite, just 1 pixel shorter.

After adding all the crouching animations and improving the falling animation, I tested them all again. The results can be found in the video labelled “**5. Animations test v2**”.

This video demonstrates tests 12 – 21.

I now moved on to implementing them in the game through a function.

```
11 @onready var animated_sprite = $AnimatedSprite2D
```

First, we have to be able to manipulate the AnimatedSprite2D node. To do this, I created a variable called `animated_sprite` and imported the actual node into the code so that I could manipulate it.

Now whenever I refer to `animated_sprite`, I am manipulating the state of the AnimatedSprite2D node.

To begin with, I was going to change the animation state every time that an action was run. For example, the line after the character jumps would've changed the animation to jump. But each statement kept crashing the entire game, so I eventually opted to put all animation handling in one function separate from everything else. I used this modular design because it is easier for testing as I can tweak and delete separate features without affecting any others.

```

142     #handles playing of all animations/states of player
143     ▾ func handle_animations(direction):
144         »     #figures out player direction
145         ▾ ▾ if direction < 0:
146             »     »     #flips the sprite into the direction it's moving/facing
147             »     »     animated_sprite.flip_h = true
148         ▾ ▾ elif direction > 0:
149             »     »     animated_sprite.flip_h = false

```

```

151     #if player's crouching
152     ▾ if Input.is_action_pressed("move_down") and is_on_floor():
153         »     #if player's only moving horizontally
154         ▾ ▾ if velocity.y == 0:
155             »     »     #if they're moving at all
156             ▾ ▾ ▾ if velocity.x != 0:
157                 »     »     »     #plays crouch running animation
158                 »     »     »     animated_sprite.play("crouch run")
159             ▾ ▾ else:
160                 »     »     »     #plays crouch idle, because player's not moving at all while crouching
161                 »     »     »     animated_sprite.play("crouch idle")
162             ▾ ▾ else:
163                 »     »     #if player's moving upwards while crouching
164             ▾ ▾ ▾ if velocity.y < 0:
165                 »     »     »     #plays crouch jump
166                 »     »     »     animated_sprite.play("crouch jump")
167         #if player is not crouching
168     ▾ else:
169         »     #if player's only moving horizontally
170     ▾ ▾ if velocity.y == 0:
171         »     »     #checks if player's on the ground

```

```

171     »     »     »     #checks if player's on the ground
172     ▾ ▾ ▾ if is_on_floor():
173         »     »     »     »     #if they're moving at all
174         ▾ ▾ ▾ ▾ if velocity.x != 0:
175             »     »     »     »     »     #plays run
176             »     »     »     »     »     animated_sprite.play("run")
177         ▾ ▾ ▾ ▾ else:
178             »     »     »     »     »     #plays idle, because they're not moving at all
179             »     »     »     »     »     animated_sprite.play("idle")
180             »     »     »     #if player's not on ground, then must be on wall
181         ▾ ▾ ▾ ▾ elif is_on_wall():
182             »     »     »     »     #vertical velocity is 0, and can only move vertically on walls, so is idle on a wall
183             »     »     »     »     animated_sprite.play("climb idle")
184             »     »     #if player's moving vertically
185         ▾ ▾ ▾ else:
186             »     »     »     #if player's climbing
187         ▾ ▾ ▾ ▾ if is_on_wall():
188             »     »     »     »     #plays climbing
189             »     »     »     »     animated_sprite.play("climb")
190         ▾ ▾ ▾ ▾ else:
191             »     »     »     »     #if player's moving upwards

```

```

191     >| >| >| >| #if player's moving upwards
192     >| >| >| >| if velocity.y < 0:
193     >| >| >| >| >| #plays jumping
194     >| >| >| >| >| animated_sprite.play("jump")
195     >| >| >| >| >| #if player's moving downwards
196     >| >| >| >| >| else:
197     >| >| >| >| >| #plays falling
198     >| >| >| >| >| animated_sprite.play("falling")

```

Line 143 creates the function that will control all the animations and when they play. It takes in the parameter of the direction that the player is currently facing.

Line 145 checks if the value of direction (can be either 1/right, 0/none, -1/left) is more than 0, in which case it will not flip the animated_sprite horizontally (meaning of _h) and keep the direction, because it is already facing the right way.

Line 148 does the opposite; flips the sprite towards the left, if the direction is less than 0 (implying it is -1) so that the player is facing left.

Line 152 runs an if statement, checking if S is being pressed and the player is currently on the floor. This deciphers whether the player wants to crouch or not.

Line 156 to 166 runs all the crouch actions.

Line 154 checks if the player is not moving upwards or downwards; its vertical velocity is 0.

If true, then the player is not jumping, and it can check the players horizontal velocity. If it's not equal to 0 (can be any other value) then it will play the crouch run animation, because the player is moving right and left only.

If false, then the player is not moving vertically or horizontally, so they are not moving at all. In this case, the crouch idle animation will run.

If the player's vertical velocity is not 0, then it checks if it is above 0, in which case it will play the crouch jump animation.

Line 168 onwards controls the regular movement animations. It uses the same if statements just without S being pressed.

Line 170 checks if the vertical velocity is 0. If true, then this means that the player is not moving up or down, or they are on the floor. To decipher between these 2, line 172 simply checks if the player is on the floor. If true, then it checks the horizontal velocity. If it is anything but 0, then the player is moving, therefore the run animation plays. If not, then the player is not moving at all, therefore the idle animation plays.

Although, if the player is not on the floor and not moving vertically, then the player can only be idle on a wall. This leads to line 95 making sure by checking if the player is on the wall and holding the climb key (shift). If true then the idle climbing animation plays. Line 97 runs if the players vertical velocity is not 0. This means that the players actions can only be between jumping, climbing or falling. We decipher between these options by checking if the player is on the wall and holding the climb key (shift). If true then the player is climbing and the animation for it plays.

If they are not on the wall, the only place the player can be is in the air, because line 97 onwards only handles animations off the ground (shown by the if statement checking vertical velocity in line 89). So, to decide between jumping and falling, the program checks if the vertical velocity is above 0. If it is above 0, then the player's vertical

position is rising, hence the player is jumping. If it is less than 0, then the player is falling.

The final result of the animations function can be seen in the video labelled “**5. Animations test v2**”.

Collectibles

Coins

In my success criteria, I state that I will include collectibles inside each level to act as a secondary goal for the player (to get all the coins). I first needed a texture for the coins. I needed to keep it small, but shiny and attractive to bring the players attention to it. I opted to download the coin texture and animation from <https://brackeysgames.itch.io/brackeys-platformer-bundle>. Again, all these resources are CC0, and free to use and distribute.

Now I can start creating the coins. I started by making a new scene, and making the root node and Area2D node. Then, as children of this, an AnimatedSprite2D, because this is what the player will see, and a CollisionShape2D node, because this is what the player will interact with. I implemented the animation using the same method as the player sprites earlier, by selecting the frames from the coin.png file and tweaking the animation to be right, such as enabling autoplay and loop. It should be known that for the coins, I am following a basic tutorial (<https://www.youtube.com/watch?v=LOhfqjmasi0>), but later I add some of my own features.

I then created a script for the root node, navigated to the node tab (for the root node) on the right of the program and selected `body_entered(body: Node2D)`.

```
▼  body_entered(body: Node2D)  
  └─>  . :: _on_body_entered()
```

I connected this function to the root node script, and a function was made linking these together.

```
5 ▼ func _on_body_entered(body: Node2D) -> void:  
6   >I queue_free()
```

Line 6 clears the queue, which essentially safely deletes the node that was just walked through. This gives the impression that the coin has been picked up by the player.

I put this into the main game scene by dragging it as a child scene, then moving it to wherever I wanted. I did this multiple times for multiple coins.

This was tested and it worked; the results can be seen in the video labelled “**6. coin collectible test**”.

But I wasn't satisfied with this. I felt like it was anticlimactic when the player collects the coins, and my success criteria said to include a small animation to reward the player

(21). So, I decided to create a collected animation that resembled the coin exploding when the player collects it.



This is the design that I came up with. It details the coin getting smaller, until a small explosion begins which gets bigger and spreads out.

I implemented the animation with the same method as the original, but without a loop and with autoplay disabled. I then wrote the line that would play the collected animation in the `body_entered` function.

```
3 @onready var coin_sprite = $AnimatedSprite2D
4
5 func _on_body_entered(body: Node2D) -> void:
6     >| coin_sprite.play("collected")
7     >| print("1 more coin!")
8     >| queue_free()
```

This would play the collected animation as soon as the coin was walked through, print the message ("1 more coin!") and then delete the node.

Running this provided an unexpected result, because as soon as the player walks through the coin, it would be deleted. I deduced that this was because the program doesn't wait until the animation is finished to run the next lines.

```
3 @onready var coin_sprite = $AnimatedSprite2D
4
5 func _on_body_entered(body: Node2D) -> void:
6     >| coin_sprite.play("collected")
7     >| print("1 more coin!")
8
9 func _on_animated_sprite_2d_animation_finished() -> void:
10    >| queue_free()
```

I tried many other methods; none working. But fortunately, I found that there is a function specifically tailored for this situation. In the node tab of the `AnimatedSprite2D`, there is an `animation_finished()` function. This function would check if there is currently an animation playing, wait until it is done, then run the contents of the function. I connected this to the script, then wrote `queue_free()` in it, and deleted that line in the `_on_body_entered` function.

This solution would play the collected animation when the coin is passed through by a player, print ("1 more coin!") to the user, and wait until the collected animation is finished, then safely delete the coin node.

Running this solution did exactly what I wanted it to. The results are shown in the video labelled "**7. coin collectible test v2**".

Score

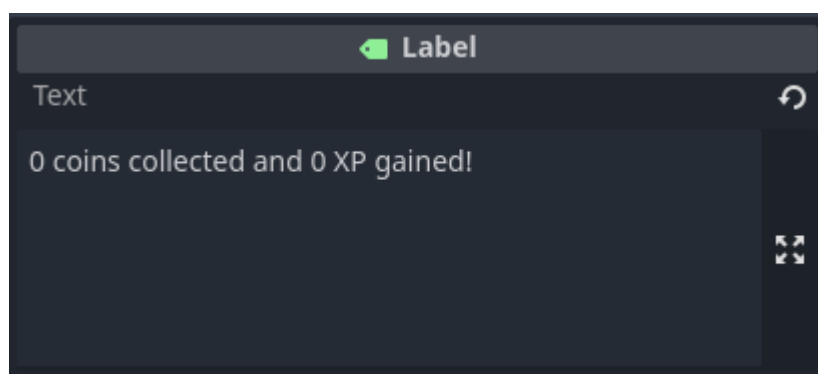
I wanted to include a score system for 2 reasons; for the player to measure their progress through the game and also in order to complete success criteria 14. I will make it so that the score will increase whenever the player collects coins or powerups, or kills enemies. A coin will be worth 25XP and power-ups will be worth 100XP. Killing enemies will also be worth 25XP. I went by implementing this by creating a Label node and putting it where I desired. I then imported the node into the script as can be seen on Line 4. This would allow me to manipulate the node however I wanted it using code.

```
4  @onready var score_label = $"../..//Score"
5
6  var score = 0
7
8  func _on_body_entered(body: Node2D) -> void:
9      >| coin_sprite.play("collected")
10     >| score += 1
11     >| print(score)
```

I wanted a counter for the number of coins that a player had collected so that they can monitor their progress. So, I created a variable called score and replaced it with the code below.

I then created another variable for XP, which would be increased by 25 each time a coin is collected. I implemented this in the function.

My stakeholders wanted a way to display the players score and XP to them in the game itself, so I created a Label node, positioned it, and set the text to



This follows and fulfils success criteria 15.

Next, in order to display the current score and XP, I needed to set the text to different values while the game is running. I did this through importing the label node into the script (Line 4), so that I could manipulate the node however I wanted with the code.

```

4  @onready var score_label = $"../Score"
5
6  var score = 0
7  var XP = 0
8
9  func _on_body_entered(body: Node2D) -> void:
10 >  coin_sprite.play("collected")
11 >  score += 1
12 >  XP += 75
13 >  score_label.text = score + " coins collected and " + XP + " XP gained!"

```

Line 11 will increment the score, which is the coin count, by 1 whenever the player collects a coin.

Line 12 increases the XP by 75 each time a coin is collected because the coins are worth 25XP each.

Line 13 changes the text of the score label so that it displays the current coin count and XP using concatenation. This will change the label displayed to the player in the game in real time.

Running this program didn't work. This is because concatenation only works with combining different strings together and the score and XP were both integers. So, I set them both to string values.

Running this program worked, but whenever a coin was collected, the coin counter displayed would be reset to 1, meaning it wouldn't show a cumulative count of the coins collected. This can be shown in the video labelled "**8. score test**"

After discussion with my stakeholders, who have computer science knowledge, we deduced that each script will be run every time that it is necessary. In this case, a coin.gd script will run for every coin. Therefore, all the code in the script will run for every coin, including the creation of the score and XP variables. This means that the variables are destroyed each time a coin is collected. Essentially, the score and XP variables are local variables, and to fix this problem, I need to make them global variables, so that they are not deleted for every coin and they can be added upon. I did this by going back to the Global script which was created for the pauseMenu, and simply adding a score and XP variable there.

```

1  extends Node
2
3  var playing: bool = false
4  var score = 0
5  var XP = 0

```

Then, in the coin script, I deleted the creation of the local score and XP variables, and changed the references to them too.

Running this program now worked exactly how my stakeholders and I wanted it to. Later, I realised that this actually wasn't working flawlessly.

During testing, I often restart the level via the pause menu, because at this point, I have not yet implemented a kill zone where the player dies and respawns. So, this is how I respawn without relaunching the entire game. But I found out that even though the restart button reloads the entire game scene, the reload_current_scene functions scope doesn't reach the global variable, so therefore it doesn't reset the score and XP whenever the level is restarted. Fortunately, I fixed this easily by manually setting the score and XP to 0 in the restart function. And I will also do this whenever the player respawns from dying.

```
23  ▾ func _on_restart_pressed() -> void:
24      >| resume()
25      >| Global.score = 0
26      >| Global.XP = 0
27      >| get_tree().reload_current_scene()
```

```
7  ▾ func _on_body_entered(body: Node2D) -> void:
8      >| coin_sprite.play("collected")
9      >| Global.score += 1
10     >| Global.XP += 75
11     >| score_label.text = str(Global.score) + " coins collected and " + str(Global.XP) + " XP ga
```

```
11  ▾ func _on_animated_sprite_2d_animation_finished() -> void:
12     >| queue_free()
13     >| Global.score += 1
14     >| Global.XP += 75
15     >| score_label.text = str(Global.score) + " coins collected and " + str(Global.XP) + " XP ga
```

I also realised that some coins were duplicating scores. This was because it was possible for the player to collect the same coin twice by entering the coin twice, before it despawns. The reason that they can do this is due to me making the program wait until the collected animation finishes playing to delete the coin. In this time, they can collect multiple times. I fixed this by moving the incrementation of the score and XP variables to the animation finished function.

So now, Line 12 deletes the coin.

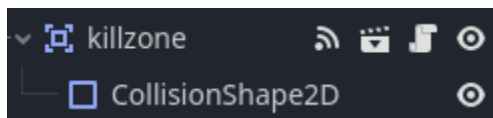
Now that the coin is deleted and nothing else will happen with it, the score and XP variables are added to, and the score label is updated on Line 15.

Now the score is implemented correctly. The ideal results can be seen in the video labelled "9. score test v2". This video shows the testing of tests 48 – 51.

Killzone

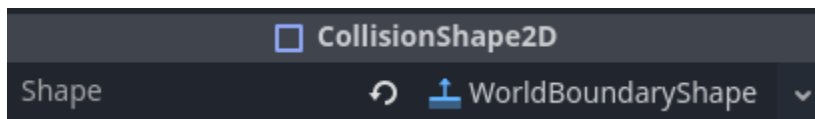
Until this moment, I have been respawning by just restarting the level. Whenever falling off a platform, the program would do nothing and let me fall infinitely. This can be shown in the video labelled “**10. Dying issue**”. And whenever my stakeholders tested the game, they reminded me to make a dying feature. So, I created the killzone for falling into the void/off the platforms.

I made a new scene and named it killzone, making the root node an Area2D node. The reason I made it a scene was so that I could reuse the killzone in every level, instead of creating a new one every time. This is one of the many reusable components in my program. I made a CollisionShape2D as a child node to give the killzone a shape. Next, I needed to code what happened when the killzone was entered. This would be at any point in the killzone at any time.



I created a script for the killzone.

I then went to the Node tab and created the function for when the body of the node is entered. The body itself will be created by the CollisionShape2D node. In the CollisionShape2D, I made the shape a WorldBoundaryShape. This essentially creates a border along the entirety of the level (goes on infinitely).



Then I simply went into the design of the level and put the killzone below the platforms where I wanted it.

```
3  func _on_body_entered(_body: Node2D) -> void:
4      >| #resets XP to 0
5      >| Global.reset_XP()
6      >| #resets game scene
7      >| get_tree().reload_current_scene()
```

So, when the player enters the killzone, the score and XP will reset, and the level will reload.

The evidence can be seen in the video labelled “**11. Dying fixed**”

Better Climbing

The problem

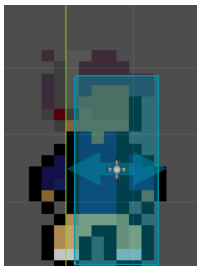
My stakeholders have often been playing my game, and they had a problem with the climbing function and the wall jumping feature. The problem they had was when climbing, 2 keys are always required: SHIFT & A/D (depends on whether the wall is on the players left or right side). Then, they have to press either W (climb up), S (climb down) or space (wall jump). They didn't like how so many buttons had to be pressed and how complicated the process was. This was especially true when wall jumping off of a wall into the other direction. The main problem was pressing A or D, as I knew and my stakeholders thought it was unnecessary. The only reason that that is a requirement in the first place, is because in my code, I use embedded functions that are included in Godot for movement. The one I use for climbing is `is_on_wall()`. This function checks if the player's physical body (provided by the `CollisionShape2D`) is currently against a wall. This condition is used in if statements which I can then use code in. But this function will only return true if the player is actively running against the wall. If they simply rest against it, the function will return false, for some reason that I still haven't figured out, and then none of the climbing functions will work.

Wall climbing function

I conducted a lot of research, and no one anywhere had had the same situation as me. But I found that `RayCast2D` nodes were commonly used for wall climbing. Although the code was very much different to my game and couldn't be used well, I tried to make it work myself.

`RayCast2D` nodes are nodes that are shaped like arrows, and detect if anything is directly on the tip of the arrow, like an obstacle, enemy or wall. I saw that I can use this to replace the `is_on_wall()` function and make my own, which I can then configure however I want, which I couldn't do with the `is_on_wall()` function because its embedded into Godot.

I went to the player scene and added 2 `RayCast2D` nodes; one for the right side and another for the left. I then positioned them to the middle (vertically) of the player's physical body (`CollisionShape2D`) and pointed them outwards. I made them as long as the `CollisionShape2D` itself so that they didn't protrude.



```
13  @onready var ray_cast_right: RayCast2D = $RayCastRight
14  @onready var ray_cast_left: RayCast2D = $RayCastLeft
```

Next, I went to the player.gd script and created an instance for the RayCast2Ds. Now I can manipulate them and check their properties.

Next, I created a new function for wall climbing.

```
65  func handle_wallClimbing():
66      var onWall = false
67      if Input.is_action_pressed("shift"):
68          if ray_cast_left.is_colliding():
69              onWall = true
70          elif ray_cast_right.is_colliding():
71              onWall = true
72      if onWall == true:
73          if Input.is_action_pressed("move_up"):
74              velocity.y = WALLJUMP_VELOCITY
75          elif Input.is_action_pressed("move_down"):
76              velocity.y = -WALLJUMP_VELOCITY
77          else:
78              velocity.y = 0
```

First, I create the new variable that will be true if the player is on the wall.

I then immediately check if shift is being pressed, because this is the first condition for climbing.

Then it checks if either RayCast2D is colliding with anything. The purpose of this is to check if there is any object with physics against the player, which at the height that I have placed the nodes, can only be a wall. So therefore, the player is now against a wall, and onWall is set to true.

This is done again for the right side.

It then checks if the player is indeed on the wall, in which case it checks if either pressing W or S, and will move on the wall accordingly. If the player does not press anything, however, then the player will cling to the wall.

Now, running this code, there were several problems. I will address them here now.

First of all, the gravity function was no longer working.

```
37  #Creates the gravity function
38  func apply_gravity(delta):
39      if not is_on_floor():
40          velocity.y += gravity * delta
```

On line 39, the program checks if the player not on the floor. This also excludes if they are on a wall. But this part only works if `is_on_wall()` is true. But because I'm never using that function, it is false, and therefore, whenever I am now on a wall, gravity still applies and I cannot climb properly without being dragged back down. To fix this, I just added a condition which checks if `onWall` is false, because if they are not on the floor, and they are not on the wall, then they are in the air in which case gravity should apply.

```
37 #Creates the gravity function
38 func apply_gravity(delta):
39     if not is_on_floor() and onWall == false:
40         velocity.y += gravity * delta
```

As clearly seen, there is an error. This is because `onWall` is a local variable that I created in the wall climbing feature. Therefore, the scope is limited to that function. I had a few options to fix this. I could either define it outside a function and pass it through each one, where it was needed, as a parameter. But I decided to make it a global variable so it would be easier and more accessible.

```
6 var onWall = false
```

```
37 #Creates the gravity function
38 func apply_gravity(delta):
39     if not is_on_floor() and Global.onWall == false:
40         velocity.y += gravity * delta
```

I then replaced all instances of `onWall` with instances of the global `onWall`.

```
65 func handle_wallClimbing():
66     if Input.is_action_pressed("shift"):
67         if ray_cast_left.is_colliding():
68             Global.onWall = true
69         elif ray_cast_right.is_colliding():
70             Global.onWall = true
71     if Global.onWall == true:
72         if Input.is_action_pressed("move_up"):
73             velocity.y = WALLJUMP_VELOCITY
74         elif Input.is_action_pressed("move_down"):
75             velocity.y = -WALLJUMP_VELOCITY
76     else:
77         velocity.y = 0
```

I then tested this again, and some other problems occurred.

The problem was that if the player was on a wall, `onWall` would be set to true. But because it is a global variable, there is nothing resetting it back to false when the player

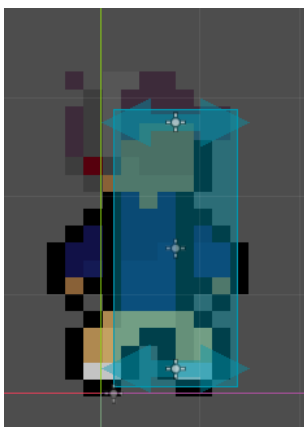
goes off the wall, so onWall would continue to be true and the player would be flying, as I saw as I tested it.

```
65  ▾ func handle_wallClimbing():
66  ▾  ▹ if Input.is_action_pressed("shift"):
67  ▾  ▹  ▹ if ray_cast_left.is_colliding():
68  ▾  ▹  ▹  ▹ Global.onWall = true
69  ▾  ▹  ▹ elif ray_cast_right.is_colliding():
70  ▾  ▹  ▹  ▹ Global.onWall = true
71  ▾  ▹  ▹ else:
72  ▾  ▹  ▹  ▹ Global.onWall = false
73  ▾  ▹ else:
74  ▾  ▹  ▹ Global.onWall = false
75  ▾  ▹ if Global.onWall == true:
76  ▾  ▹  ▹ if Input.is_action_pressed("move_up"):
77  ▾  ▹  ▹  ▹ velocity.y = WALLJUMP_VELOCITY
78  ▾  ▹  ▹ elif Input.is_action_pressed("move_down"):
79  ▾  ▹  ▹  ▹ velocity.y = -WALLJUMP_VELOCITY
80  ▾  ▹  ▹ else:
81  ▾  ▹  ▹  ▹ velocity.y = 0
```

I fixed this by adding an else statement so that if neither RayCast2Ds are colliding with a wall, onWall is false. This is something basic that I just forgot to add. I did the same thing for pressing shift, and if they are not, onWall is immediately set to false, regardless of whether they actually are on a wall or not. This emphasises the fact that this is the climb button.

Next, I realised that the RayCast2D can only see things that are on the same vertical height as it; nothing above and nothing below. This became a problem when I realised if the player is trying to get on top of a wall, the program sets onWall to false the second that the RayCast2D goes above the wall, because it is not against the tip of the arrow anymore. In simple terms, the player cannot go all the way up the wall. This problem is shown in the video “[12. Better climbing](#)”.

To fix this, I needed to measure all points on the player itself if they are on the wall. I did this by making more RayCast2Ds and putting them in all 4 corners of the CollisionShape2D.



Now, I could measure if any point is on the wall at any time. I implemented this into my code.

```
85 #creates player climbing function
86 func handle_wallClimbing():
87     >| #main requirement is holding shift on walls
88     >| if Input.is_action_pressed("shift"):
89         >| >| #checks if either raycast is on a wall
90         >| >| if ray_cast_bottom_left.is_colliding() or ray_cast_top_left.is_colliding():
91             >| >| >| #player is on a wall
92             >| >| >| Global.onWall = true
93             >| >| #other side
94         >| >| elif ray_cast_bottom_right.is_colliding() or ray_cast_top_right.is_colliding():
95             >| >| >| Global.onWall = true
96         >| >| else:
97             >| >| >| Global.onWall = false
98     >| else:
99         >| >| Global.onWall = false
100     >| #if player is on a wall and pressing shift
101     >| if Global.onWall == true:
102         >| >| #if player then presses W
103         >| >| if Input.is_action_pressed("move_up"):
104             >| >| >| #moves up the wall
105             >| >| >| velocity.y = WALLJUMP_VELOCITY
106             >| >| #if player then presses S
107         >| >| elif Input.is_action_pressed("move_down"):
108             >| >| >| #moves down the wall
109             >| >| >| velocity.y = -WALLJUMP_VELOCITY
110         >| >| else:
111             >| >| >| #otherwise, clings/sticks to the wall
112             >| >| >| velocity.y = 0
```

The changes are on line 69 and 71, which check if either left RayCast2Ds are colliding with a wall or either of the right ones are colliding with a wall. Now, if either right or left bottom arrow is still on a wall, onWall will still be true until neither the top or bottom are on it, in which case the player will finally be off the wall. In simple terms, the entire player is now being checked to see if it is on the wall. The problem has been fixed and the player can now climb all the way up the wall.

During this process, I increased the size of the arrows in the RayCast2D so that it protruded just a little bit, so that the scope of the RayCast2D looking for a wall was larger and more sensitive. This can be done by changing the target position in the Inspector tab of each RayCast2D.

The climbing function has now been redone, and it is acting exactly how my stakeholders and I want it to.

Next, I moved to the wall jumping function.

Wall jumping function

```
58  func handle_wallJump(direction):
59  >|  if is_on_wall():
60  >| >|  if Input.is_action_just_pressed("spacebar"):
61  >| >| >|  velocity.x = -direction * WALLJUMP_PUSHBACK
62  >| >| >|  velocity.y = WALLJUMP_VELOCITY * 2.5
```

This is the original code for the wall jump. But since I'm no longer using `is_on_wall()`, I have to rewrite all of it.

I began by creating a new function for wall jumping.

```
114  #creates jumping off the wall
115  func handle_wallJump():
116  >|  #can only wall jump if on a wall
117  >|  if Global.onWall == true:
118  >| >|  #checks for wall jump key
119  >| >|  #can't hold space (hence, "just")
120  >| >|  if Input.is_action_just_pressed("spacebar"):
121  >| >| >|  #figures out player's facing direction (if wall is on left/right side of player)
122  >| >| >|  if ray_cast_bottom_left.is_colliding() or ray_cast_top_left.is_colliding():
123  >| >| >| >|  #shoots them up
124  >| >| >| >|  velocity.y = WALLJUMP_VELOCITY * 2.5
125  >| >| >| >|  #sends them in direction away from the wall
126  >| >| >| >|  velocity.x = WALLJUMP_PUSHBACK
127  >| >| >|  else:
128  >| >| >| >|  velocity.y = WALLJUMP_VELOCITY * 2.5
129  >| >| >| >|  #sends them in direction away from the wall (hence, "-")
130  >| >| >| >|  velocity.x = -WALLJUMP_PUSHBACK
```

Line 74 checks if the player is on the wall, using the custom `onWall` variable. It also checks if the player is pressing shift. If both of these conditions are true, then the player is definitely climbing on the wall.

The program then checks for the spacebar being pressed. If true, then the player is attempting to wall jump.

Line 76 and 79 decide the direction of the player. I have to find the direction of the player manually because I'm not using the `is_on_wall()` function. Instead, I use the `RayCast2Ds` to decipher the direction. If the left `RayCasts` are colliding with a wall, then the player is on the left wall.

I also encountered another problem. While wall climbing, the variable `direction` which is used for normal movement isn't given a value here, so during the animation function, it can sometimes present the player climbing in the wrong direction. Now granted, this doesn't happen often, but I thought I should eliminate this problem anyway. To do this, I went to the animation function and added another condition:

```
func handle_animations(direction):
> if direction > 0 or ray_cast_bottom_right.is_colliding() or ray_cast_top_right.is_colliding():
>     animated_sprite.flip_h = false
> elif direction < 0 or ray_cast_bottom_left.is_colliding() or ray_cast_top_left.is_colliding():
>     animated_sprite.flip_h = true
```

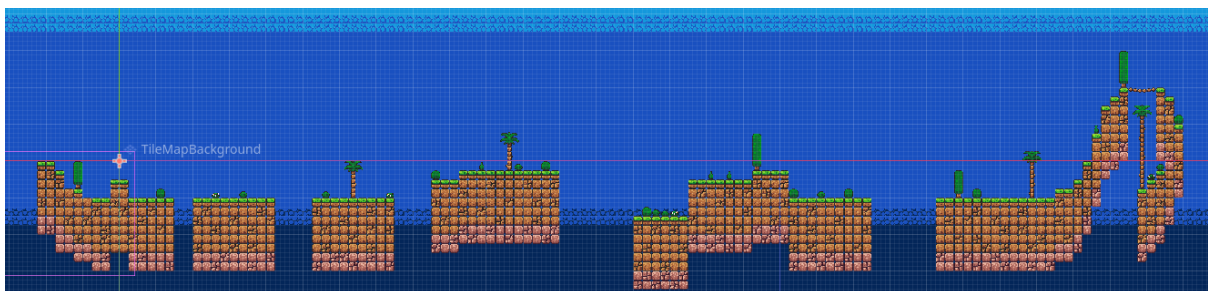
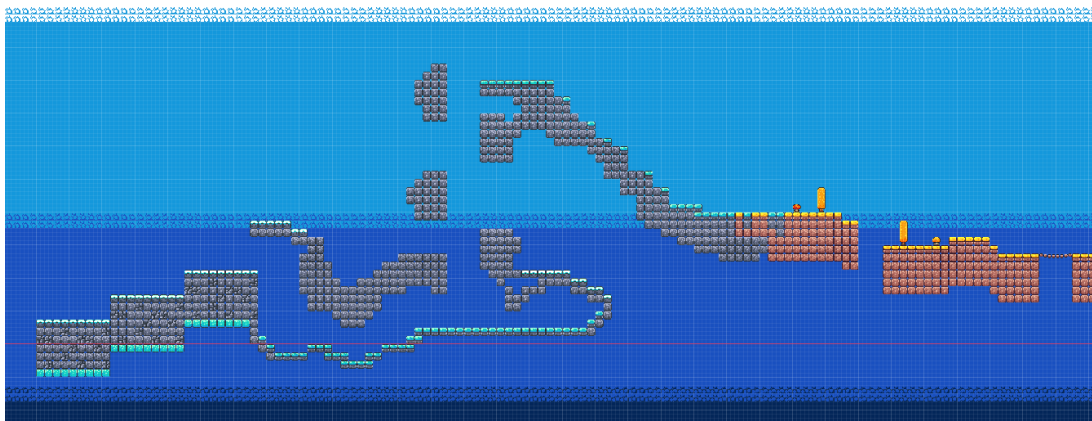
This would check if either the right side or left side of the player is on a wall, in which case it will flip the AnimatedSprite2D accordingly.

I have now redone the entirety of my wall movement, which the results of which can be seen in the video “**13. Better climbing v2**”.

Level 2

Next, I chose to make my second level of gameplay. The theme I’m going for is the player starts below ground level (level 1), then makes it to the surface (level 2), then goes up into space (level 3). This main theme of the game follows success criteria 8 of having a different landscape and biome for each level

I created a new scene, then added all the necessary nodes and scenes, such as the TileMaps for the background and platforms, the pauseMenu, killzone and player. All methods are the same as the first level. I then began to design how the level would look.



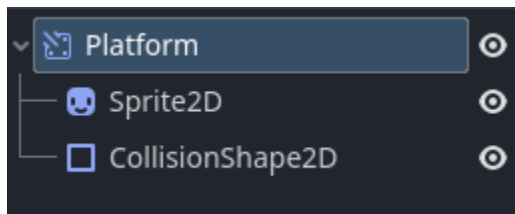
Platforms

I wanted a dynamic way of moving around, to add a challenge for the player. After suggestions from my stakeholders some of which suggested enemies and others moving platforms, I chose to make platforms.

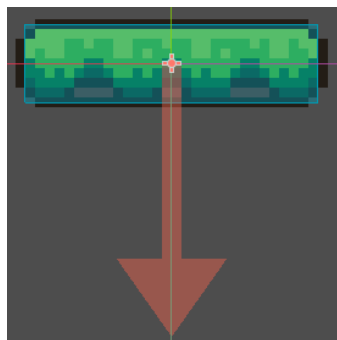
What I wanted is a looping platform that changes direction whenever it hits a wall.

There are very limited methods of making moving interactable objects. I chose to make an animation of the movement of the platform.

I began by making a new scene, for the platform itself. This is so I can reuse the platform easily multiple times, without creating it from scratch. The root node is an `AnimatableBody2D`. I used this so that it can be manipulated using animations, while keeping its physical structure. I then created child nodes as a `Sprite2D` node, for the visual aspect, and `CollisionShape2D`, for the physical aspect.



I created the shape of the `CollisionShape2D` as a rectangle fitting the shape of the `Sprite2D` image.

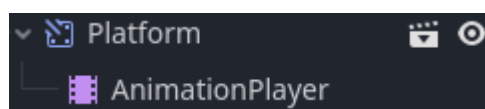


The downwards facing arrow is the representation of the setting “One Way Collision”.

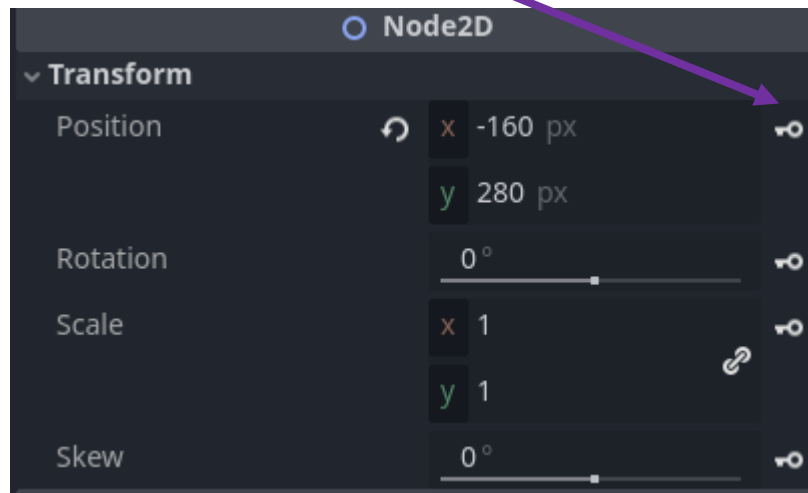
This means that the player will only behave to the objects physics while above the platform, allowing the player to jump through the platform in order to get on top of it.

This is a commonly used feature of several games in my Analysis (Celeste, Mario). I can use these platforms now as a way to gain horizontal and vertical distance quickly.

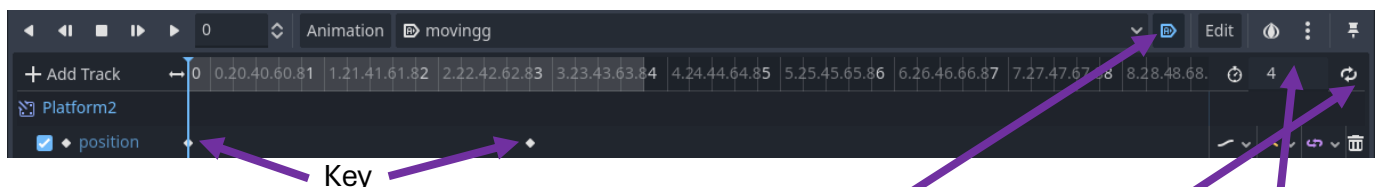
To implement the moving part, I used animations to move the platform. I dragged the platform scene into my level, and added an `AnimationPlayer` as a child.



I then went to the Animation tab at the bottom of the screen, created a new animation then created positioning keys for the animation. A key is a position that is presented in the animation (for example, if there are 2 keys at 2 positions, the platform will move from one to the other key position). I made these by going to the Inspector tab of the platform node, and clicking the key symbol when the position was what I desired it to be.



The key creates a point in the animation. It can signify the start and end of the animation.



Once I made the first one, I went to the end of the timeframe of the animation, moved the platform to the point where I wanted the animation to end, and pressed the key again. I tested this by playing the animation via the controls seen in the top left corner of this screenshot. I can then tweak the speed of the animation by editing the time taken for it to occur. I can also enable autoplay and for the animation to loop, which I did for platforms.

In essence, the platform will move between the 2 key positions, at the speed of distance between keys divided by time of animation. Looping the animation from side to side (achieved by pressing the loop button twice) means that the animation will go side to side, which is how I wanted animations to work.

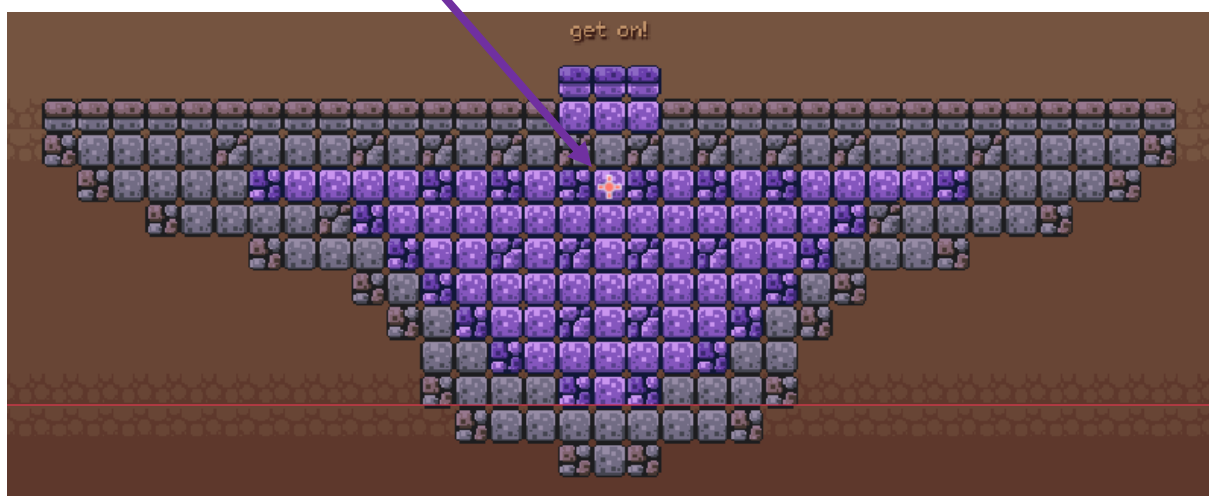
An animation for a moving platform has now been made. I had to do this for every platform I implemented, and the results of this and the new Level 2 are shown in the video “**14. Level 2 and platforms**”

Transition between levels

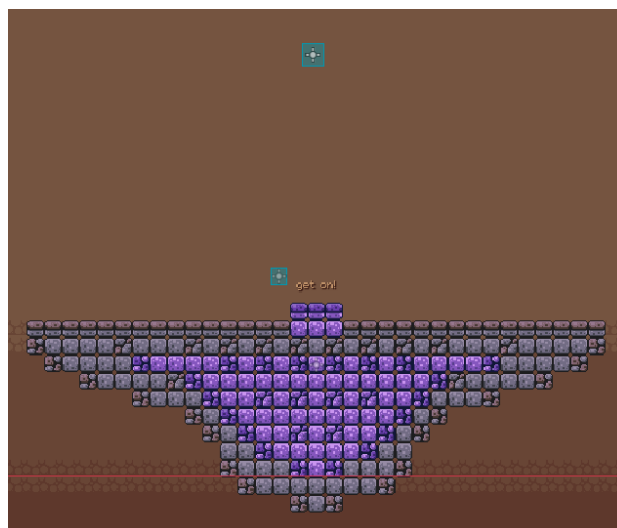
I wanted to create a seamless transition between the end of level 1 and the start of level 2 for several reasons. One was to reiterate the theme in the game of each level increasing in altitude until level 3 in space. Another was to allow the player to start the second level without having to go back to the main menu, creating a seamless transition.

The plan was to create a platform which moved upwards, out of the visible screen which then entered a teleporter which would change the scene to level 2.

I began by dragging in a platform and placing it under a TileMap island/platform I had made.



I then made an Area2D node in the level 1 scene and a CollisionShape2D child node. This would be the activator for the platform, that the player would walk/jump through. I also made duplicate of this for the teleporter from level 1 to 2. After making the rectangle shape for the CollisionShape2D nodes, I created a script for level 1 and started inputting the signal functions



```

1  extends Node2D
2
3  #level 1 platform that goes to level 2
4
5  @onready var animation_player: AnimationPlayer = $"teleporter to level2/transporter platfor
6  @onready var collision_shape_2d: CollisionShape2D = $CollisionShape2D
7
8  func _on_transporter_activator_body_entered(body: Node2D) -> void:
9      >| #starts the moving animation
10     >| animation_player.play("new_animation")
11
12  func _on_teleporter_body_entered(body: Node2D) -> void:
13     >| #when the player goes out of screen, cuts to level 2
14     >| Global.level1 = false
15     >| Global.level2 = true
16     >| Global.level3 = false
17     >| Global.level2music = true
18     >| MusicLevel1.refresh_music()
19     >| MusicLevel2.refresh_music()
20     >| get_tree().change_scene_to_file("res://scenes/level_2.tscn")

```

Line 5 and 6 create instances of the animation player for the platform and CollisionShape2D.

Line 8 is the signal function that will play the platform animation of it moving upwards out of the screen (Line 10).

Line 12 is the signal function that detects if the player enters the teleporter. If the player enters it, then all variables for level switching will be set (Line 14, 15, 16 & 17). I then refresh all necessary music functions and change the scene to level 2.

Now, when the player approaches the platform, it will activate, allowing them to jump onto it. The platform will then rise up to the teleporter, which switches the scene to level 2 once the player enters it. This is the transition that goes from level 1 to level 2.



Level Selector

Next was the method to navigate to different levels. I began by making a new User Interface scene, then adding a HBoxContainer. This is the same as a VBoxContainer which I've used before, but it stores its child nodes horizontally instead of vertically. I did this to follow my screen designs and so I could produce areas that preview the level in this screen.



I then added 3 button nodes, level 1, 2 & 3, spacing them apart from each other so that they looked appropriate when running the game. I then added the script for the level selector scene.

```
1  extends Control
2
3  ▾ func _on_level_1_pressed() -> void:
4      >|  get_tree().change_scene_to_file("res://scenes/game.tscn")
5
6  ▾ func _on_level_2_pressed() -> void:
7      >|  get_tree().change_scene_to_file("res://scenes/level_2.tscn")
```

```
3  ▾ func _on_start_button_pressed() -> void:
4      >|  get_tree().change_scene_to_file("res://scenes/level_selector.tscn")
```

Linking the level 1 and 2 buttons to the script, I made each button change the scene to their corresponding game scenes.

I then corrected the script for the title screen, making the play button now move to the level selector screen

When testing the whole game from this point, I encountered a problem.

When going towards the settings screen via the pauseMenu in the game scene, then returning to the game scene from the settings screen would always take you to level 1, regardless of which level you are actually on. This can be seen in the video labelled “**15. Pause settings screen issue**”.

The reason for this is simple; I only made the back button route to the level 1 scene. But I needed a way to make the settings scene script know what level the player was just in, and then route them back to the correct scene.

```
4 var level1: bool = false
5 var level2: bool = false
6 var level3: bool = false
```

I tried to implement this by creating global variables for level 1, 2 & 3 and setting them to true or false in the script for the pauseMenu, so that it's the last act done before they are routed to the settings screen, and the settings screen script now knows which Boolean is true and therefore can route the player to the correct scene. But then I realised I cannot do this, because I am using the same pauseMenu scene for all levels, and they are all identical, therefore I cannot differentiate code where one will set level 1 to true, and another will set level 2 to true. This is one disadvantage of using reusable components: the purpose must always be identical.

After trying different methods and consulting my stakeholders, Shazil suggested that I set the global Booleans to a value in the level selector screen, depending on whether level 1 or 2 or 3 is pressed. This was a perfect solution, so I quickly implemented this.

As seen, the code sets the necessary Boolean variable to true or false depending on what button is pressed. I can now go to the settings screen to implement the solution.

```
3 func _on_back_button_pressed() -> void:
4     #checks if player is playing
5     if Global.playing == true:
6         #then checks what level they are in
7         if Global.level1 == true:
8             #then navigates to the correct level
9             get_tree().change_scene_to_file("res://scenes/game.tscn")
10        elif Global.level2 == true:
11            get_tree().change_scene_to_file("res://scenes/level_2.tscn")
12        #put level 3 here
13    else:
14        #if they're not playing, take player back to title screen
15        get_tree().change_scene_to_file("res://scenes/title_screen.tscn")
```

Now, when the back button is pressed, the program checks what level the player is coming from, if any level at all, and then routes them to the correct scene. This can be seen in the video “**16. Pause settings screen fixed**”, which also shows the result of the level selector screen.

Music

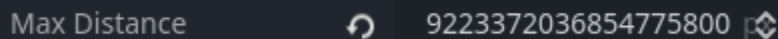
Music is an essential feature in all games to break the silence and prevent player from getting bored easily. It is outlined in success criteria 17.

First, I created an AudioStreamPlayer2D in the level scene to play music. I then input my music file which I downloaded from:

- <https://freemusicarchive.org/music/holiznacc0/winter-lofi/winter-blues/>
- <https://freemusicarchive.org/music/holiznacc0/lazy-summer-lofi-1/blue-skiesmp3/>
- <https://freemusicarchive.org/music/holiznacc0/winter-lofi/fire-place/>

All resources are CC0, so I am free to use them.

While running this, I realised that the music would stop after approximately 30 seconds. This can be shown in the video labelled “**17. Music**”. I then found out that this was because there is a maximum distance to the playback of an AudioStreamPlayer2D. To fix this and have music playing throughout the entire level, I set the maximum distance to the biggest number that I could.



Max Distance 9223372036854775800

This issue is now fixed, because the playback will not stop past a certain distance. The proof is shown in the video labelled “**18. Music v2**”.

I then realised, through testing, that music wouldn't play in the menu screens. This is obviously because I didn't put the music node in each menu scene, but I realised that if I did, the music would restart every time I went into a new scene (e.g. from title screen to settings screen). Instead, I wanted continuous music to play across the different scenes. I did this by putting the music node in its own scene. I did this so that it would become a reusable component, and I wouldn't have to create a new one every time I wanted music. I then went to Project Settings, Globals, Autoload, and added the music scene to autoload. This would make the scene autoplay as soon as the game is run, and make it immune to scene changes; essentially it is above the scene changes and normal restart functions, because it is global.

Now, music would play across the entire game. I chose to play level 1 music because all the menu screens are modelled after level 1, so it would be most appropriate. My stakeholders also suggested this and agreed with my decision.

Next, I wanted to stop playing level 1 music and start playing level 2 music when the player enters level 2. I did this by going into the script for the music_level_1 script and writing the following function:

```
9  func play_music():
10  if Global.muted:
11      #pauses music
12      music_l1.stream_paused = true
13      #level 1 music shouldn't play in level 2
14  elif Global.level2 or Global.level3:
15      music_l1.stream_paused = true
16  else:
17      music_l1.stream_paused = false
```

Before I analyse this, it should be clear that I also enabled autoplay for the `AudioStreamPlayer2D` in the Inspector tab. There is a difference between autoloading a scene and auto playing an audio node, and auto playing is crucial for this working. Line 9 begins by creating a function which will control the playback of the music. It has several conditions:

- The global variable “muted” must be false (this will be explained in the audio screen)
- The player must not be in level 2 or 3

If these conditions are fulfilled, then the `AudioStreamPlayer2D`’s `stream_paused` Boolean attribute is set to false and music plays. If not, then it is set to true and the music pauses.

I quickly found that this would not work, which can be seen in “**19. Music v3**”. Ismail pointed out that this was because it was not continuously being checked, because the function was not being called repeatedly so that the product of the function was correct in real time.

So, I then added 2 other functions.

```
1  extends AudioStreamPlayer2D
2
3  @onready var music_l1: AudioStreamPlayer2D = self
4
5  #called when the game launches immediately
6  func _ready() -> void:
7      >| update_music_playback()
8
9  func update_music_playback():
10 >| if Global.muted:
11     >| >| #pauses music
12     >| >| music_l1.stream_paused = true
13     >| #level 1 music shouldn't play in level 2 or 3
14 >| elif Global.level2 or Global.level3:
15     >| >| music_l1.stream_paused = true
16 >| else:
17     >| >| music_l1.stream_paused = false
18
19 func refresh_music():
20     >| update_music_playback()
```

Line 6 creates the ready function which will run as soon as the game is launched, so it immediately checks the music playback.

I also created a refresh function which I can call whenever there is a scene change so that corrections can be made and music can be paused/played.

I then did the same for the `AudioStreamPlayer2D` for level 2.

```

3  @onready var music_l2: AudioStreamPlayer2D = self
4
5  #called when the game launches immediately
6  func _ready() -> void:
7      >| update_music_playback()
8      >|
9  func update_music_playback():
10     >| #this music should only play in level 2
11     >| if Global.level2 == true:
12         >| >| #begins playback of music
13         >| >| music_l2.play()
14         >| #no music should play while audio is muted
15     >| elif Global.muted == true:
16         >| >| music_l2.stop()
17         >| #no music should play in the menu screens
18     >| elif playing == false:
19         >| >| music_l2.stop()
20     >| else:
21         >| >| music_l2.play()
22
23 func refresh_music():
24     >| update_music_playback()

```

The conditions for music playback have changed for level 2 music:

- The global variable “muted” should be false
- The player must only be in level 2
- The player must not be in a menu screen

Autoplay has been disabled for level 2 music, so I have used different functions to play() and stop() the music. These functions are incorporated into the manipulation of audio nodes. These are different from setting the stream_paused value to true or false, because this directly affects the playback of the node, playing and stopping them. Once these were done, I called the music refresher wherever was necessary.

```

10 func _on_level_2_pressed() -> void:
11     >| Global.level1 = false
12     >| Global.level2 = true
13     >| Global.level3 = false
14     >| MusicLevel1.refresh_music()
15     >| MusicLevel2.refresh_music()
16     >| get_tree().change_scene_to_file("res://scenes/level_2.tscn")

```

This is the level selector script.

```

47  ▾ func _on_quit_pressed() -> void:
48      >| #player is no longer playing
49      >| Global.playing = false
50      >| Global.level1 = false
51      >| Global.level2 = false
52      >| Global.level3 = false
53      >| #updates music playback settings
54      >| MusicLevel1.refresh_music()
55      >| MusicLevel2.refresh_music()
56      >| resume()
57      >| get_tree().change_scene_to_file("res://scenes/title_screen.tscn")

```

```

32  ▾ func _on_restart_pressed() -> void:
33      >| #unpauses game
34      >| resume()
35      >| #refreshes music check
36      >| MusicLevel1.refresh_music()
37      >| MusicLevel2.refresh_music()
38      >| MusicLevel3.refresh_music()
39      >| #resets game scene
40      >| get_tree().reload_current_scene()

```

This is the pauseMenu script, in the restart and quit button function.

In all the other points in the script where I call the refresh_music functions, they are always before the game is resumed via the resume function. But in the restart button, I call the functions after the resuming of the scene. This is a solution to the problem that occurs when restarting level 2 or 3. Whenever restarting the level via the button in the pause menu, level 1 music would continue playing, even though it shouldn't. This can be shown in the video labelled “**20. Music v4**”. However, the fixed version is made by refreshing the music scripts after the game is resumed, as seen in the screenshot above. The results of this can be seen as fixed in the video labelled “**21. Music v5**”.

The functions are called whenever, essentially, there is a scene change.

There was now another problem.

When navigating to the settings screen or title screen via the pauseMenu in level 2, both music tracks would play (from both level 1 and 2). This is because Global.level2 is still true, and the music for level 1 is always playing. This is shown in the video labelled “**22. Music v6**”.

I attempted to fix this in a number of ways, but I will go through the method that finally worked.

I figured out that I needed to set level 2 to false, so that level 2 music would stop playing. But I realised that this also meant that the problem with navigating back from

the settings screen to the game scene was broken, because this method also used these variables. These 2 features were using the same variables and their purposes were clashing. This is a disadvantage to using global variables: their purposes and values can clash.

I fixed this by creating 2 new variables

```
4 var level2music: bool = false
5 var level3music: bool = false
```

I didn't create a music variable for level 1 because it isn't needed. Autoplay will play level 1 music in any situation where level 2 or 3 music are false. This is a small use of abstraction.

I then replaced all mentions of the level variables in the music functions and any other scripts where it is necessary.

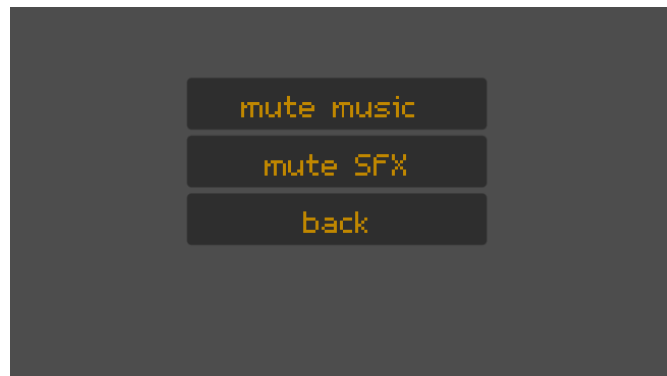
```
9 func update_music_playback():
10     if Global.muted == false:
11         #level 1 music shouldn't play in level 2 or 3
12         if Global.level2music or Global.level3music:
13             music_l1.stream_paused = true
14         elif Global.muted == false:
15             music_l1.play()
16         else:
17             #pauses music
18             #used this instead of stream_paused because assigning it to true doesnt directly stop it
19             music_l1.stop()
```

I also changed Line 15 and 19 to play() and stop () instead of stream_paused, for a reason that will be explained in the audio screen.

```
9 func update_music_playback():
10     #no music should play while audio is muted
11     if Global.muted == false:
12         #this music should only play in level 2
13         if Global.level2music == true:
14             #begins playback of music
15             music_l2.play()
16         else:
17             music_l2.stop()
18     else:
19         music_l2.stop()
```

These functions now work exactly how I want them to, which is play level 1 music in the level 1 and menu scenes, and play level 2 music in only the level 2 scene. I also create verification for muting music, which I will go into depth in the Audio Screen

Audio Screen



This is a feature in almost every game, where the user can control the volume and the state of music and sound playback. Due to time restrictions, I will not be able to implement volume controls, but I will make on and off buttons for sounds.

I began by making a new scene, and doing the same thing I have done for every menu screen yet; creating the VBoxContainer with button nodes. I then created the script and linked the buttons.

```
3  func _on_mute_button_pressed() -> void:
4      #switches the value of muted to the opposite (acts as a toggle)
5      Global.muted = !Global.muted
```

Now whenever the mute button is pressed, the value of muted will go to whatever it is not (i.e. if it is true, it will be set to false etc.). This creates a toggle button, so I can use the same button for the same purpose.

Global.muted is then used in the script for the music nodes, and it is the main condition that they play. When it is pressed, Global.muted is set to true or false, in which case the update_music_playback() function in both scripts of the AudioStreamPlayer2D's will update with the new Global.muted value. If it is true, then it will run the function stop(). The reason that I have used this instead of stream_paused = true is because this command didn't stop the music sufficiently. This can be seen in the video labelled "**23. Music v7**". This video shows the effects of stream_paused, and how it pauses music, but as soon as you return to the game scene, the music resumes, even though stream_paused is still true. This didn't make sense to me, so I changed the command to the function stop(). This worked better, and the music stayed stopped.

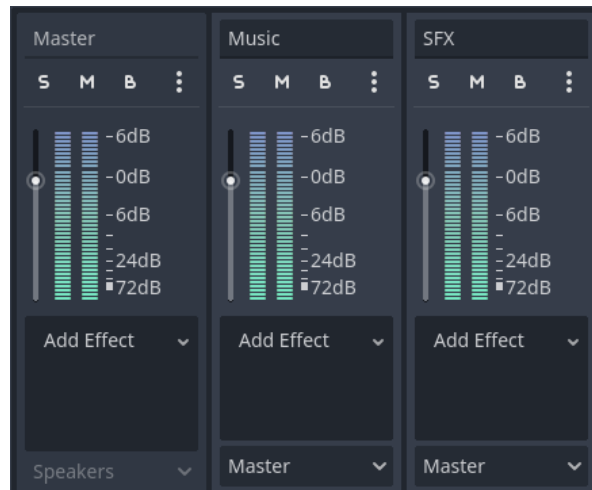
I then made the back button, which is again the same for every menu screen.

Although the mute music function was acting more or less how I wanted, there were little nitpicks that I found with the functionality, such as whenever you left somewhere where level 1 music was playing, and you came back, the music would restart, which was very noticeable and didn't sound continuous.

Although this method does work fine, I then found another more efficient method to mute the sounds.

In Godot, there are buses. Audio buses are a way to control different elements of audio, which are then outputted to the speaker. These are commonly used in dedicated audio software, but Godot has a simplified version of this.

By default, there is only 1 bus: the master bus, controlling all audio and outputs the audio. But I can add 2 more for music and SFX.



Now, I set both AudioStreamPlayer2D's to be controlled by the music bus



Now, I don't need to check if Global.muted is true to mute/stop music in the update_music_playback functions, because I will mute all music together. I chose to do this in the audio screen script with the mute button.

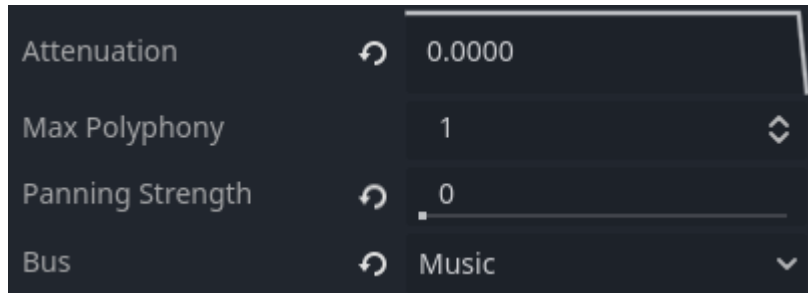
```
3 func _on_mute_button_pressed() -> void:
4     > #switches the value of muted to the opposite (acts as a toggle)
5     > Global.muted = !Global.muted
6     > #creates variable of music bus to manipulate it
7     > var music_bus_index = AudioServer.get_bus_index("Music")
8     > #sets the music bus to the value of muted
9     > AudioServer.set_bus_mute(music_bus_index, Global.muted)
```

The function begins with the same command, toggling the value of Global.muted towards whatever it wasn't.

I then create a variable which holds the properties of the music bus, which are retrieved by using the function get_bus_index. This is essentially the same process as creating a variable from a node in order to manipulate it.

Line 9 then sets the mute property (which can either be true or false) of the music bus to whatever the value of Global.muted is.

This will mute the music and unpauses the music whenever the mute button is pressed. No connection to the music scripts is needed anymore; everything is done locally and the effects are global.



A small problem I did notice, was that audio could be heard positionally, meaning as you go further into the level (rightwards), you could hear the music coming from the start of the level (leftwards) instead of being centered around where the player is. I fixed this by going to the Inspector tab for each music node and setting their Panning Strength and Attenuation both to 0.

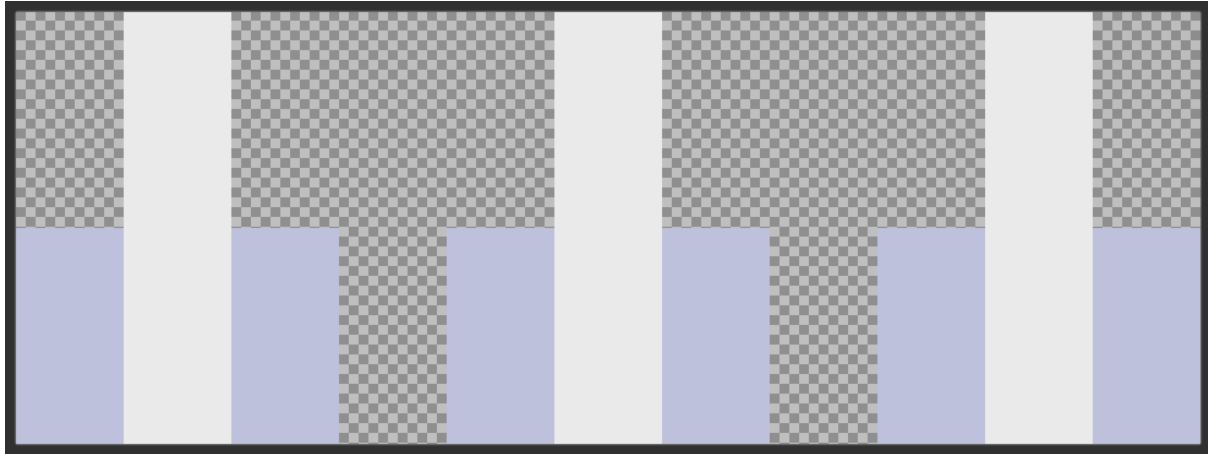
The final results of the mute function and the music playback is shown in the video labelled “**24. Music and mute function final**”.

Spikes

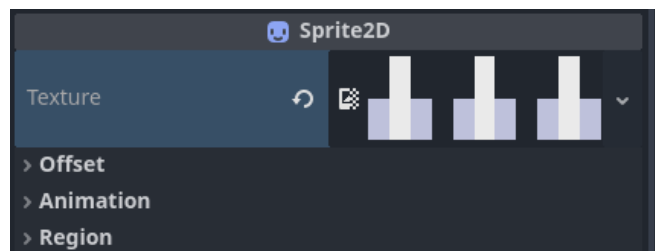
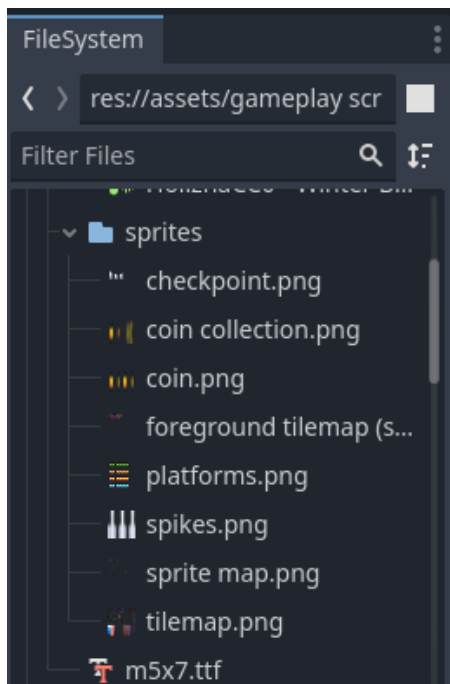
Spikes are another popular obstacle as seen, in the games I have researched (Geometry Dash & Celeste), apart from the void (falling death), that I will be using in level 2 and 3. This is to add a variety of obstacles that the player must individually think about how to avoid. Spikes are also easier to manipulate in terms of their shape and form, so I can create harder obstacles. The way that they will work is that they will be placed on a wall or floor etc., and whenever the player lands on them in any way, they will die and the level will reset.

To implement them, I used a similar method to the coins I made earlier. In this way, I created a new scene, so that I could reuse the spikes as much as I want. I then made the root node an Area2D node. I used this node instead of something like a StaticBody2D because a StaticBody2D is physically already there, and therefore whenever the player collided with it, it would've stopped the player's movement. I don't want that; I want a seamless transition between the player colliding with the spikes and the level resetting. I can also use built in function to detect whether anything has entered the Area2D, which is much easier than finding another way if something has collided with the StaticBody2D. I then moved on and made children nodes out of a Sprite2D and a CollisionShape2D. The Sprite2D is for the picture of the spikes, and the CollisionShape2D represents the physical aspect.

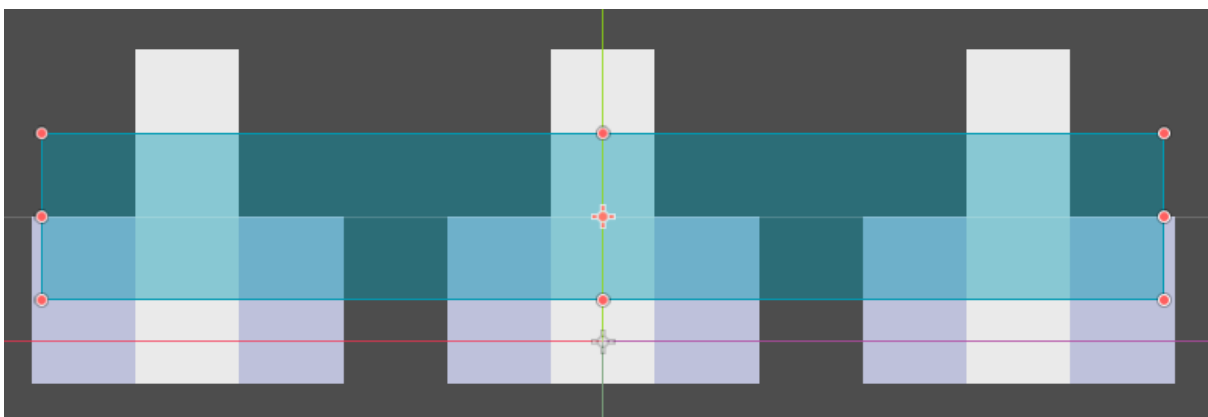
I then created my own version of a spike. This is loosely based on the spikes used in Celeste.



I put this into the Sprite2D by dragging the file from the FileSystem in Godot, to the Texture drop box in the Inspector tab of the Sprite2D node.



I then moved on to the CollisionShape2D, creating a new rectangle shape and resizing it to fit the shape of the Sprite2D.



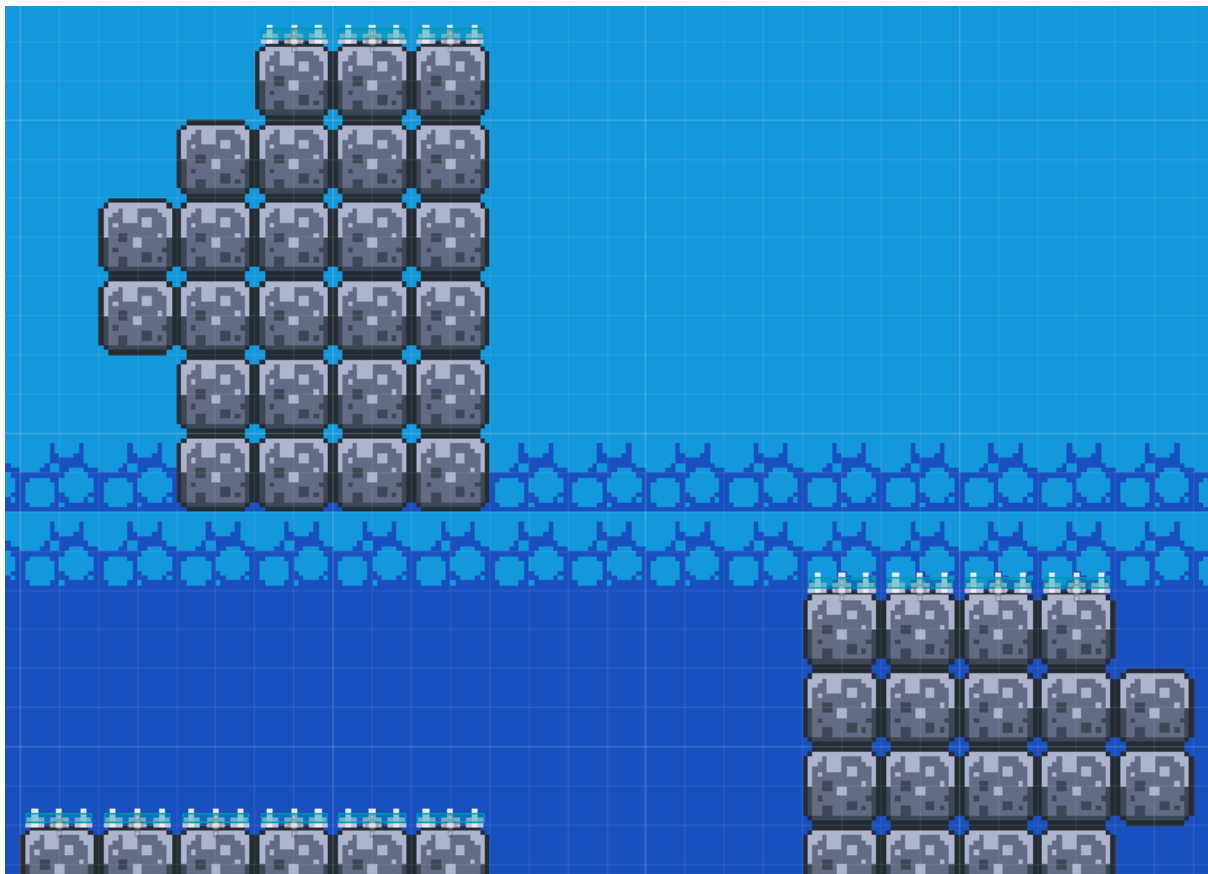
The blue rectangle is the rectangle shape of the CollisionShape2D. I purposely made it smaller than the spike itself to create a degree of leniency for the player; this ensures the player has fully fallen into the spike and eliminates accidental triggers.

After the entire shape and image of the spikes had been completed, I moved onto the script and functionality of the spike.

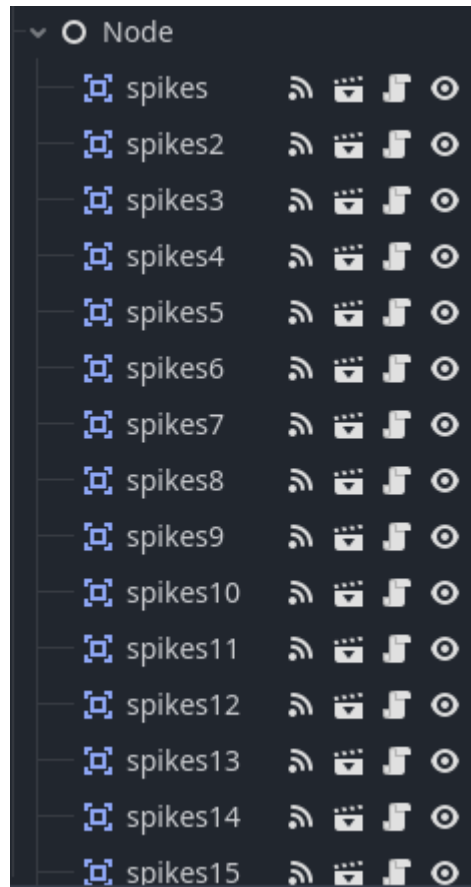
This was simple to do; I used the built-in function which detects if the body of the spike had been entered and then reloads the level scene.

```
1 extends Area2D
2
3 func _on_body_entered(body: Node2D) -> void:
4     >| get_tree().reload_current_scene()
```

The spike itself has now been made. I can now drag the spike scenes into the level scenes individually.



Now obviously, there are a lot of individual spike scenes in this picture (13), so I organised them by creating a new Node, and placed all the spike scenes in the Node as children.



The spikes have now been created. Whenever the player falls into some spikes, the level will reload. This can be shown in the video labelled “**25. Spikes**”

Dash

The dash function is one inspired from Celeste, one of the main games from my Research section.

The main function of a dash is to move the player in their current direction by a magnitude faster than they can move normally. This serves the purpose of adding variety to the movement, allowing for harder level design and obstacles and overall, a fun new way to move around for the player.

I first thought of making a super jump instead, which I implemented by creating another jump function, similar to a double jump. This feature was inspired from the glowing bounce orb from Geometry Dash.

I asked my stakeholders to see if there should be a dedicated dash/double jump button to press, or I should chain it on top of the existing jump keys. They told me to make a dedicated button, so that the jump couldn't accidentally be triggered. After further consideration, we agreed to make Enter the correct button, as it made use of a 2-

handed playstyle and took the additional stress of the left hand of all the keys it already has to press simultaneously.

```
78  >| >|  #initiates dash if Enter is pressed and the player is able to dash
79  ▼ >| >|  if Input.is_action_just_pressed("dash"):
80  >| >| >|      dash_direction = Input.get_axis("move_down", "move_up")
81  >| >| >|      velocity.y = -DASH_SPEED * dash_direction
```

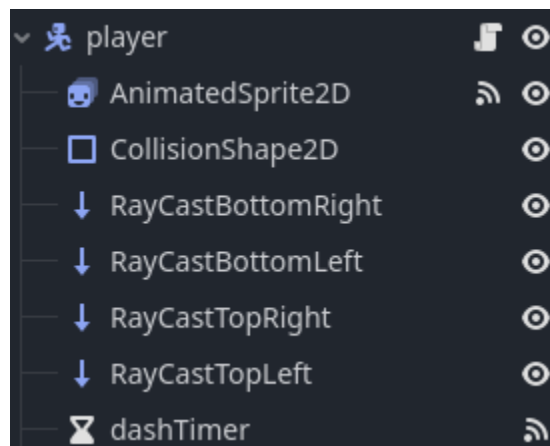
Line 79 checks for Enter being pressed. If it is, then a vertical axis is made from the components of the up and down movement keys. The program then uses this to boost the player in that vertical direction at the speed of the DASH_SPEED. This is shown in the video labelled “**26. Dash v1**”.

This did work, but I felt as if the movement itself felt anti-climactic, and I could only create obstacles that use this movement that were vertically far/challenging. I consulted my stakeholders, who are familiar with the games in my Research, and they suggested I take inspiration from those games, which led me to create a dash inspired by Celeste.

This mechanic is based on a tutorial found at <https://www.youtube.com/watch?v=A-Y1zxJ6oH4>.

I started by scrapping the vertical aspect and switching to 2 directional axes (one for vertical, one for horizontal movement). I also watched many tutorials on how dashes were implemented in other projects, and many of them included a timer. At first, I was confused as to how this was relevant, but I eventually understood that the timer was used for propelling the player to the terminal dash velocity for as long as the timer went for, and when the timer ran out, the velocity went back to normal.

The way that this dash will work is that I will propel the player to a terminal velocity immediately, for 0.08 seconds, then return the velocity back to normal. This is the dash itself. I then tried to implement this myself. I began by creating a timer node in my Player scene, and setting the length to 0.08 seconds. This is how long the acceleration of the player will be.



I then began making some key variables:

```
10  #dash variables
11  var dash_direction = Vector2.ZERO
12  var dashing = false
```

Line 11's dash direction will be used for the direction that the player dashes in. I set this to an empty vector so that it can store an 8-directional vector value.

I then make a Boolean variable called dashing. This will measure if the player is in the dashing state or not. I will use this to trigger the start and end of the dash.

```
204  func start_dash():
205      >| #sets dashing to true
206      >| dashing = true
207      >| #starts dash timer
208      >| dash_timer.start()
209      >|
210  func handle_dash():
211      >| #if player is dashing
212      >| if dashing:
213          >| >| #creates a 8-way dash direction
214          >| >| var dash_directionY = Input.get_axis("move_up", "move_down")
215          >| >| var dash_directionX = Input.get_axis("move_left", "move_right")
216          >| >| #sets velocity to terminal velocity immediately in a dash-like fashion
217          >| >| velocity.x = dash_directionX * DASH_SPEED
218          >| >| velocity.y = dash_directionY * DASH_SPEED
219
220  func _on_dash_timer_timeout() -> void:
221      >| #after the timer has run out, the player will no longer be dashing
222      >| dashing = false
```

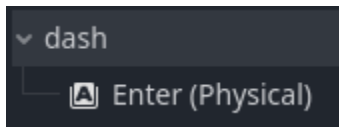
Line 206 sets dashing to true, so handle_dash()'s if statement runs and creates the 2 axes for horizontal and vertical movement. Line 208 then starts the dash timer which controls the length of the dash.

Line 217 & 218 then affect the player's velocity, setting them to DASH_SPEED in the direction that the player has input.

I then linked the timer to the script, creating a timeout function. This will run when the time left on the dash timer reaches 0. So, when the timer runs out, dashing will be set to false, the condition on Line 212 will no longer run, and the velocity of the player will return to normal.

I then needed a way to trigger the dash with a key press. As I mentioned before, the dash key will be Enter. So, I went to the Input Map in Project Settings and created a custom key called "dash" mapped to Enter.

This was the same process I used for the custom mapping of the WASD movement keys.



I now could create a trigger for the dash.

```
67 #creates player moving function
68 func handle_movement(direction):
69     #ensures that movement is disabled during the dash
70     if dashing == false:
71         #if direction has any value apart from 0 (i.e. moving anywhere at all)
72         if direction:
73             #moves in the direction being pressed at the speed of SPEED
74             velocity.x = direction * SPEED
75         else:
76             #slows down player until horizontal speed = 0, at the speed of SPEED
77             velocity.x = move_toward(velocity.x, 0, SPEED)
78
79     #initiates dash if Enter is pressed and the player is able to dash
80     if Input.is_action_just_pressed("dash"):
81         #starts dash function
82         start_dash()
```

I chose to place the trigger in the movement function; it felt the most appropriate since it was a branch of movement and it was more efficient than making another function just to check for Enter being pressed. It then runs the start dash function which starts a chain reaction of the dash being triggered.

```
30 #calls each movement function so that they are always active and being called
31 func _physics_process(delta):
32     get_direction()
33     apply_gravity(delta)
34     handle_movement(get_direction())
35     handle_jump()
36     handle_wallClimbing()
37     handle_crouchWalk(get_direction())
38     handle_wallJump()
39     handle_animations(get_direction())
40     handle_dash()
41     move_and_slide()
```

I also put the handle_dash function in the physics_process function so that it would continuously check if dashing is set to true.

This version of the dash can be seen in the video labelled “**27. Dash v2**”. It can be seen in the video that dashing in any upwards direction sends the player much further than dashing horizontally (which is acting how I want it to). I thought that this was because of the direction being different, because that is the only thing that could be different

between the vertical velocity and horizontal velocity, which can be seen in the screenshot below.

```
#creates a 8-way dash direction
var dash_directionY = Input.get_axis("move_up", "move_down")
var dash_directionX = Input.get_axis("move_left", "move_right")
#sets velocity to terminal velocity immediately in a dash-like fashion
velocity.x = dash_directionX * DASH_SPEED
velocity.y = dash_directionY * DASH_SPEED
```

As can be seen, the velocities are being multiplied by the same constant DASH_SPEED.

So, I changed the 2 direction axes into 1 direction vector, in an effort to fix this issue. I also did this to minimise the amount of memory that the game could take up while running.

```
203 func start_dash():
204     #creates a 8-way dash direction
205     dash_direction = Input.get_vector("move_left", "move_right", "move_up", "move_down")
206     #sets dashing to true
207     dashing = true
208     #starts dash timer
209     dash_timer.start()
210
211 func handle_dash():
212     #if player is dashing
213     if dashing:
214         #sets velocity to terminal velocity immediately in a dash-like fashion
215         velocity = dash_direction * DASH_SPEED
216
217 func _on_dash_timer_timeout() -> void:
218     #after the timer has run out, the player will no longer be dashing
219     dashing = false
```

On line 205, instead of creating 2 axes for the vertical and horizontal directions individually, I create a vector, which is capable of storing all 8 directions at once, retrieved from the combination of the values (1, 0, -1) returned by all WASD movement keys.

I then use this dash direction multiplied by DASH_SPEED to affect the overall player velocity.

This had a small effect on the diagonal upwards dash, minimising it so that it was in the middle of the distance of the vertical and horizontal dash. But it didn't solve the

largened strength of the vertical dash. This can be seen in the video labelled “**28. Dash v3**”.

After careful consideration and many more attempts, I couldn’t find an efficient reason or solution as to why the vertical dash was so much more powerful than the horizontal. And so, I resorted to manually decreasing the vertical velocity.

```
205 ▾ func start_dash():
206     » #creates a 8-way dash direction
207     » dash_direction = Input.get_vector("move_left", "move_right", "move_up", "move_down")
208     » #sets dashing to true
209     » dashing = true
210     » #starts dash timer
211     » dash_timer.start()
212     »
213 ▾ func handle_dash():
214     » #if player is dashing
215     ▾ if dashing:
216         » » #sets velocity to terminal velocity immediately in a dash-like fashion
217         » » velocity = dash_direction * DASH_SPEED
218     ▾ if velocity.y < 0:
219         » » » #reduces vertical velocity
220         » » » velocity.y *= 0.65
```

Line 218 now checks if the vertical velocity of the dash is more than 0, ensuring that the player is dashing upwards in any way. If true, then Line 220 reduces the vertical velocity by a factor of 0.65x.

Testing this reduced the vertical velocity to a point where it was similar to the horizontal dash distance, which can be seen in the video labelled “**29. Dash v4**”.

After showing this to my stakeholders and them testing the dash, they pointed out how you should only be able to dash once, to prevent players from abusing this mechanic. So, I implemented a Boolean variable called can_dash. This will determine whether the player can currently dash or not. If can_dash is false and the player attempts to dash by pressing Enter with a direction, nothing will happen.

```
10     #dash variables
11     var dash_direction = Vector2.ZERO
12     var dashing = false
13     var can_dash = false
```

I can now use this new variable in a new function that allows the player to dash in certain situations

```

229  ▾ func check_dash():
230      »    #can only dash in level 3
231  ▾ »    if Global.level3:
232      »        »    #can only dash once; resets when player is on floor/wall
233  ▾ »        »    if is_on_floor() or Global.onWall:
234      »            »    can_dash = true

```

This function will set can_dash to true only in certain situations where conditions are met, allowing the character to perform a dash again. In this function, the player is only allowed to dash if they are in level 3 (THIS IS ALSO SET TO LEVEL 2 FOR THE SAKE OF TESTING AS LEVEL 3 HAS NOT BEEN MADE YET). On top of this, the player can only dash once, until they land on the floor or a wall. I have purposely left out an else statement because I needed can_dash to be left as true, instead of always being updated with the correct value.

For example, the player lands on the floor; can_dash is now true. But if the player then simply jumps off of the floor, can_dash will then be set to false, because is_on_floor() is no longer true. I do not want this. And so, this is why I left out the else statement.

I then input this into the functions:

```

81      »    »    #initiates dash if Enter is pressed and the player is able to dash
82  ▾ »    »    if Input.is_action_just_pressed("dash") and can_dash:
83      »        »        »    #starts dash function
84      »        »        »    start_dash()

```

```

211  ▾ func start_dash():
212      »    #creates a 8-way dash direction
213      »    dash_direction = Input.get_vector("move_left", "move_right", "move_up", "move_down")
214      »    #player can only dash once so they can no longer dash
215      »    can_dash = false
216      »    #sets dashing to true
217      »    dashing = true
218      »    #starts dash timer
219      »    dash_timer.start()

```

The fully functional version of the dash has now been made, the results of which can be seen in the video labelled “**30. Dash v5**”.

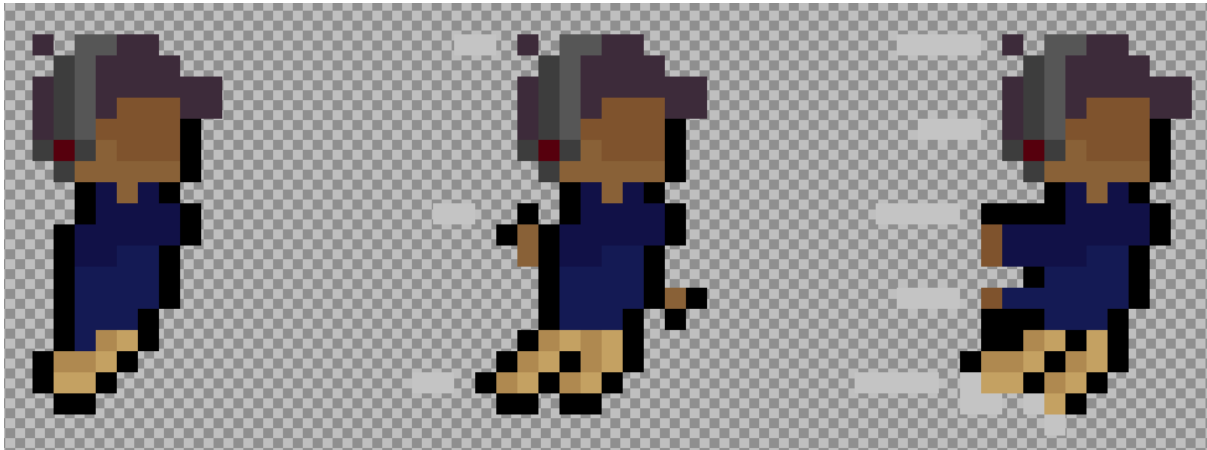
I now moved onto the animation of the dash.

Dash Animation

I decided that if I wanted to accentuate the effect of the dash, I would need to include an animation that detailed the player going fast. My stakeholders agreed with this feature that follows success criteria 22.

I started by going back to my Player scene and adding another animation to the AnimatedSprite2D, calling it dashing.

I then input my sprite frames that I had created for the dash.



I then went to the handle animation's function and added another condition for while dashing is true, for the animation to play.

```
157 >| #if player is dashing
158 v>| if dashing == true:
159 >| >| #play dashing animation
160 >| >| animated_sprite.play("dashing")
```

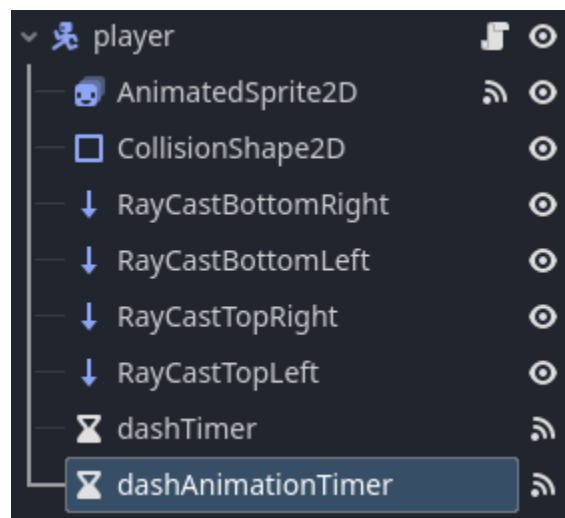
Now when dashing is true and the player is dashing, the suitable animation will run .

These results can be seen in the video labelled “**31. Dash animation v1**”. In the video, a small change in animation can be seen. It’s very quick and hard to spot, but that shift in animation is actually the dash animation playing. The reason it’s so quick and hard to see is because it will only play while dashing is true, and dashing is only true for the length of the dash itself, which is determined by the length of the dash timer (0.08 secs). After dashing is set to false again, it doesn’t matter what stage of the animation is playing; it will stop after the dash timer runs out.

To fix this, I needed to change the length of the dash, because the animation depended on that. I first of all increased the duration of the dash from 0.08 to 0.5, and in turn decreased the value of DASH_SPEED from 500 to 180. I did this because I needed to increase the time taken for the dash without affecting the distance of the dash too much.

This effect can be seen in the video labelled “**32. Dash animation v2**”. In this video, it can be seen that the dash is now long enough for the animation to play fully, but the snappiness of the dash is now gone, and the player will travel slowly for a longer amount of time. This takes away from the purpose of the “dash”. I consulted my stakeholders on what to do, to which they tested both versions of the dash, and responded with approval towards the first, snappier version of the dash. They told me to just find a way to make the animation play for longer without affecting the dash function itself.

I did this by implementing another timer node, dedicated to the dash animation.



I set this to a time longer than the dash itself, but a suitable time for the animation to play for, which was 0.4 seconds.

I now can start the dash animation timer when the player starts their dash:

```
206 ▾ func start_dash():
207     » #creates a 8-way dash direction
208     » dash_direction = Input.get_vector("move_left", "move_right", "move_up", "move_down")
209     » #player can only dash once so they can no longer dash
210     » can_dash = false
211     » #sets dashing to true
212     » dashing = true
213     » #starts dash timer
214     » dash_timer.start()
215     » dash_animation_timer.start()
```

I then switched the dependency of the dash animation from the dash timer to the dash animation timer.

```

152  >|  #for as long as dash animation timer runs
153  ▼ >|  if dash_animation_timer.time_left > 0:
154  >|  >|  #play dash animation
155  >|  >|  animated_sprite.play("dashing")

```

Line 153 checks if the animation timer has run out. If not, then the dashing animation plays.

I also had to switch it back to the idle and other normal animations once the animation timer had run out.

```

240  ▼ func _on_dash_animation_timer_timeout() -> void:
241  >|  #switches animation back to normal idle after dash is over
242  ▼ >|  if animated_sprite.animation == "dashing":
243  >|  >|  animated_sprite.play("idle")

```

Line 240 is a function that runs when the dash animation timer runs out, which checks if the dashing animation is still playing (line 242), in which case the idle animation will play. This doesn't force specifically the idle animation to play, just taking the handling of animations back to normal, specifically just stopping the dash animation from playing now that the dash is over (Line 243).

The final version of the dash has now been made. It can be seen in the video labelled **"33. Dash and dash animation final"**.

Level 3

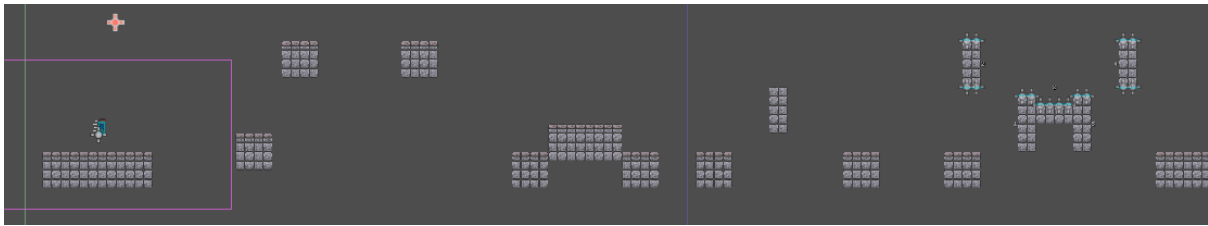
I felt that, because most of the necessary functionality was almost completed in the game, I could focus on the final level. This level will be the longest and the hardest, combining all the skills and abilities collected in the last levels, as well as a new ability.

I wanted to make this level somehow different from the rest, and I chose to do this in terms of the decoration and design of the levels details. This included using a parallax background to create a new perspective view of the background in parallel to the player's movements.

Parallax2D Background

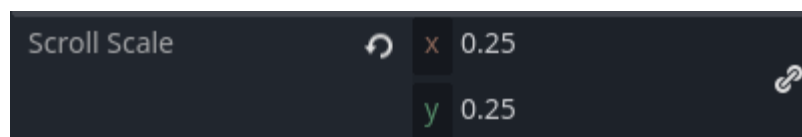
A Parallax2D node is used to create the illusion of depth in a 2D game by making background layers move at different speeds when the player moves. I can use this for the background, and other details that fall into the space theme I'm going for.

Now to implement it. I created a new level scene and did the first steps of everything I did for the first 2 levels. This included making the obstacles and platforms as normal. I also added spikes to certain obstacles which I created earlier.

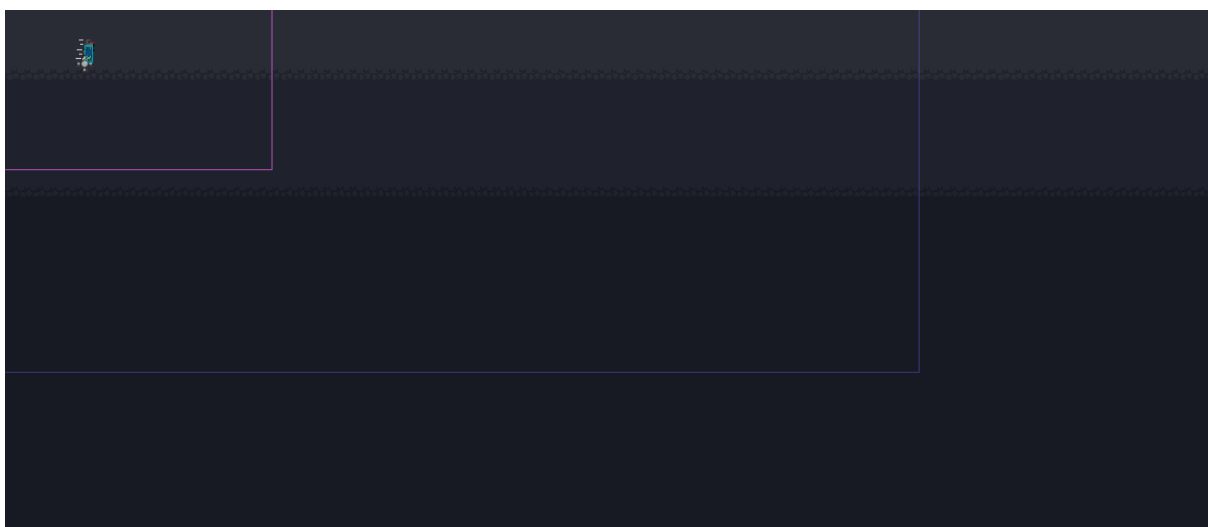


I began by creating a Parallax2D node and making a TileMap as a child of that. This will be for the background itself, while the Parallax2D node will provide the parallax view of it. I designed the background TileMap like normal, with darker recoloured versions of the sky tiles.

The main feature of the Parallax2D node is the function of the scroll scale. This determines how fast the TileMap will move relative to the player. It can be set to 0, which gives the effect of not moving at all on the players screen. It will essentially stay on the players screen regardless of how fast they are moving. It can also be set to 1, which means it will behave normally (i.e. how it would look without a Parallax2D in the first place).



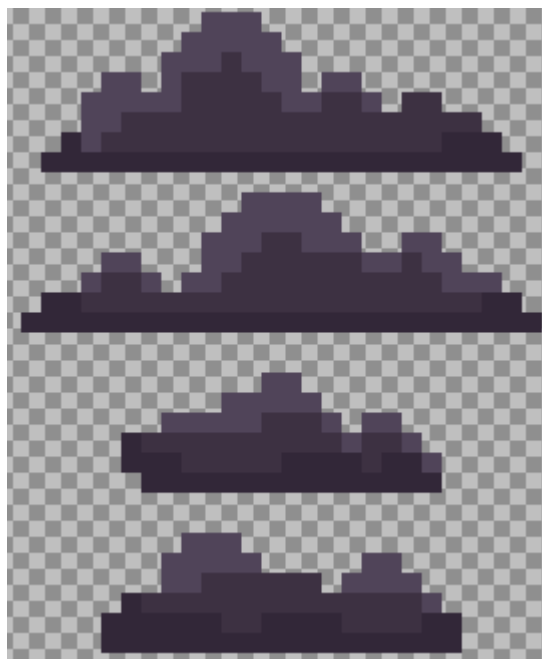
For this, I've set the scroll scale to 0.25, so that a bit of movement is visible, but not enough that its moving as fast as the player. This makes it look far away from the player, so it won't move a lot. An advantage to using Parallax2D nodes is that they move slower than the player, so I don't need to make them go as far as the platform so I don't need to waste time designing the far extent of the background.



Although this looks normal right now, while running along the platforms, the background will move less in parallel to the platform. This can be seen in the video labelled “**35. Level 3 v2 parallax**”. The comparison between parallax and no parallax can also be seen in the video between this video and the other video labelled “**34. Level 3 v1 no parallax**”.

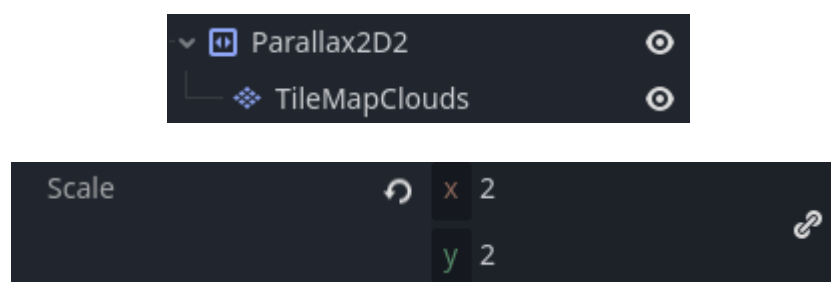
Next, I wanted to add some decoration to the background of the level to emphasise the space theme of the level. I did this using clouds.

I intended to add another parallax node for the clouds with a different scroll scale to make them move at different speeds in relative to the player. However, I first had to design these clouds.

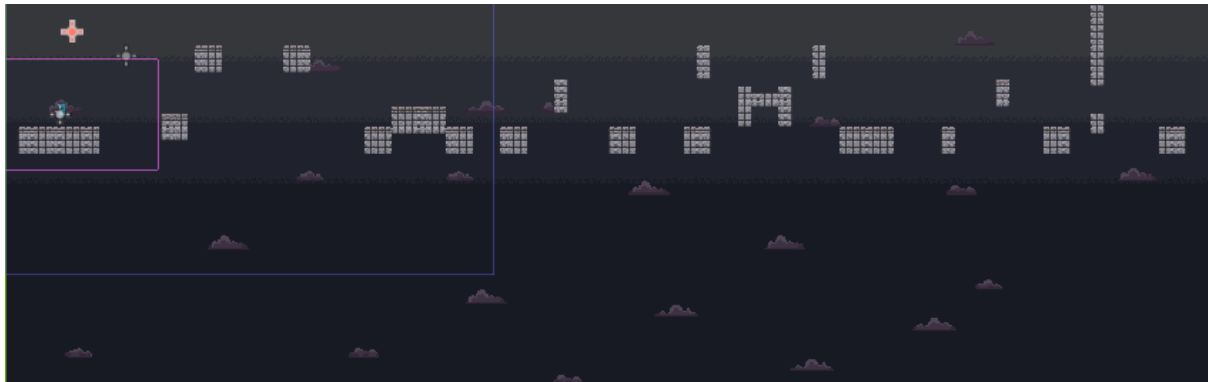


I based this design of many designs from the internet, which I then turned purple to take away from the realism, while adding to the fantasy of the level and also to keep the level darker.

I then made another Parallax2D, with another TileMap, which I scaled up to 2x to make the clouds bigger.



I then could start drawing my clouds around the level.



I set this Parallax2D node's scroll scale to 0.75; so, it will move more than the background, giving the illusion that the clouds are closer to the player than the background is.

This is the majority of the creation of the third level of my game. From this point on, I just have to update all the scripts that are dependent on level 3, such as the level selector buttons, music scripts etc.

The playthrough of level 3 can be seen in the videos above.

Interview

At this point, I interviewed one of my stakeholders on their opinion on my progress with the dash and the 3rd level.

Shazil

What are your brief thoughts about the design for level 3?

I like how the sky changed colour. The spikes are also good because they're another way for you to die and they blend in well to the level design. I also like the clouds because they really show how you're in the sky and in space. The level overall just isn't too big of a step up from level 2 in terms of difficulty, mostly because of the dash, but if you look at it straight up, the actual design of the level is very different, especially because how far the platforms are from each other.

Are the obstacles challenging while remaining enjoyable (Success Criteria 5)?

They might be a bit too challenging, but experienced players who are familiar with the controls will find it fine. There's a reason the hardest obstacles are saved for the last level.

Does the course provide a good challenge, while making good use of the dash ability?

Yes.

What are your brief thoughts about the dash function?

The dash is very good; it's a brand-new mechanic that is good to play with and it feels satisfying to use. It also allows for much harder obstacles.

What are your brief thoughts about the dash animation?

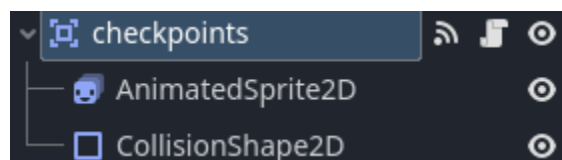
I really like it. No complaints

Checkpoints

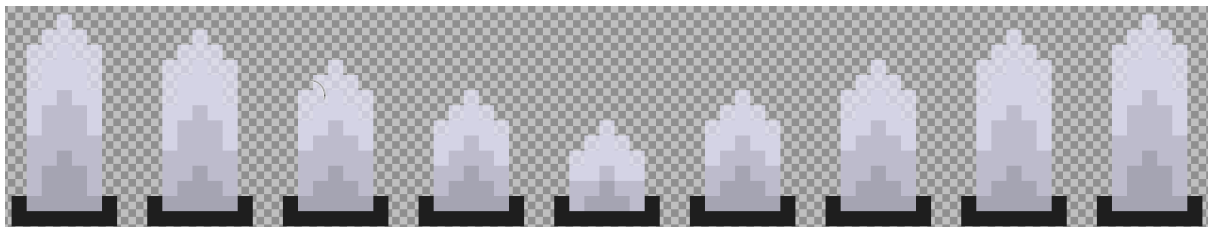
This is a common feature in games designed to make the player's life a little bit easier. The main function of it is to change the player's spawn point so that when they die, they don't have to restart the entire level and can save some progress. Checkpoints will be activated by walking through them when finding them throughout the level.

The way I plan to implement it is using the class system in Godot to recognise the different checkpoints being activated and changing the spawn point dependent on those checkpoints. I am basing this method off of a tutorial found at (https://www.youtube.com/watch?v=O7iGt8H_FB4&t=658s)

I began implementing this by creating a new checkpoints scene, with a root Area2D node, and children nodes consisting of an AnimatedSprite2D and CollisionShape2D.



I then created the animations for the normal state and the activated state of the checkpoint. I included separate animations for both states to easily signify to the player if the checkpoint is activated or not.



This is the design of the checkpoint. It resembles a beacon that can easily stand out to the player once activated. This animation will be looping. However, the sprite for when the checkpoint is idle (not activated) will just be the black frame of the checkpoint visible. This idle sprite is set to autoplay so it plays by default, and is only 1 frame so looping or FPS of animation is irrelevant here.

Moving on to the CollisionShape2D, I created a rectangle shape larger than the sprite itself to ensure no errors can be made and the process of activating the checkpoint is easier. This is a quality-of-life decision for the player.

Now I could start inputting the signal functions into the script for the checkpoint.

```
1  extends Area2D
2  class_name Checkpoint
3
4  var activated = false
5
6  @onready var checkpoint_sprite: AnimatedSprite2D = $AnimatedSprite2D
7
8  func _on_body_entered(body: Node2D) -> void:
9      >| #can only be activated if not already
10     >| if !activated:
11         >| activate()
12
13     func activate():
14         >| #current checkpoint is this checkpoint
15         >| Global.current_checkpoint = self
16         >| #checkpoint is activated
17         >| activated = true
18         >| #plays activation animation
19         >| checkpoint_sprite.play("activated")
```

Line 2 creates the Checkpoint class. I will use this along with the Player class to recognise the positions of the items later.

Line 4 creates an activated state variable that will be used to signal if the checkpoint is activated or not. It is set to false for now.

Line 6 creates a reference to the AnimatedSprite2D of the checkpoint in order to start the animation when it is activated.

If the CollisionShape2D of the checkpoint is entered on Line 8, then the program checks if activated is still false. This ensures that the checkpoint cannot be activated twice. If activated is false for this checkpoint, then the activate function is called.

Line 13 creates the activate function which tells the current_checkpoint variable in the Global script which checkpoint is currently activated. This will be further explained in the Global script. The activated state is set to true and the activated animation plays.

```

19 #creates references of the instances of the current checkpoint and the player
20 var current_checkpoint : Checkpoint
21 var player : Player
22
23 func respawn_player():
24     #if any checkpoint has been activated
25     if current_checkpoint != null:
26         #move the player to the checkpoint
27         player.position = current_checkpoint.global_position
28     #if no checkpoint has been activated
29     else:
30         #respawn player at the start of the level
31         get_tree().reload_current_scene()

```

This is the global script. This is where I will be making the dying function, which utilises the checkpoints.

First, Line 20 and 21 create references to the instances of the current checkpoint, which receives a signal from the checkpoint when it is activated by the player, and the player, which is just the player.

Line 23 begins the respawning function, which checks if any checkpoints have been activated at all. If this is true, then Line 27 changes the player position to the global position of the checkpoint, essentially respawning them.

However, if no checkpoint has been activated, then there is nowhere to teleport the player to, so the program simply reloads the scene and the player is respawned at the start of the level.

The testing of this can be seen in the video labelled “**36. Checkpoints v1**”. It can be seen that the player does not respawn cleanly, and they continue falling at a higher velocity until they fall through the screen over 60 times a second and disappear. This is because the checkpoint isn’t respawning the player at the correct point, and the player’s velocity is being carried over to what it was when they died. This accumulated after each death until they weren’t visible on the screen anymore.

I fixed this by changing the code from respawning the player at the checkpoints global position to its normal relative position.

```

27     player.position = current_checkpoint.position

```

This fixed the issue, and the result can be seen in the video labelled “**37. Checkpoints v2**”. This is the final version of the checkpoint and respawning function.

Game Progress

Game saving and loading

Game saving is a good feature to have in games that can't be completed in one gaming session. For the majority of people (including my stakeholders) this is the case. So, I should have a feature that allows them to store their progress if they wish to stop playing, so that they don't have to start again from scratch each time they launch the game. This feature was included in accordance with success criteria 16, stating "include a game saving system".

The way that I will go around saving the progress of the player is by using file handling to store the values that I need to in a text file, which can be written to and read from when needed. However, I first needed to decide which values were important enough to save in the first place. I consulted my stakeholders in an interview about this:

- Shazil
 - o Progress in the level
 - o Number of coins collected
- Adham
 - o How far you got in the level
 - o Score
- Ismail
 - o Where you are in the level
 - o What checkpoint did you reach
 - o Number of coins you collected on the way there
 - o Current score

From these interviews, I knew that progression was the main thing that I needed to save, so I needed to prioritise that.

I began by researching how data was commonly saved in Godot, and most of the results were related to text files. Specifically, converting game data into text, writing that data into a file and storing it, then being able to read it and converting the text back into game data.

Now to implement this.

I began by going to the Global script, as this is where the functions for saving and loading.

```

37     const save_folder = "user://saves"
38
39     var save_data = player.position
40
41     func save_game():
42         if player:
43             var file = FileAccess.open(save_folder, FileAccess.WRITE)
44             file.store_var(save_data)
45             file.close()
46
47     func load_game():
48         var file = FileAccess.open(save_folder, FileAccess.READ)
49         player.position = file.get_var(save_data)
50         file.close()

```

In this prototype, I start off by just saving the player's position, as it is my priority. I put it into a variable called `save_data`.

I also prepare a constant which has the address of the directory of where I will be saving my data. Saving and loading data in Godot is very efficient because it creates a new save file whenever called upon in the user files, so I don't need to dedicate space towards the save file. This can save memory usage and storage space for the game. I then open the save file at the directory with the write command ready. I then use the command `store_var` to write the `save_data` onto the file, and then close it. In theory, this method should have worked, but it didn't. This was because of this error:

Invalid access to property or key 'position' on a base object of type 'Nil'.

This essentially means that the data types for things I am using are not compatible with each other. For example, the `player.position` key is a `Vector2` data type, and the `save_data` variable is not compatible with this. One way to fix this is to set the `save_data` variable to the compatible data type.

```

39     var save_data: Vector2 = player.position

```

But this will cause another error in the `load_game` function (on Line 49).

Error at (49, 36): Invalid argument for "get_var()" function: argument 1 should be "bool" but is "Vector2".

This error is caused because a `Vector2` data type cannot be used in the command `get_var()`. This function retrieves a certain variable from the file, but it isn't compatible with the `Vector2` data type.

At this point, I consulted one of my stakeholders, who is experienced in file handling in order to save game data. He suggested that I convert all the necessary data into strings, which will make it much easier to store.

I then moved on to this method. I began by scrapping the old version of the save_game and load_game functions in order to rewrite them.

First of all, I researched different methods of saving data in Godot. I found that a commonly used format was JSON, since it is supported by the majority of programming languages, including GDScript, and it is directly used by the Godot Engine to store user data.

I then researched the Vector2 data type, and found that it is mostly used to store X and Y coordinates together. With this knowledge, I began to store the position of the player as separate X and Y positions

```
52  ▾ func save_game():
53      » #puts player position into an array
54      » var save_data = [player.position.x, player.position.y]
55      » #opens file to write to it
56      » var file = FileAccess.open(save_folder, FileAccess.WRITE)
57      » #stores the string version of the save data
58      » file.store_string(JSON.stringify(save_data, "\t"))
59      » #closes the file
60      » file.close()
```

I now put the X and Y coordinates of the player in an array called save_data. Later, I can input more pieces of data I wish to save into this array. I then open the file with the write command.

On Line 58, I first turn the save_data into a string with the command stringify. I then put it into the JSON format for easy storage. I then write the string value of the player position onto the text file.

I then close the file after writing the data onto it.

```
62  ▾ func load_game():
63      » #opens the file to read from it
64      » var file = FileAccess.open(save_folder, FileAccess.READ)
65      » #retrieves data from file as string
66      » var content = file.get_as_text()
67      » #closes the file
68      » file.close()
69      » #converts string data back into an array
70      » var save_data = JSON.parse_string(content)
71      ▾ » if save_data:
72          » » #converts array data into Vector2 (i.e. data that can be used for positioning)
73          » » player.position = Vector2(save_data[0], save_data[1])
```

This is the function for loading the save_data from the text file written to by the save_game function.

The first step is to open the file again with the read command. Line 66 retrieves all string data from the file and stores it in a variable called content. I then close the file, as I have retrieved everything I need from it. Line 70 uses the JSON format to format the string data from the file back into the original array format that it was saved from in the save_game function. I then verify that there is indeed some data in the text file. This form of input validation fixes the zero-input error. Line 73 then converts the pieces of data in the save_data variable into the format compatible with the Vector2 data type as well as the format that the data was entered in the first place (X coordinate, Y coordinate). It then inputs it into the position property of the player, changing the player's position and moving it to wherever the game was saved at.

I then needed a way to use these functions in the game. I chose to do this in the pause menu. I would create 2 more buttons: 1 for saving progress and 1 for loading progress.



I put these 2 new buttons in the bottom left corner away from the main functionality of the menu because these are additional features, instead of the core features. After creating these 2 new buttons, I attached their signal pressed functions to the pause menu script.

```
80  ▼ func _on_save_pressed() -> void:
81    > Global.save_game()
82
83  ▼ func _on_load_pressed() -> void:
84    > Global.load_game()
```

The only purpose of these buttons is to call the save and load functions. This is also why I created these functions in the Global script; so that I can refer to them and use them anywhere in the game necessary.

To summarise, the `save_game` function takes the current player position, separates the X and Y coordinates and stores them in an array as string values. This array is then stored in a text file. The `load_game` function then takes the text file, takes all the data from it and puts it into a variable, which is then formatted back into an array, where the individual pieces of data (X & Y coordinate) are then input back into the game.

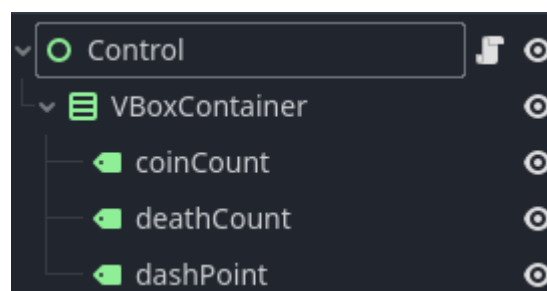
Another advantage of game saving is that the data is being saved externally from the program, in a text file, so the data isn't overridden when the game is launched again. This means that the user can close the game, come back and still have their data.

This is the main functionality of the game saving and loading function. This can be shown in the video labelled “**38. Game saving & loading**”. Later on, I will add support for more pieces of data that were suggested by my stakeholders like the coin count and score.

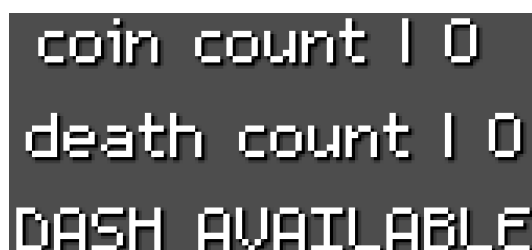
HUD

I will use this feature to show the player's statistics throughout the game, as well as some helpful reminders. The main stats that I will show will be whether the player can dash or not (only in level 3), the death count and the coin count. This follows success criteria 19.

I began by making a new User Interface scene, along with a `VBoxContainer` which included labels for each piece of information



I then tweaked the text of each label after aligning the `VBoxContainer` against the top left corner of the players screen.

A screenshot of the game's HUD display. It shows three lines of text in a pixelated font: 'coin count | 0', 'death count | 0', and 'DASH AVAILABLE'.

I then created a script for the scene.

```
1  extends Control
2
3  #initiates label nodes for manipulation
4  @onready var coin_count: Label = $VBoxContainer/coinCount
5  @onready var death_count: Label = $VBoxContainer/deathCount
6  @onready var dash_point: Label = $VBoxContainer/dashPoint
7
8  #function that runs every frame
9  func _physics_process(delta: float) -> void:
10     #creates dash indicator
11     if Global.can_dash == true:
12         dash_point.text = "DASH"
13     else:
14         dash_point.text = "NO DASH"
15     #only displays dash indicator in level 3
16     if Global.level3:
17         dash_point.show()
18     else:
19         dash_point.hide()
20     #updates with death and coin count
21     death_count.text = "death count | " + str(Global.death_count)
22     coin_count.text = "coin count | " + str(Global.score)
```

Since I wanted to create an indicator for the dash, I needed to check the value of `can_dash`. But this is a local variable from the player script, so I couldn't access its properties. I fixed this by making it a global variable (and I also fixed all the references towards `can_dash` in the player script).

Line 3, 4, 5 create references to the label nodes so that I can change their text. I then created the `_physics_process` function that would run and update after each frame in the game.

In this function, I will check if `can_dash` is true, in which case I will set the dash indicator label's text attribute to show "DASH" to tell the player that a dash is available. On the other hand, if `can_dash` is false, the label will show "NO DASH". Additionally, this label will only be shown on the HUD if the player is in level 3, because the dash is not available in level 1 or 2, so there is no need for it.

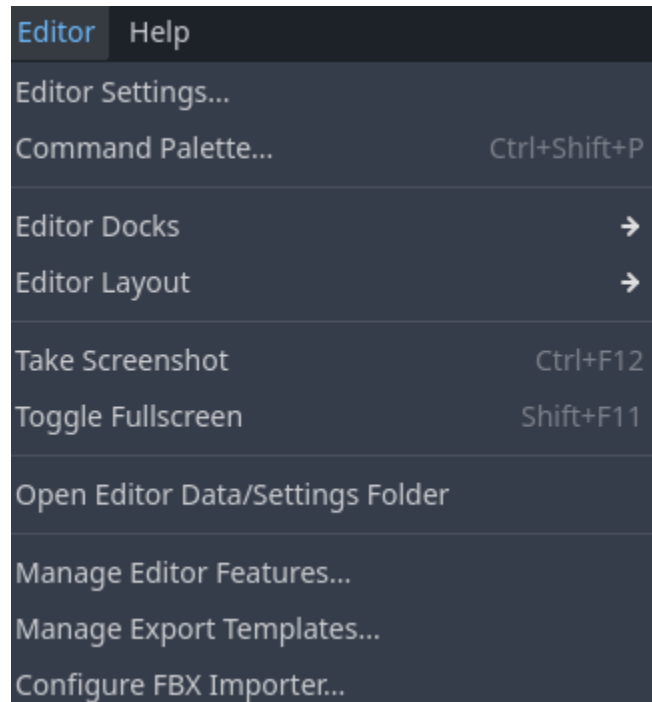
Line 21 and 22 update the death and coin count with the string value of the death count and score in real time.

This is the final version of the HUD, which can be seen in the video labelled "**39. Hud**".

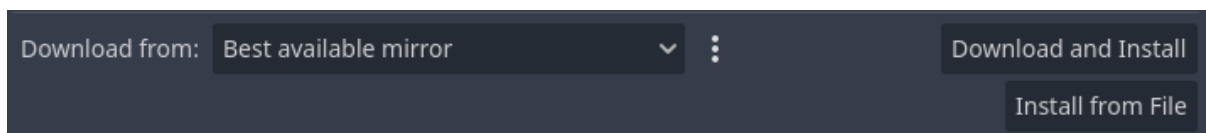
Exporting the project

I am at the end of my development stage now, so I will now export the project, creating a single executable file to launch my game, instead of the player having to download the Godot Engine in order to run the game.

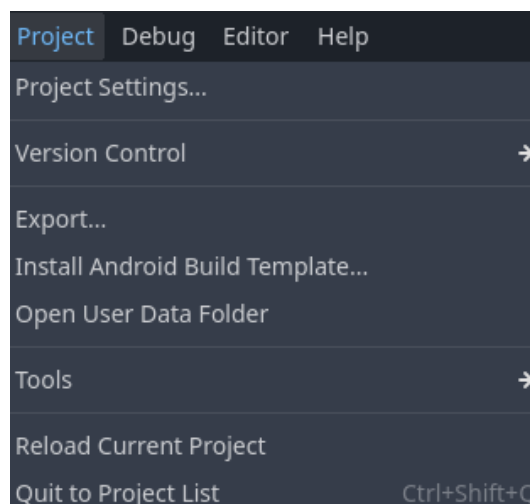
To do this, I went to the Editor tab and selected Manage Export Templates.

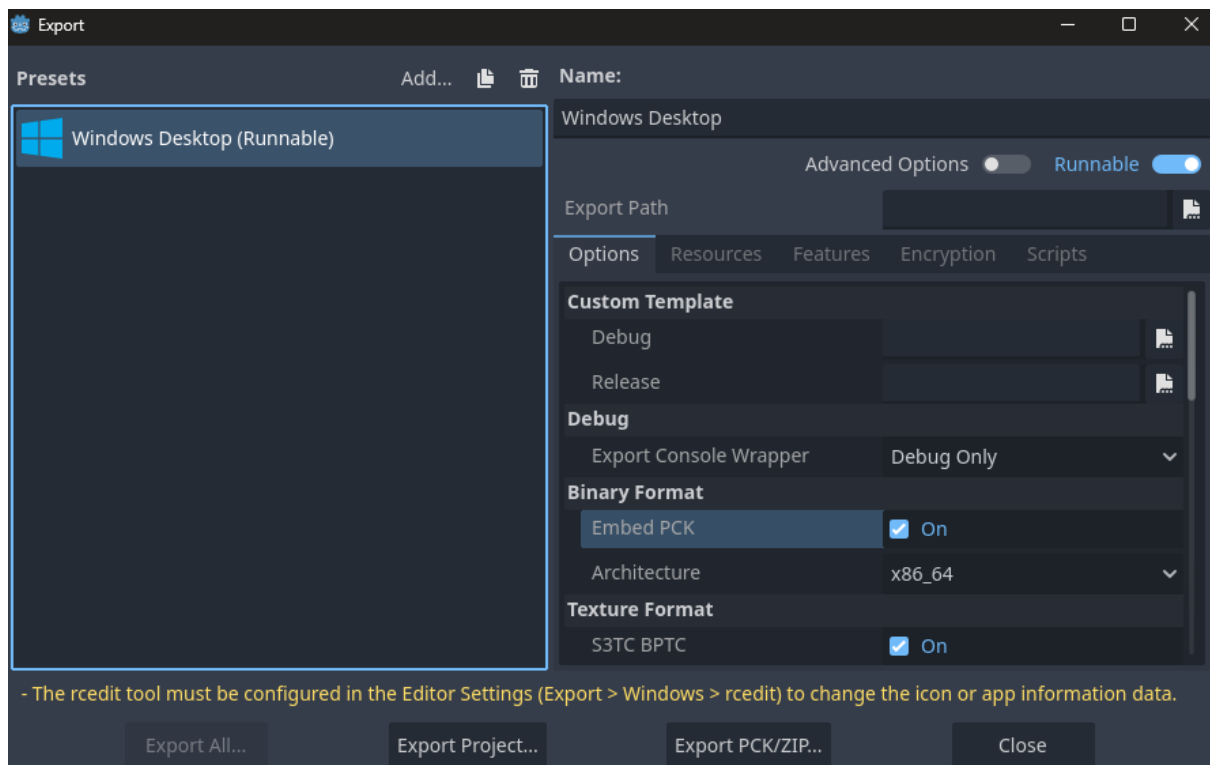


I then downloaded the correct export templates for my project



After downloading the correct templates, I could now export the project by clicking Export.

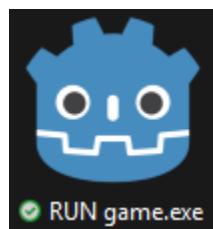




I selected the tickbox titled “Embed PCK” which will compress the entire project into 1 executable file.

I then chose the Windows export preset so that the project would be compatible with Windows. But in the future, I can create more support for OS’s like macOS.

After choosing a place in storage of where to put the .exe file, the project is now successfully exported.



Evaluation

At this point, I have come to the end of the time frame for my NEA. I will now test everything I have done in the development stage.

Testing for the evaluation

Test number	Video proof	Name of test	Type of test	Expected output	Instructions	Actual output
1	"tests 1-24"	Collision with wall/floor	Valid	Stops movement into wall	Walk to into wall or fall into floor	Expected output
2	"tests 1-24"	"W"	Valid / borderline	Jump	Press W	Expected output
3	"tests 1-24"	"A"	Valid / borderline	Move left	Press A	Expected output
4	"tests 1-24"	"S"	Valid / borderline	Move down	Press S	Expected output
5	"tests 1-24"	"D"	Valid / borderline	Move right	Press D	Expected output
6	"tests 1-24"	"Spacebar"	Valid / borderline	Jump	Press spacebar	Expected output
7	"tests 1-24"	Up	Invalid	Do nothing	Press up arrow	Expected output
8	"tests 1-24"	Left	Invalid	Do nothing	Press left arrow	Expected output
9	"tests 1-24"	Down	Invalid	Do nothing	Press down arrow	Expected output
10	"tests 1-24"	Right	Invalid	Do nothing	Press right arrow	Expected output
11	"tests 1-24"	"Esc"	Valid	Pause game	Press escape key	Expected output
12	"tests 1-24"	Climb idle	Valid	Climb on wall	Hold shift on wall	Expected output
13	"tests 1-24"	Climb up	Valid	Climb up wall	Hold shift on wall and press W	Expected output
14	"tests 1-24"	Climb down	Valid	Climb down wall	Hold shift on wall and press S	Expected output

Test number	Video proof	Name of test	Type of test	Expected output	Instructions	Actual output
15	"tests 1-24"	Do nothing	Valid	Play idle animation	Don't press any keys in any	Expected output

					game scene	
16	"tests 1-24"	Jump movement	Valid	Jump animation plays	Press W or spacebar	Expected output
17	"tests 1-24"	Run left movement	Valid	Flip sprite horizontally and play run animation	Press A	Expected output
18	"tests 1-24"	Run right movement	Valid	Play run animation	Press D	Expected output
19	"tests 1-24"	Climb on wall	Valid	Change to climb sprite	Go against a wall and hold shift	Expected output
20	"tests 1-24"	Climb up/down wall	Valid	Play climbing animation	While on a wall and holding shift, press W or S	Expected output
21	"tests 1-24"	Fall down	Valid	Play falling animation	Fall down a significant height	Expected output
22	"tests 1-24"	Crouch and do nothing	Valid	Play crouch idle animation	Hold S	Expected output
23	"tests 1-24"	Crouch and jump	Valid	Play crouch jump animation	Hold S and press W or spacebar	Expected output
24	"tests 1-24"	Crouch and run left/right	Valid	Play crouch run animation in appropriate direction	Hold S and press A/D	Expected output

Test number	Video proof	Name of test	Type of test	Expected output	Instructions	Actual output
25	"tests 25-32"	Level Selector scene (LMC)	Valid	Open Level Selector scene	Press "Play" in title scene	Expected output
26	"tests 25-32"	Settings scene (LMC)	Valid	Open Settings scene	Press "Settings" in title scene	Expected output
27	"tests 25-32"	Quit game (LMC)	Valid	Quit game	Press "Quit" in title scene	Expected output
28	"tests 25-32"	Click anywhere	Invalid	Do nothing	Press LMC anywhere on	Expected output

		where there is no button (LMC)			the screen where there is no button	
29	"tests 25-32"	Level Selector scene (RMC)	Invalid	Do nothing	Press "Play" in title scene with right click	Expected output
30	"tests 25-32"	Settings scene (RMC)	Invalid	Do nothing	Press "Options" in title scene with right click	Expected output
31	"tests 25-32"	Quit game (RMC)	Invalid	Do nothing	Press "Quit" in title scene with right click	Expected output
32	"tests 25-32"	Click anywhere where there is no button (RMC)	Invalid	Do nothing	Press RMC anywhere on the screen where there is no button	Expected output

Test number	Video proof	Name of test	Type of test	Expected output	Instructions	Actual output
33	"tests 33-38"	Mute music button (LMC)	Valid	Mute music	Press "Mute music" button in audio scene with left click	Expected output
34	"tests 33-38"	Mute SFX button (LMC)	Valid	Mute SFX	Not implemented	N/A
35	"tests 33-38"	Unmute music button (LMC)	Valid	Unmute music	Press "Mute music" button in audio scene with left click while music is muted	Expected output
36	"tests 33-38"	Unmute SFX button (LMC)	Valid	Unmute SFX	Not implemented	N/A

37	"tests 33-38"	Mute music button (RMC)	Invalid	Do nothing	Press "Mute music" button in audio scene with right click	Expected output
38	"tests 33-38"	Mute SFX button (RMC)	Invalid	Do nothing	Not implemented	N/A

Test number	Video proof	Name of test	Type of test	Expected output	Instructions	Actual output
39	"tests 39-50"	Dash (W being held)	Valid	An upwards dash	Press Enter while holding W	Expected output
40	"tests 39-50"	Dash (A being held)	Valid	A leftwards dash	Press Enter while holding A	Expected output
41	"tests 39-50"	Dash (S being held)	Valid	A downwards dash	Press Enter while holding S	Expected output
42	"tests 39-50"	Dash (D being held)	Valid	A rightwards dash	Press Enter while holding D	Expected output
43	"tests 39-50"	Dash (W+A being held)	Valid	An upward left (diagonal) dash	Press Enter while holding W+A	Expected output
44	"tests 39-50"	Dash (W+D being held)	Valid	An upward right (diagonal) dash	Press Enter while holding W+D	Expected output
45	"tests 39-50"	Dash (S+A being held)	Valid	A downward left (diagonal) dash	Press Enter while holding S+A	Expected output
46	"tests 39-50"	Dash (S+D being held)	Valid	A downward right (diagonal) dash	Press Enter while holding S+D	Expected output

47	"tests 39-50"	Dashing without a direction (none of WASD being held)	Invalid	No dash would occur	Press Enter	Expected output
48	"tests 39-50"	Dashing into a wall	Invalid	Player would collide with the wall and not go through	Press Enter while holding a direction key that makes you dash into a wall (A/D)	Expected output
49	"tests 39-50"	Dashing into the floor	Invalid	Player would collide with the floor and not go through	Press Enter while holding S above the floor	Expected output
50	"tests 39-50"	Dashing into the ceiling	Invalid	Player would collide with the ceiling and not go through	Press Enter while holding W below a ceiling	Expected output

Test number	Video proof	Name of test	Type of test	Expected output	Instructions	Actual output
51	"tests 51-55"	Walking into a coin to collect it	Valid	Plays collected animation and despawns	Enter the body of a coin	Expected output
52	"tests 51-55"	Collecting a coin twice before it despawns	Invalid	Only able to collect once, despawns before can be collected again	Enter a body of 1 coin twice before it despawns	Expected output
53	"tests 51-55"	Collecting coin to add to XP	Valid	XP will be added by 25 per coin collected	Enter the body of a coin, then check the XP	Expected output
54	"tests 51-55"	Collecting coin to add	Valid	Score will be added by 1	Enter the body of a coin, then	Expected output

		to coin count / score		per coin collected	check the coin count	
55	"tests 51-55"	Dying to reset score & XP	Valid	Score & XP will reset upon death	Die, respawn, then check the coin count & XP	Expected output

Test number	Video proof	Name of test	Type of test	Expected output	Instructions	Actual output
56	"tests 56-61"	Level 1 button (LMC)	Valid	Open Level 1 scene	Press "Level 1" button with left click	Expected output
57	"tests 56-61"	Level 2 button (LMC)	Valid	Open Level 2 scene	Press "Level 2" button with left click	Expected output
58	"tests 56-61"	Level 3 button (LMC)	Valid	Open Level 3 scene	Press "Level 3" button with left click	Expected output
59	"tests 56-61"	Level 1 button (RMC)	Invalid	Do nothing	Press "Level 1" button with right click	Expected output
60	"tests 56-61"	Level 2 button (RMC)	Invalid	Do nothing	Press "Level 2" button with right click	Expected output
61	"tests 56-61"	Level 3 button (RMC)	Invalid	Do nothing	Press "Level 3" button with right click	Expected output

Test number	Video proof	Name of test	Type of test	Expected output	Instructions	Actual output
62	"tests 62-69"	Launch game	Valid	Level 1 music plays	Launch game and check music playback	Expected output

63	"tests 62-69"	Select "Options"	Valid	Level 1 music plays	Press "Options" button (LMC) in title scene and check music playback	Expected output
64	"tests 62-69"	Select "Controls"	Valid	Level 1 music plays	Press "Controls" button (LMC) in settings scene and check music playback	Expected output
65	"tests 62-69"	Select "Audio"	Valid	Level 1 music plays	Press "Audio" button (LMC) in title scene and check music playback	Expected output
66	"tests 62-69"	Select "Play"	Valid	Level 1 music plays	Press "Play" button (LMC) in the title scene and check music playback in level selector scene	Expected output
67	"tests 62-69"	Select "Level 1"	Valid	Level 1 music plays	Press "Level 1" button (LMC) in level selector scene and check music playback	Expected output
68	"tests 62-69"	Select "Level 2"	Valid	Level 2 music plays Level 1 music stops	Press "Level 2" button (LMC) in level selector scene and check music	Expected output

					playback in Level 2	
69	"tests 62-69"	Select "Level 3"	Valid	Level 3 music plays Level 1 music stops	Press "Level 3" button (LMC) in level selector scene and check music playback in Level 3	Expected output

Test number	Video proof	Name of test	Type of test	Expected output	Instructions	Actual output
70	"tests 70-73"	Save Level 1 progress (while in Level 1)	Valid	No visual output Write player position to text file	Press "save progress" button in paused menu in Level 1	Expected output
71	"tests 70-73"	Load Level 1 progress from save (while in Level 1)	Valid	Teleport player to saved position	Press "load progress" button in paused menu in Level 1	Expected output
72	"tests 70-73"	Load Level 1 progress from save (while in Level 2)	Invalid	Do nothing (because Level 1 coordinates are not compatible in Level 2)	Press "load progress" button in paused menu in Level 2	Player was teleported to coordinate location in Level 2
73	"tests 70-73"	Load Level 1 progress from save (while in Level 3)	Invalid	Do nothing (because Level 1 coordinates are not compatible in Level 3)	Press "load progress" button in paused menu in Level 3	Player was teleported to coordinate location in Level 3. Caused player

						death
74	"tests 74-77"	Save Level 2 progress (while in Level 2)	Valid	No visual output Write player position to text file	Press "save progress" button in paused menu in Level 2	Expected output
75	"tests 74-77"	Load Level 2 progress (while in Level 2)	Valid	Teleport player to saved position	Press "load progress" button in paused menu in Level 2	Expected output
76	"tests 74-77"	Load Level 2 progress from save (while in Level 1)	Invalid	Do nothing (because Level 2 coordinates are not compatible in Level 1)	Press "load progress" button in paused menu in Level 1	Player was teleported to coordinate location in Level 1. Caused player to be glitched and stuck in floor
77	"tests 74-77"	Load Level 2 progress from save (while in Level 3)	Invalid	Do nothing (because Level 2 coordinates are not compatible in Level 3)	Press "load progress" button in paused menu in Level 3	Player was teleported to coordinate location in Level 3.
78	"tests 78-81"	Save Level 3 progress (while in Level 3)	Valid	No visual output Write player position to text file	Press "save progress" button in paused menu in Level 3	Expected output
79	"tests 78-81"	Load Level 3 progress (while in Level 3)	Valid	Teleport player to saved position	Press "load progress" button in paused menu in Level 3	Expected output

80	"tests 78-81"	Load Level 3 progress (while in Level 1)	Invalid	Do nothing (because Level 3 coordinates are not compatible in Level 1)	Press "load progress" button in paused menu in Level 1	Player was teleported to coordinate location in Level 1. Caused player to be glitched and stuck in floor
81	"tests 78-81"	Load Level 3 progress (while in Level 2)	Invalid	Do nothing (because Level 3 coordinates are not compatible in Level 2)	Press "load progress" button in paused menu in Level 2	Player was teleported to coordinate location in Level 2. Caused player death
82	"tests 82 & 84"	Load progress without initial saves	Invalid	Do nothing (because there is no saved data)	Press "load progress" button in paused menu in appropriate level	Game crashes
83	"test 83"	Load progress from save after relaunching game	Valid	Teleport player to saved position	Press "load progress" button in paused menu in appropriate level	Expected output
84	"tests 82 & 84"	Load progress without initial saves after relaunching game	Invalid	Do nothing (because there is no saved data)	Press "load progress" button in paused menu in appropriate level	Game crashes
85	Not	Save coin	Valid	No visual	Not	N/A

	implemented	count		output Write coin count to text file	implemented	
86	Not implemented	Load coin count from save	Valid	Update coin count to saved value	Not implemented	N/A
87	Not implemented	Load coin count without initial saves	Invalid	Do nothing (because there is no saved data)	Not implemented	N/A
88	Not implemented	Load coin count without initial saves after relaunching game	Invalid	Do nothing (because there is no saved data)	Not implemented	N/A
89	Not implemented	Load coin count from save after relaunching game	Valid	Update coin count to saved value	Not implemented	N/A

Has the Success Criteria been met?

I will now evaluate all that has been accomplished in the development stage of this project, and compare it to the success criteria I created in the research stage with the feedback and suggestions of my stakeholders, and see what has been accomplished and what has not.

Success Criteria	Has it been met? If not, why?	Stakeholder (Ismail) feedback on success criterion
1. Include a start screen on startup	Yes	It's a good opportunity to show the title of the game, the main options and you're able to define the entire theme of the game and what it's based around.
2. Use a simple loading screen.	No, because it was not needed	

3. Include a minimum of 3 levels	Yes	3 levels is a good amount, especially when you factor in the theme you're going for, which is each level gets higher and higher up; it fits a lot better.
4. Create a 3rd person perspective from the side	Yes	You said to me that you had to choose
5. Make the obstacles/platforms challenging to the player but still make sure it is enjoyable to play	Yes	You interviewed me before, and I said I like how they gradually get harder from level 1 to 3, and especially how they take advantage of stuff like dashing.
6. Have only 1 life per checkpoint	Yes	This is good and simple, especially because there aren't any enemies to deduct health; this makes more sense
7. Include a variety of enemy types	No, because enemies weren't implemented	
8. Have a different biome for each level	Yes	This looks nice
9. Have Esc bring up the in-game menu	Yes	This adds usability to the game because it's an easy way to include the option to tweak settings from the convenience of the game scene. It's good
10. It will be a single-player game	Yes	With the time-frame you had on top of the idea of the game, it's understandable that you couldn't make a multiplayer game
11. Include a random number of collectibles per level	Yes	This is good because the levels won't have barren spots where there's nothing to do but get through it; each part of each level has coins to collect so nothing is boring.
12. Include a life count system and game over	No, because a life count system wasn't implemented	
13. New lives can be earned for XP threshold in each level	No, because a life count system wasn't implemented	
14. Include a score system	Yes	To me, it's a secondary purpose to the game; collect

		all the coins. It's a popular feature of a lot of games and there a reason why it's so popular. But in the future, it would've been nice to give them more of a purpose than just a big number.
15. At the end of the level, the total XP will be calculated and presented	Yes	The end of the level is important, so this is important. Like in one of the games in your Research (Rayman Legends) there's like a celebration of the collectibles the player got in the level where the stats are put on the screen. So, the fact that you put this in is good.
16. Include a game saving system.	Yes	I don't want to have to restart the game each time I play it, because I can't really finish the entire game in like a 15-minute gaming session, so the fact that I can save where I am, and load it back up when I come back is a great feature. But I'm disappointed that other stuff wasn't saved like the player's score.
17. Include a range of background music which is unique for each level and situation	Yes	Without background music, there's nothing to keep me focused on the game, but the game itself, and that can get boring really fast. Also, I think music can accentuate the theme or concept that you're going for
18. Include necessary sound effects	No, because of time, the SFX system wasn't implemented	
19. Create an informative but not distracting heads up display (HUD)	Yes	You did this very well. I like how it's in the corner of the screen, small but still easy to read and clear in presenting the information. Speaking of which, the information displayed is pretty good, but I would've liked a timer like you said you were going to do.
20. Make menu navigational by mouse	Yes	I don't think menu's should be navigational by solely keyboard, so this was a good addition

21. Include a cool animation for when the player collects the collectibles	Yes	Like it stated in your success criteria, it gives a sense of reward to the player. This is good because it incentivizes them to get them more. On top of that, people want to see a high number when they look at their score.
22. Include a snappy and satisfying dash with a good animation	Yes	Before you added the animation for the dash, it looked bad and didn't make sense for the movement to be that fast but no visual aspect to be there, so this was as essential as any other animation. I especially like the white lines you put behind the player that gives the impression that he's going really fast.
23. Player will not fall into the void while on a platform	Yes	
24. Player will be able to stand and walk normally on a platform	Yes	

Unimplemented features

Looking at my feature list, there are a few features I was unable to put into my game. On top of that, points of my success criteria were also unable to be implemented. I will now provide explanations of how they could be implemented in the future.

Feature not implemented	Why it wasn't implemented	How it can be in the future
Feature List 5 & Success Criteria 18 = There will be sound effects that are relevant to each feature that they are used for (enemies, collectibles etc.)	Time constraints	For each different feature, there will be an <code>AudioStreamPlayer2D</code> node that can play the corresponding sound effect. Assuming I make a scene for that individual object, I can then use the script for the scene to play the sound effect when necessary.

Feature List 7 = Secret sections of a level	Time constraints and lack of focus on specific level design	I would create duplicate tiles on the TileMap that look identical to floor tiles, but I would remove the physics layer that makes the tiles real/physical. This would cause the player to be able to go through said tiles. I can then use this to construct things like fake walls with coins inside.
Feature List 8 & Success Criteria 7 = Enemies	Time constraints	I would create an enemy scene, with an Area2D node and a CollisionShape2D. If the body of the enemy, which will stretch largely around the enemy, is entered by the player, then they will die and respawn back at the checkpoint. If I wanted to make them moving, then I may have used an Animation Player to move them around in a loop, while retaining the function of killing the player if they get too close
Success Criteria 12 & 13 = Life count system, game over screen and XP being traded for more lives	Time constraints	I would have a global variable called lives set to 5. Each time the player dies and respawns, lives would be decremented by 1. In the scenario that it reaches 0, then all checkpoints would be erased, and the entire game would be reset.

Usability Features

I will now go into more detail about the features that I have included, along with feedback from a stakeholder.

Title screen



Clear, centered, simple menu buttons for easy navigation

Detailed textures that resemble the first level design for a seamless transition between menu and game

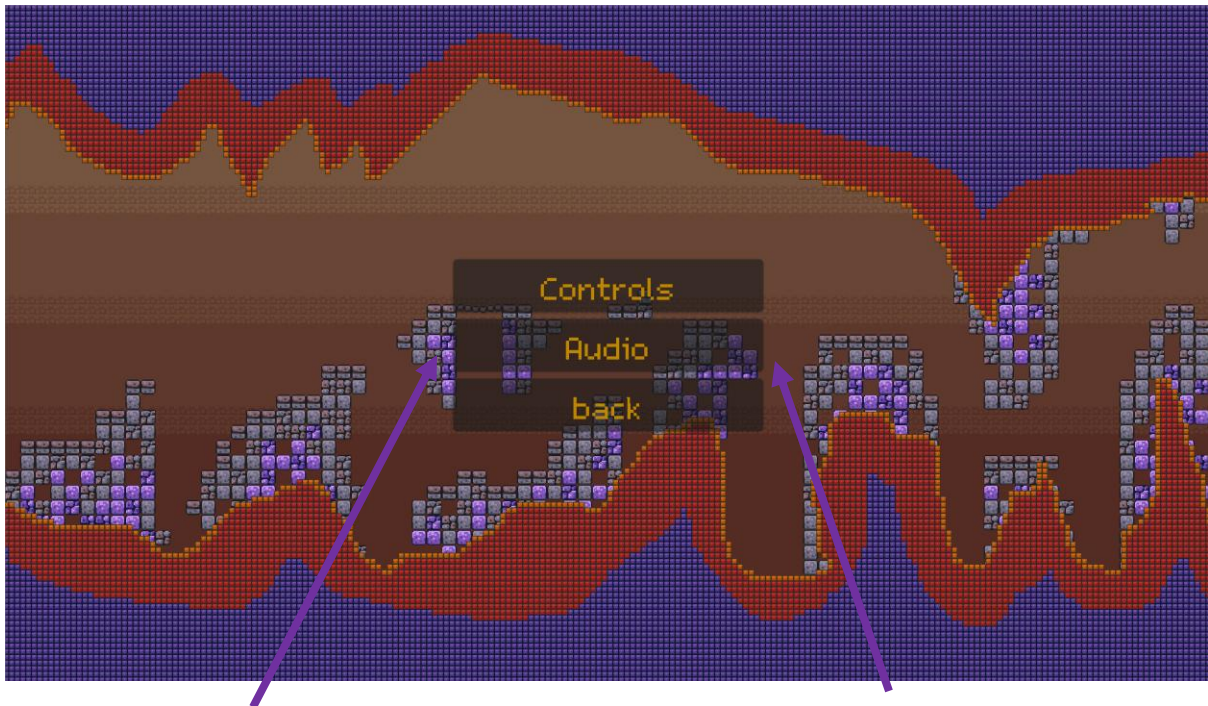
Clear title for the game that stands out to the player

The menu's are navigational only by mouse, to ensure familiarity with the game for all users. There are 3 buttons: Play, Options & Quit, to ensure all desirable settings are available to the user.

Stakeholder feedback:

- It would be nice if you added support for keyboards as well, just so there are no gaps in knowledge and everyone can feel comfortable traversing the menus.
- But other than that, everything is very clear and easy to find, and the background looks nice.

Options screen



The 2 most common option features are controls and audio settings, so they are included here.

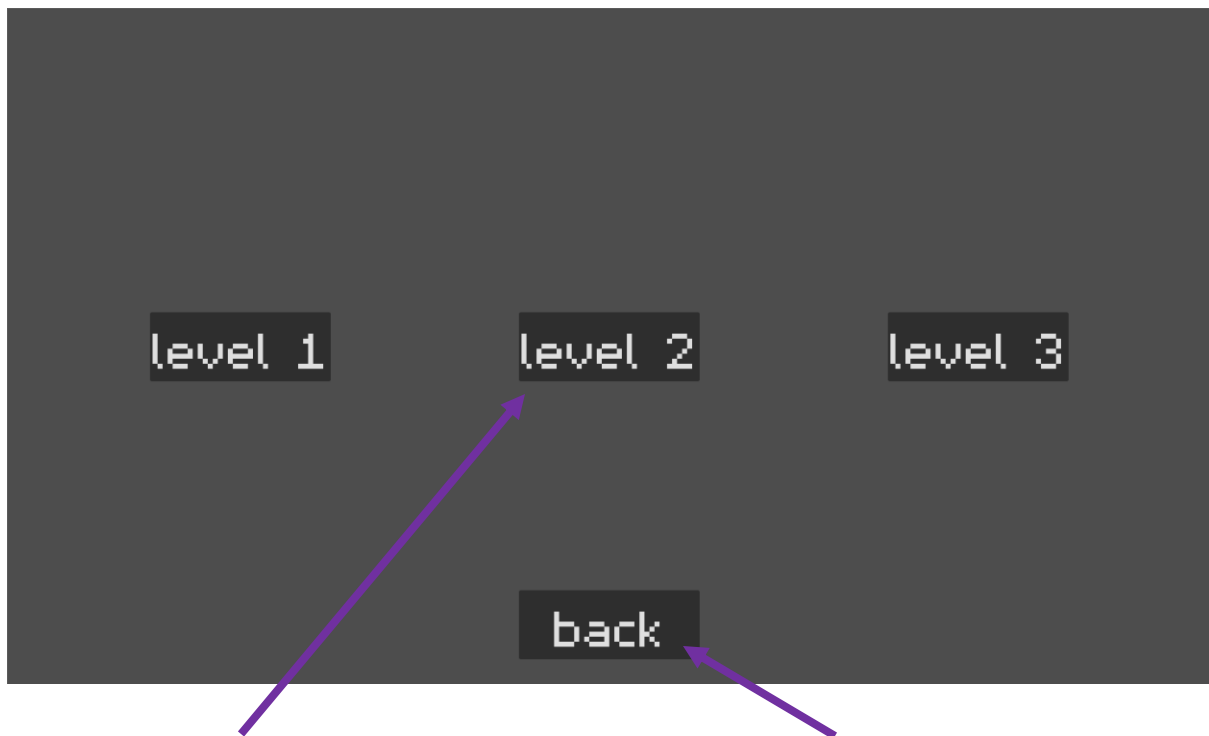
Same button layout as well as same background design as title screen to ensure a seamless and easy to understand transition between screens

This screen acts as a hub or a point of access to the main settings: Controls and Audio. The controls screen displays the controls of the game in an easy to understand and visually appealing format, while the audio screen provides the options to mute and unmute different sounds in the game.

Stakeholder feedback:

- I like how it's pretty much identical to the other screen, because it makes it easy to understand of how the menus work.
- I would've liked it a lot more if the controls were able to be changed for the players liking more.
- Not including sound effects is also a big loss, especially in a game with collectibles.

Level Selector screen



Clear separated and easy to read buttons that take the player to the correct level

Player still has the option to go back to the title screen if they need to change some settings to quit the game

The level selector screen, like the options screen, acts like a hub or a point of access for the rest of the levels. The player can easily access whatever level they would like from here.

Stakeholder feedback:

- You should've designed it to look like the other menu screens instead of leaving it unfinished.
- You should've included functionality that locked the other levels, then unlocked them when you finished the previous one.

Level 1



Controls listed here
like a mini tutorial

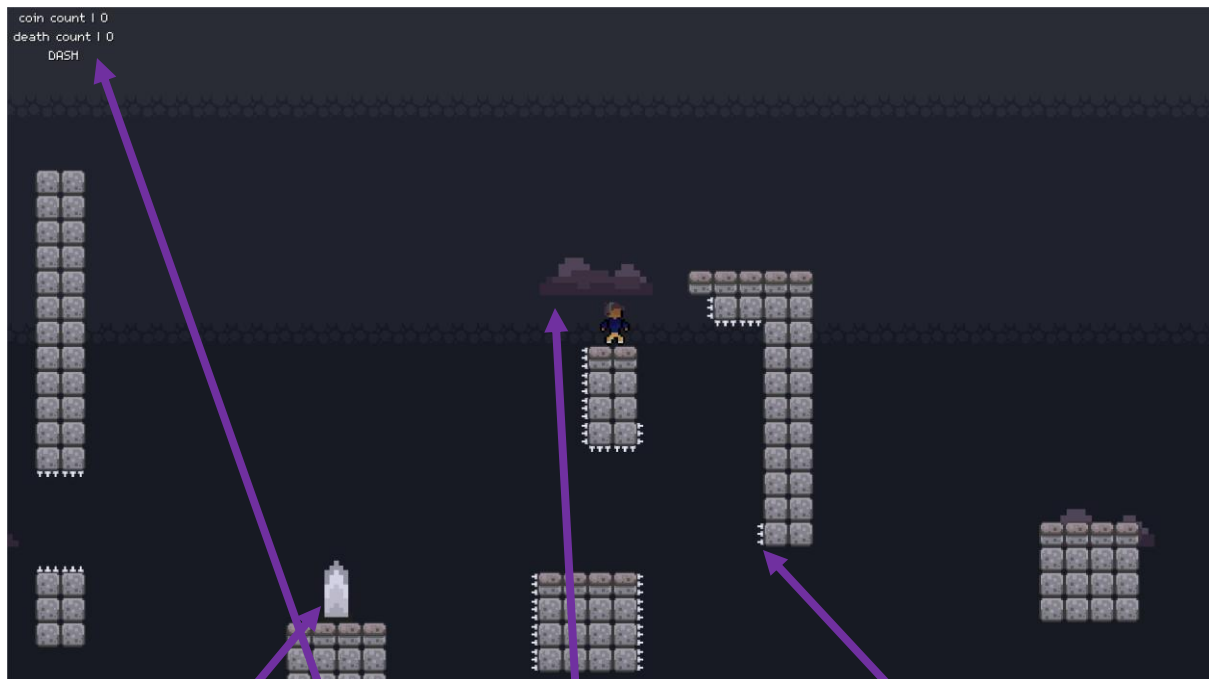
Very easy first
obstacles to help the
player get used to the
controls

This is the start of the first level. Because it's the start of the entire game, I thought it necessary to include a short tutorial for new players. I made it as big as it needed to be for new players to learn, without it becoming a burden to get through for experienced players.

Stakeholder feedback:

- I really like how you've done the tutorial area; making it big enough for new players to experiment and try out the controls, and making it small enough for real players to quickly get through.
- The control tutorial is also good because instead of putting it in one big poster-like object, you space them out to where they are n

Level 3



Checkpoints provide a way to keep some progress after dying

Clouds floating in the sky create an atmosphere in space

New form of obstacles: Spikes. If touched, they will kill the player

HUD shows information to the player (coin count, death count and dash state)

Level 3 is the accumulation of all the skills and abilities the player has learnt, along with new ones. This also comes with new and harder obstacles.

Stakeholder feedback:

- I like the clouds. They look nice and it's a good reminder of the theme of the game you're going for of space.
- The HUD is extremely helpful. It not protruding into the screen at all because its tucked away in the corner, and it's still easy to look at and read. You should've made it display the XP though too.
- Spikes are a good addition to the obstacles, because they are common in platformer games and they're not a total pain to avoid and get past them, so they're not too difficult.

- But if the obstacles are too difficult, then checkpoints can help the player finish the level quicker and have more fun while doing it, since they don't have to restart the level each time they die.

Maintenance & Limitations

Current and future maintenance

I have included several features that make my program more maintainable for the future. One includes consistent and clear commenting of my code. I have been careful to distribute these in the majority of my scripts so that anyone can understand the purpose of each command and each line.

I have also used meaningful variable names so they can easily be tracked and understood in the future.

My project isn't fully complete, but if it were, I would include a reporting system where players could report any issues or bugs that were present. They could also message in feedback for future updates of the game. Although, depending on the number of messages coming in, I may need more than one person to handle user requests. Additionally, depending on how successful the game will be in a scenario where it's complete, further updates could be made with the feedback from the reporting system as well as the programmers' ideas.

These updates will be available to be closed or open source, so developers in the community can tweak the game however they like and can get support from other developers in the community, and casual players can use the game normally, without the risk of new bugs appearing.

In the future, I may also try to find new methods that avoid using global variables, as they are not, in only some situations, considered good programming practice. I experienced this myself as the values of global variables found themselves clashing with different purposes several times in my program.

Limitations

I will list some limitations to my project and how they can be approached

Limitation	Justification	Method of implementation
Support for controllers	To support players on all platforms; not just desktop OS's	Add custom commands for joysticks and get values from them being used

Support for mobile devices	To support players on all platforms; not just desktop OS's	Add custom on screen buttons that the player can press
Make power-ups	To give the player more special abilities which can both make the game easier and harder	Add collectibles that trigger a variable called power-up to be true. This then gives the player special abilities
Include a more detailed background	The current background looks rather plain and boring; some stars would go nice in the sky of level 3, for example.	Add more detailed tiles to the TileMap

Project Appendix

audio_screen.gd

```

1  extends Control
2
3  ▼ func _on_mute_button_pressed() -> void:
4      >| #switches the value of muted to the opposite (acts as a toggle)
5      >| Global.muted = !Global.muted
6      >| #creates variable of music bus to manipulate it
7      >| var music_bus_index = AudioServer.get_bus_index("Music")
8      >| #sets the music bus to the value of muted
9      >| AudioServer.set_bus_mute(music_bus_index, Global.muted)
10     >|
11  ▼ func _on_sfx_button_pressed() -> void:
12     >| var SFX_bus_index = AudioServer.get_bus_index("SFX")
13     >| AudioServer.set_bus_mute(SFX_bus_index, true)
14     >| >|
15  ▼ func _on_back_button_pressed() -> void:
16     >| get_tree().change_scene_to_file("res://scenes/setting_screen.tscn")

```

checkpoints.gd

```
1  extends Area2D
2  class_name checkpoint
3
4  var activated = false
5
6  @onready var checkpoint_sprite: AnimatedSprite2D = $AnimatedSprite2D
7
8  ▾ func _on_body_entered(body: Node2D) -> void:
9      » #can only be activated if not already
10     ▾ » if !activated:
11         » » activate()
12
13     ▾ func activate():
14         » #current checkpoint is this checkpoint
15         » Global.active_checkpoint = self
16         » #checkpoint is activated
17         » activated = true
18         » #plays activation animation
19         » checkpoint_sprite.play("activated")
20         » Global.save_game()
```

coin.gd

```
1 extends Area2D
2
3 #initiates the nodes of the coins animatedsprite and the label as variables
4 #so they can be manipulated and used
5 @onready var coin_sprite = $AnimatedSprite2D
6 @onready var score_label: Label = $"../labels/Score"
7
8 func _on_body_entered(body: Node2D) -> void:
9     #plays the collected animation when the coin is ran into
10    coin_sprite.play("collected")
11
12 func _on_animated_sprite_2d_animation_finished() -> void:
13    #deletes the coin
14    queue_free()
15    #increments score and XP
16    Global.score += 1
17    Global.XP += 25
18    #updates the score label (in level 1) in real time
19    if Global.level1:
20        score_label.text = str(Global.score) + " coins collected and " + str(Global.XP) + " XP gained!"
```

control_screen.gd

```
1 extends Control
2
3 func _on_back_button_pressed() -> void:
4     >| get_tree().change_scene_to_file("res://scenes/setting_screen.tscn")
```

global.gd

```
1 extends Node
2
3 var playing: bool = false
4 var paused: bool = false
5 var level2music: bool = false
6 var level3music: bool = false
7 var level1: bool = false
8 var level2: bool = false
9 var level3: bool = false
10 var score = 0
11 var XP = 0
12 var onWall: bool = false
13 var muted: bool = false
14 var can_dash: bool = false
15 var death_count = 0
16
17 #-----
```

```
17 #-----
18
19 func reset_XP():
20     >| score = 0
21     >| XP = 0
22
23 #-----
```

```

23  #-----
24
25  #creates references of the instances of the current checkpoint and the player
26  var active_checkpoint : checkpoint
27  var player : Player
28
29  func respawn_player():
30      death_count += 1
31      #if any checkpoint has been activated
32      if active_checkpoint != null:
33          #move the player to the checkpoint
34          player.position = active_checkpoint.position
35          #if no checkpoint has been activated
36      else:
37          #respawn player at the start of the level
38          get_tree().reload_current_scene()
39
40  #-----

```

```

40  #-----
41
42  const save_folder = "user://saves"
43
44  func save_game():
45      #puts player position into an array
46      var save_data = [player.position.x, player.position.y]
47      #opens file to write to it
48      var file = FileAccess.open(save_folder, FileAccess.WRITE)
49      #stores the string version of the save data
50      file.store_string(JSON.stringify(save_data, "\t"))
51      #closes the file
52      file.close()
53
54  func load_game():
55      #opens the file to read from it
56      var file = FileAccess.open(save_folder, FileAccess.READ)
57      #retrieves data from file as string
58      var content = file.get_as_text()
59      #closes the file
60      file.close()
61      #converts string data back into an array
62      var save_data = JSON.parse_string(content)
63      if save_data:
64          #converts array data into Vector2 (i.e. data that can be used for positioning)
65          player.position = Vector2(save_data[0], save_data[1])

```

hud.gd

```
1 extends Control
2
3 #initiates label nodes for manipulation
4 @onready var coin_count: Label = $VBoxContainer/coinCount
5 @onready var death_count: Label = $VBoxContainer/deathCount
6 @onready var dash_point: Label = $VBoxContainer/dashPoint
7
8 #function that runs every frame
9 func _physics_process(delta: float) -> void:
10     > #only displays dash indicator in level 3
11     > if Global.level3:
12         > dash_point.show()
13     > else:
14         > dash_point.hide()
15     > #creates dash indicator
16     > if Global.can_dash == true:
17         > dash_point.text = "DASH"
18     > else:
19         > dash_point.text = "NO DASH"
20     > #updates with death and coin count
21     > death_count.text = "death count | " + str(Global.death_count)
22     > coin_count.text = "coin count | " + str(Global.score)
```

killzone.gd

```
1 extends Area2D
2
3 func _on_body_entered(_body: Node2D) -> void:
4     > #resets XP to 0
5     > Global.reset_XP()
6     > #respawns player
7     > Global.respawn_player()
```

level_1.gd

```
1  extends Node2D
2
3  #level 1 platform that goes to level 2
4
5  @onready var animation_player: AnimationPlayer = $"teleporter to level2/transporter platform/AnimationPlayer"
6  @onready var collision_shape_2d: CollisionShape2D = $CollisionShape2D
7
8  func _on_transporter_activator_body_entered(body: Node2D) -> void:
9      > #starts the moving animation
10     > animation_player.play("new_animation")
11
12  func _on_teleporter_body_entered(body: Node2D) -> void:
13     > #when the player goes out of screen, cuts to level 2
14     > Global.level1 = false
15     > Global.level2 = true
16     > Global.level3 = false
17     > Global.level2music = true
18     > MusicLevel1.refresh_music()
19     > MusicLevel2.refresh_music()
20     > get_tree().change_scene_to_file("res://scenes/level_2.tscn")
```

level_2.gd

```
1  extends Node2D
2
3  #level 2 platform that goes to level 3
4
5  @onready var platform_animator: AnimationPlayer = $"teleporter from l2 to l3/PlatformMountain3/AnimationPlayer"
6  @onready var label_2: Label = $Label2
7
8  func _ready() -> void:
9     > #hides the "wait here" label until the player reaches the end
10    > label_2.hide()
11
12  func _on_activator_body_entered(body: Node2D) -> void:
13     > #begins the platform movement
14     > platform_animator.play("moving")
15     > label_2.show()
16
17  func _on_teleporter_body_entered(body: Node2D) -> void:
18     > #cuts to level 3
19     > get_tree().change_scene_to_file("res://scenes/level_3.tscn")
```

level_selector.gd

```
1  extends Control
2
3  ▼ func _on_level_1_pressed() -> void:
4      » #sets all variables appropriately for pausemenu and music playback use
5      » Global.level1 = true
6      » Global.level2 = false
7      » Global.level3 = false
8      » get_tree().change_scene_to_file("res://scenes/game.tscn")
9
10 ▼ func _on_level_2_pressed() -> void:
11     » Global.level1 = false
12     » Global.level2 = true
13     » Global.level3 = false
14     » Global.level2music = true
15     » Global.level3music = false
16     » MusicLevel1.refresh_music()
17     » MusicLevel2.refresh_music()
18     » get_tree().change_scene_to_file("res://scenes/level_2.tscn")
19
20 ▼ func _on_level_3_pressed() -> void:
21     » Global.level1 = false
22     » Global.level2 = false
23     » Global.level3 = true
24     » Global.level2music = false
25     » Global.level3music = true
26     » MusicLevel1.refresh_music()
27     » MusicLevel3.refresh_music()
28     » get_tree().change_scene_to_file("res://scenes/level_3.tscn")
29
30 ▼ func _on_back_pressed() -> void:
31     » get_tree().change_scene_to_file("res://scenes/title_screen.tscn")
```

music_level_1.gd

```
1  extends AudioStreamPlayer2D
2
3  @onready var music_l1: AudioStreamPlayer2D = self
4
5  #called when the game launches immediately
6  func _ready() -> void:
7      update_music_playback()
8
9  func update_music_playback():
10     #level 1 music shouldn't play in level 2 or 3
11     if Global.level2music or Global.level3music:
12         #pauses music
13         music_l1.stream_paused = true
14     else:
15         music_l1.stream_paused = false
16
17  func refresh_music():
18     update_music_playback()
```

music_level_2.gd

```
1  extends AudioStreamPlayer2D
2
3  @onready var music_l2: AudioStreamPlayer2D = self
4
5  #called when the game launches immediately
6  func _ready() -> void:
7      » update_music_playback()
8
9  func update_music_playback():
10     » #this music should only play in level 2
11     » if Global.level2music:
12         » » #begins playback of music
13         » » music_l2.play()
14     » else:
15         » » music_l2.stop()
16
17  func refresh_music():
18     » update_music_playback()
```

music_level_3.gd

```
1  extends AudioStreamPlayer2D
2
3  @onready var music_l3: AudioStreamPlayer2D = $"."
4
5  #called when the game launches immediately
6  func _ready() -> void:
7      update_music_playback()
8
9  func update_music_playback():
10     #this music should only play in level 2
11     if Global.level3music:
12         #begins playback of music
13         music_l3.play()
14     else:
15         music_l3.stop()
16
17  func refresh_music():
18     update_music_playback()
```

pause_menu.gd

```
1  extends Control
2
3  #called when the game launches immediately
4  func _ready():
5      > #hides the pausemenu so it isnt constantly on the screen
6      > hide()
7
8  func pause():
9      > #sets paused state to true
10     > Global.paused = true
11     > #pauses the game by setting the timescale to 0
12     > Engine.time_scale = 0
13     > #get_tree().paused = true
14     > #shows the pause menu
15     > show()
16
17  func resume():
18     > #sets paused state to false
19     > Global.paused = false
20     > #resets time scale back to default value
21     > Engine.time_scale = 1
22     > #get_tree().paused = false
23     > #hides the pause menu
24     > hide()
```

```

26  ▾ func EscPressed():
27  ▾  ▸ if Input.is_action_just_pressed("esc") and Global.paused == false:
28  ▾  ▸  ▸ #if the game isnt paused and player presses esc
29  ▾  ▸  ▸ #calls pause function
30  ▾  ▸  ▸ pause()
31  ▾  ▸ elif Input.is_action_just_pressed("esc") and Global.paused == true:
32  ▾  ▸  ▸ #can unpause the game if player presses esc again
33  ▾  ▸  ▸ #calls resume function
34  ▾  ▸  ▸ resume()
35  ▾  ▸  ▸
36  #called every frame of the game
37  ▾ func _process(_delta):
38  ▾  ▸ #is always checking for esc being pressed
39  ▾  ▸ EscPressed()

```

```

41  #button functions
42
43  ▾ func _on_resume_pressed() -> void:
44  ▾  ▸ resume()
45
46  ▾ func _on_restart_pressed() -> void:
47  ▾  ▸ #unpauses game
48  ▾  ▸ resume()
49  ▾  ▸ #refreshes music check
50  ▾  ▸ MusicLevel1.refresh_music()
51  ▾  ▸ MusicLevel2.refresh_music()
52  ▾  ▸ MusicLevel3.refresh_music()
53  ▾  ▸ #resets game scene
54  ▾  ▸ get_tree().reload_current_scene()
55
56  ▾ func _on_settings_pressed() -> void:
57  ▾  ▸ Global.level2music = false
58  ▾  ▸ Global.level3music = false
59  ▾  ▸ MusicLevel1.refresh_music()
60  ▾  ▸ MusicLevel2.refresh_music()
61  ▾  ▸ MusicLevel3.refresh_music()
62  ▾  ▸ #player is currently playing
63  ▾  ▸ Global.playing = true
64  ▾  ▸ #resumes timescale for music etc.
65  ▾  ▸ resume()
66  ▾  ▸ get_tree().change_scene_to_file("res://scenes/setting_screen.tscn")

```

```
68  ▾ func _on_quit_pressed() -> void:
69      > Global.level2 = false
70      > Global.level3 = false
71      > Global.level2music = false
72      > Global.level3music = false
73      > MusicLevel1.refresh_music()
74      > MusicLevel2.refresh_music()
75      > MusicLevel3.refresh_music()
76      > #player is no longer playing
77      > Global.playing = false
78      > resume()
79      > get_tree().change_scene_to_file("res://scenes/title_screen.tscn")
```

```
81  ▾ func _on_save_pressed() -> void:
82      > Global.save_game()
83
84  ▾ func _on_load_pressed() -> void:
85      > Global.load_game()
```

player.gd

```
1  extends CharacterBody2D
2  class_name Player
3
4  #movement constants
5  const SPEED = 155
6  const JUMP_VELOCITY = -245
7  const WALLJUMP_VELOCITY = -100
8  const WALLJUMP_PUSHBACK = 775
9  const DASH_SPEED = 500
10
11 #dash variables
12 var dash_direction = Vector2.ZERO
13 var dashing = false
14
15 #gravity value (created by godot libraries)
16 var gravity = ProjectSettings.get_setting("physics/2d/default_gravity")
17
18 #initiates all nodes to be used
19 @onready var animated_sprite = $AnimatedSprite2D
20 @onready var player = $"."
21 @onready var ray_cast_bottom_right: RayCast2D = $RayCastBottomRight
22 @onready var ray_cast_bottom_left: RayCast2D = $RayCastBottomLeft
23 @onready var ray_cast_top_right: RayCast2D = $RayCastTopRight
24 @onready var ray_cast_top_left: RayCast2D = $RayCastTopLeft
25 @onready var dash_timer: Timer = $dashTimer
26 @onready var dash_animation_timer: Timer = $dashAnimationTimer
27
28 #-----
```

```

28  #-----
29
30  ▾ func _ready():
31      » Global.player = self
32
33      #calls each movement function so that they are always active and being called
34  ▾ func _physics_process(delta):
35      » get_direction()
36      » apply_gravity(delta)
37      » handle_movement(get_direction())
38      » handle_jump()
39      » handle_wallClimbing()
40      » handle_crouchWalk(get_direction())
41      » handle_wallJump()
42      » handle_animations(get_direction())
43      » handle_dash()
44      » check_dash()
45      » move_and_slide()
46
47  #-----

```

```

47  #-----
48
49  #function retrieving the players direction
50  ▾ func get_direction():
51      » #retrieves integer value (-1, 0, 1) based off left/right being pressed
52      » var direction = Input.get_axis("move_left", "move_right")
53      » return direction
54
55      #creates the gravity function
56  ▾ func apply_gravity(delta):
57      » #player will be immune to gravity while dashing
58  ▾ » if not dashing:
59      » » #checks if player is in mid-air only
60  ▾ » » if not is_on_floor() and Global.onWall == false:
61      » » » #changes vertical velocity to go down at speed of time (delta) X gravity
62      » » » velocity.y += gravity * delta
63
64      #creates player jumping function
65  ▾ func handle_jump():
66      » #can only jump if on ground
67  ▾ » if is_on_floor():
68      » » #checks for jump keys being pressed
69  ▾ » » if Input.is_action_pressed("spacebar") or Input.is_action_pressed("move_up"):
70      » » » velocity.y = JUMP_VELOCITY

```



```

72 #creates player moving function
73 func handle_movement(direction):
74     #ensures that movement is disabled during the dash
75     if dashing == false:
76         #if direction has any value apart from 0 (i.e. moving anywhere at all)
77         if direction:
78             #moves in the direction being pressed at the speed of SPEED
79             velocity.x = direction * SPEED
80         else:
81             #slows down player until horizontal speed = 0, at the speed of SPEED
82             velocity.x = move_toward(velocity.x, 0, SPEED)
83
84         #initiates dash if Enter is pressed and the player is able to dash
85         if Input.is_action_just_pressed("dash") and Global.can_dash:
86             #starts dash function
87             start_dash()

```

```

89 #creates player climbing function
90 func handle_wallClimbing():
91     #main requirement is holding shift on walls
92     if Input.is_action_pressed("shift"):
93         #checks if either raycast is on a wall
94         if ray_cast_bottom_left.is_colliding() or ray_cast_top_left.is_colliding():
95             #player is on a wall
96             Global.onWall = true
97             #other side
98             elif ray_cast_bottom_right.is_colliding() or ray_cast_top_right.is_colliding():
99                 Global.onWall = true
100         else:
101             Global.onWall = false
102         else:
103             Global.onWall = false
104         #if player is on a wall and pressing shift
105         if Global.onWall == true:
106             #if player then presses W
107             if Input.is_action_pressed("move_up"):
108                 #moves up the wall
109                 velocity.y = WALLJUMP_VELOCITY
110             #if player then presses S
111             elif Input.is_action_pressed("move_down"):
112                 #moves down the wall
113                 velocity.y = -WALLJUMP_VELOCITY
114             else:
115                 #otherwise, clings/sticks to the wall
116                 velocity.y = 0

```

```

118     #creates jumping off the wall
119     ▾ func handle_wallJump():
120         >|     #can only wall jump if on a wall
121         ▾ >|     if Global.onWall == true:
122             ▾ >|         #checks for wall jump key
123             >|         #can't hold space (hence, "just")
124             ▾ >|         if Input.is_action_just_pressed("spacebar"):
125                 >|         >|         #figures out player's facing direction (if wall is on left/right side of player)
126                 ▾ >|         >|         if ray_cast_bottom_left.is_colliding() or ray_cast_top_left.is_colliding():
127                     >|         >|         >|         #shoots them up
128                     >|         >|         >|         velocity.y = WALLJUMP_VELOCITY * 2.5
129                     >|         >|         |>|         #sends them in direction away from the wall
130                     >|         >|         >|         velocity.x = WALLJUMP_PUSHBACK
131                 ▾ >|         >|         else:
132                     >|         >|         >|         velocity.y = WALLJUMP_VELOCITY * 2.5
133                     >|         >|         >|         #sends them in direction away from the wall (hence, "-")
134                     >|         >|         >|         velocity.x = -WALLJUMP_PUSHBACK
135
136     #creates crouching
137     ▾ func handle_crouchWalk(direction):
138         >|     #checks is player is pressing S
139         ▾ >|     if Input.is_action_pressed("move_down"):
140             >|         #can only crouch if on ground
141             ▾ >|         if is_on_floor():
142                 >|         >|         #makes them slow down
143                 >|         >|         velocity.x = (direction * SPEED) / 2
144                 >|         >|         velocity.y = velocity.y / 1.5

```

```

146 #handles playing of all animations/states of player
147 func handle_animations(direction):
148     #figures out player direction
149     if direction < 0 or ray_cast_bottom_left.is_colliding() or ray_cast_top_left.is_colliding():
150         #flips the sprite into the direction it's moving/facing
151         animated_sprite.flip_h = true
152     elif direction > 0 or ray_cast_bottom_right.is_colliding() or ray_cast_top_right.is_colliding():
153         animated_sprite.flip_h = false
154
155     #for as long as dash animation timer runs
156     if dash_animation_timer.time_left > 0:
157         #play dash animation
158         animated_sprite.play("dashing")
159
160     #if player's crouching
161     elif Input.is_action_pressed("move_down") and is_on_floor():
162         #if player's only moving horizontally
163         if velocity.y == 0:
164             #if they're moving at all
165             if velocity.x != 0:
166                 #plays crouch running animation
167                 animated_sprite.play("crouch run")
168             else:
169                 #plays crouch idle, because player's not moving at all while crouching
170                 animated_sprite.play("crouch idle")
171         else:
172             #if player's moving upwards while crouching
173             if velocity.y < 0:
174                 #plays crouch jump
175                 animated_sprite.play("crouch jump")

```

```

176     #if player is not crouching
177     else:
178         #if player's only moving horizontally
179         if velocity.y == 0:
180             #checks if player's on the ground
181             if is_on_floor():
182                 #if they're moving at all
183                 if velocity.x != 0:
184                     #plays run
185                     animated_sprite.play("run")
186             else:
187                 #plays idle, because they're not moving at all
188                 animated_sprite.play("idle")
189             #if player's not on ground, then must be on wall
190             elif Global.onWall:
191                 #vertical velocity is 0, and can only move vertically on walls, so is idle on a wall
192                 animated_sprite.play("climb idle")

```

```

193     >| >| #if player's moving vertically
194   ▾ >| >| else:
195     >| >| >| #if player's climbing
196   ▾ >| >| >| if Global.onWall:
197     >| >| >| >| #plays climbing
198     >| >| >| >| animated_sprite.play("climb")
199   ▾ >| >| >| else:
200     >| >| >| >| #if player's moving upwards
201   ▾ >| >| >| >| if velocity.y < 0:
202     >| >| >| >| >| #plays jumping
203     >| >| >| >| >| animated_sprite.play("jump")
204     >| >| >| >| >| #if player's moving downwards
205   ▾ >| >| >| >| else:
206     >| >| >| >| >| #plays falling
207     >| >| >| >| >| animated_sprite.play("falling")

```

```

209 ▾ func start_dash():
210   >| #creates a 8-way dash direction
211   >| dash_direction = Input.get_vector("move_left", "move_right", "move_up", "move_down")
212   >| #player can only dash once so they can no longer dash
213   >| Global.can_dash = false
214   >| #sets dashing to true
215   >| dashing = true
216   >| #starts dash timer
217   >| dash_timer.start()
218   >| dash_animation_timer.start()
219   >|
220 ▾ func handle_dash():
221   >| #if player is dashing
222   ▾ >| if dashing:
223     >| >| #sets velocity to terminal velocity immediately in a dash-like fashion
224     >| >| velocity = dash_direction * DASH_SPEED
225   ▾ >| >| if velocity.y < 0:
226     >| >| >| #reduces vertical dash velocity
227     >| >| >| velocity.y *= 0.65
228
229 ▾ func _on_dash_timer_timeout() -> void:
230   >| #after the timer has run out, the player will no longer be dashing
231   >| dashing = false

```

```
233  ▾ func check_dash():
234      >|    #can only dash in level 3
235  ▾ >|    if Global.level3:
236      >|    >|    #can only dash once; resets when player is on floor/wall
237  ▾ >|    >|    if is_on_floor() or Global.onWall:
238      >|    >|    >|    Global.can_dash = true
239  ▾ >|    >|    else:
240      >|    >|    >|    #play undashable animation
241      >|    >|    >|    pass
242
243  ▾ func _on_dash_animation_timer_timeout() -> void:
244      >|    #switches animation back to normal idle after dash is over
245  ▾ >|    if animated_sprite.animation == "dashing":
246      >|    >|    animated_sprite.play("idle")
```

settingsScreen.gd

```
1  extends Control
2
3  ▾ func _on_back_button_pressed() -> void:
4      >| #checks if player is playing
5      ▾ >| if Global.playing == true:
6          >| >| #then checks what level they are in
7          ▾ >| >| if Global.level1 == true:
8              >| >| >| #then navigates to the correct level
9              >| >| >| MusicLevel1.refresh_music()
10             >| >| >| MusicLevel2.refresh_music()
11             >| >| >| MusicLevel3.refresh_music()
12             >| >| >| get_tree().change_scene_to_file("res://scenes/game.tscn")
13         ▾ >| >| elif Global.level2 == true:
14             >| >| >| Global.level2music = true
15             >| >| >| MusicLevel1.refresh_music()
16             >| >| >| MusicLevel2.refresh_music()
17             >| >| >| MusicLevel3.refresh_music()
18             >| >| >| get_tree().change_scene_to_file("res://scenes/level_2.tscn")
19         ▾ >| >| elif Global.level3 == true:
20             >| >| >| Global.level3music = true
21             >| >| >| MusicLevel1.refresh_music()
22             >| >| >| MusicLevel2.refresh_music()
23             >| >| >| MusicLevel3.refresh_music()
24             >| >| >| get_tree().change_scene_to_file("res://scenes/level_3.tscn")
25         ▾ >| else:
26             >| >| #if they're not playing, take player back to title screen
27             >| >| get_tree().change_scene_to_file("res://scenes/title_screen.tscn")
```

```
29 ▾ func _on_control_scene_button_pressed() -> void:
30     >| #changes to controls screen
31     >| get_tree().change_scene_to_file("res://scenes/control_screen.tscn")
32
33 ▾ func _on_audio_scene_button_pressed() -> void:
34     >| #changes to audio screen
35     >| get_tree().change_scene_to_file("res://scenes/audio_screen.tscn")
```

spikes.gd

```
1 extends Area2D
2
3 func _on_body_entered(body: Node2D) -> void:
4     Global.respawn_player()
```

titleScreen.gd

```
1 extends Control
2
3 #when start button is pressed
4 func _on_start_button_pressed() -> void:
5     #changes to game scene
6     get_tree().change_scene_to_file("res://scenes/level_selector.tscn")
7
8 #when settings button is pressed
9 func _on_option_button_pressed() -> void:
10    #player is not in playing state (came from title screen, not game scene)
11    Global.playing = false
12    #changes to settings scene
13    get_tree().change_scene_to_file("res://scenes/setting_screen.tscn")
14
15 #when quit button is pressed
16 func _on_quit_button_pressed() -> void:
17    #quits game
18    get_tree().quit()
```

Sources/References

- <https://www.youtube.com/watch?v=oJRMsExy6ac&t=1687s>
- https://www.youtube.com/watch?v=O7iGt8H_FB4&t=658s
- https://www.youtube.com/watch?v=zLdl0TnD_0w&t=403s
- <https://www.youtube.com/watch?v=LOhfqjmasi0>
- <https://www.youtube.com/watch?v=e9-WQg1yMCY&t=4s>
- <https://www.youtube.com/watch?v=JEQR4ALlwVU&t=167s>
-